Sistemi Operativi

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Sistemi Operativi" erogato nell'A.A. 2026 dal professore Gabriele Tolomei per il C.d.L Informatica (33503)*

12 novembre 2026

Indice

1 Introduzione all'hardware dei calcolatori 5

1.1 Lo spettro Hardware-Software 6

1.2 L'architettura di von Neumann (1945) 7

1.2.1 Il Bus di Sistema 9

1.2.2 Il ciclo di istruzione 9

1.2.3 L'Unità Centrale di Elaborazione (CPU) 10

1.2.4 Registri della CPU 11

1.3 Pipelining delle istruzioni 11

1.3.1 Hazard della pipeline 12

1.4 Instruction Set Architectures (ISA) 13

1.5 Processori Multi-Core 13

1.5.1 Prestazioni della CPU: Velocità di Clock e Cicli 13

1.6 La gerarchia di memoria 15

1.6.1 Il principio di località 15

1.6.2 Architettura della Cache Memory 16

1.6.3 Politiche di scrittura della Cache 17

1.6.4 Politiche di mapping della Cache Line 17

1.7 Il sistema Input/Output (I/O) 18

1.7.1 Protocolli di trasferimento I/O 19

2 Introduzione ai Sistemi Operativi 20

2.1 Cos'è un sistema operativo? 20

2.2 I livelli di astrazione forniti dal SO 20

2.3 Processi e Thread 21

2.4 La virtualizzazione della memoria 22

2.5 Perché il Sistema Operativo necessita di supporto hardware 22

2.5.1 Esecuzione in due modalità 22

2.6 Protezione degli accessi in memoria 25
2.6.1 Memory Management Unit (MMU) 26

3 Il linguaggio di programmazione C 27

3.1 Le variabili in C 27
3.1.1 Visibilità delle variabili 28
3.1.2 I tipi di variabili 29
3.2 Gli Array 29
3.3 Le stringhe 30
3.3.1 Funzioni per le stringhe 31
3.4 I puntatori 31
3.4.1 Aritmetica dei puntatori 32
3.5 La gestione della memoria 33
3.5.1 Lo Stack 34
3.5.2 Lo Heap 35
3.5.3 Errori di memoria 36
3.6 La traduzione da sorgente ad eseguibile 36
3.6.1 Il preprocessore 37
3.6.2 La compilazione 37
3.6.3 L'assemblaggio 38
3.6.4 Il linking 38
3.6.5 Librerie statiche e dinamiche 38

4 Processi e API di sistema 39

4.1 L'astrazione dei processi 39
4.2 Gli stati di un processo 40
4.3 Il Process Control Block (PCB) 41
4.4 La gerarchia dei processi 41
4.5 La funzione fork() 42
4.5.1 La filosofia Copy-on-Write 45

4.6 La sincronizzazione dei processi con wait() 46
4.7 La funzione exec() 47
4.8 Altre API di sistema 49

5 Scheduling della CPU 50
5.1 Assunzioni sul carico di lavoro e metriche 50
5.2 I primi passi: FIFO e SJF 52
5.2.1 FIFO - First In, First Out 52
5.2.2 SJF - Shortest Job First 53
5.3 Interruzione dei processi e STCF 55
5.4 Round Robin (RR) 56
5.5 Incorporazione dell'I/O 58
5.6 Algoritmi di scheduling più realistici 59
5.6.1 Multi-Level Feedback Queue (MLFQ) 59
5.6.2 Lottery Scheuduling 63
5.6.3 Stride Scheduling 64
5.6.4 Linux CFS 66

6 La virtualizzazione della memoria 69
6.1 Un primo approccio: l'allocazione statica 70
6.2 Un approccio migliorato: il Base & Bounds 71
6.2.1 Creazione di un processo 73
6.2.2 Terminazione di un processo 74
6.2.3 Cambio di contesto tra processi 74
6.2.4 Spostare un processo da un'area di memoria all'altra 74
6.2.5 La frammentazione: un problema critico 74
6.3 La segmentazione 76
6.3.1 Traduzione degli indirizzi con la segmentazione 77
6.3.2 Doveri del SO con la segmentazione 79
6.3.3 La frammentazione esterna 80

6.4 La paginazione della memoria (paging) 80
6.4.1 Il Translation Lookaside Buffer (TLB) 83
6.4.2 Multi-Level Page Tables 85
6.4.3 Le Page Faults 88
6.5 Lo Swapping 88
6.5.1 Da disco a RAM 89
6.5.2 Politiche di rimpiazzo pagina 90
6.5.3 Criticità su sistemi reali 92

7 La concorrenza **94**
7.1 I thread 94
7.1.1 La libreria `pthread` 96
7.2 La condivisione della memoria 99
7.2.1 I lock 101
7.3 Condition Variables 110
7.3.1 Il problema dei Produttori-Consumatori 112
7.3.2 Semantiche Mesa e Hoare 114
7.3.3 Broadcast 115
7.4 Semafori 115
7.4.1 I semafori come mutex 116
7.4.2 I semafori come ordinamento 117
7.4.3 Semafori come gestori di un pool di risorse 117
7.4.4 Il problema dei filosofi a cena 118
7.5 Il deadlock 120
7.5.1 Prevenzione del deadlock 121

1

Introduzione all'hardware dei calcolatori

Per comprendere appieno il ruolo dei sistemi operativi, è utile richiamare una celebre affermazione del matematico e informatico Richard Hamming:

> *«Lo scopo del calcolo è la comprensione, non i numeri.»* — Richard Hamming

In modo del tutto analogo, possiamo affermare che lo scopo primario di un **Sistema Operativo** è l'**astrazione**, e non la semplice gestione meccanica dell'hardware.

Tuttavia, per poter costruire astrazioni corrette, stabili ed efficaci, è indispensabile padroneggiare in primo luogo il funzionamento della macchina fisica sottostante.

Osservazione 1.0.1: Virtualizzazione dell'hardware

I buoni sistemi operativi si distinguono proprio per la loro capacità di **virtualizzare** con successo i diversi aspetti dell'architettura hardware, fornendo ai programmi applicativi un'interfaccia semplificata e indipendente dai dettagli implementativi del livello fisico.

In questo capitolo introduttivo verranno dunque fornite delle basi riguardanti l'architettura degli elaboratori, in quanto queste conoscenze sono essenziali e necessarie al fine di comprendere *come* e *dove* lavora il Sistema Operativo. In particolare verranno trattate:

1. **L'architettura di von Neumann**: la struttura e il layout generale dell'elaboratore.
2. **La CPU**: l'organizzazione interna comprendente registri, ALU, pipeline e suddivisione in core.

3. **La gerarchia di memoria**: le tecnologie SRAM/DRAM, il principio di località e la gestione delle memorie cache.

4. **Il sistema di Input/Output (I/O)**: il ruolo dei controller, la gestione degli interrupt e il meccanismo DMA (*Direct Memory Access*).

1.1 Lo spettro Hardware-Software

Per comprendere il ruolo di un sistema operativo, è fondamentale analizzare la sua collocazione all'interno della gerarchia che separa il software applicativo dall'hardware sottostante. Ci si chiede quindi: *dove si posiziona esattamente il sistema operativo?*

L'architettura del sistema può essere suddivisa in tre macro-aree di esecuzione: lo **User Space** (spazio utente), il **Kernel Space** (spazio kernel) e la **Physical Machine** (macchina fisica).

Livello dell'Architettura	**Ambito di Esecuzione**
Applicazioni (*Python, Java, C++, Web*)	**User Space**
Librerie di Sistema (*libc*)	**User Space**
Sistema Operativo (*Kernel*)	**Kernel Space**
Firmware / Device Driver	**Hardware / Low-Level Space**
Hardware (*CPU, RAM, Controller I/O*)	**Physical Machine**

Parleremo in modo più approfondito del kernel e delle componenti di un Sistema Operativo più avanti nel corso. Come possiamo però vedere, il sistema operativo agisce come un vero e proprio *ponte* tra il livello logico (le applicazioni in esecuzione) e il livello fisico (gli effettivi componenti hardware).

Le astrazioni software fornite dai sistemi operativi devono necessariamente rispettare i limiti fisici imposti dall'hardware sottostante. La progettazione del software di sistema non può prescindere da questi vincoli, tra i quali spiccano in particolare:

Divario di velocità (*Speed gap*): Le CPU operano a frequenze molto elevate, riuscendo ad eseguire le istruzioni in frazioni di nanosecondo. Al contrario, i tempi di accesso alla memoria principale (DRAM) si misurano nell'ordine delle decine di nanosecondi. Questa marcata differenza prestazionale rende la memoria un frequente collo di bottiglia durante l'esecuzione dei processi.

Compromesso sulla memorizzazione (*Storage trade-off*): Esiste un vincolo fisico ed economico inscindibile tra velocità, capacità e costo della memoria. I dispositivi di storage più veloci (come le cache) sono estremamente costosi e di piccole dimensioni, mentre le memorie più economiche (come i dischi secondari) garantiscono capacità enormi ma a fronte di velocità decisamente inferiori.

Concorrenza (*Concurrency*): L'esecuzione parallela nei reali sistemi multi-core richiede un supporto hardware esplicito e dedicato per la sincronizzazione (ad esempio tramite istruzioni atomiche), indispensabile per evitare race condition e garantire la coerenza dei dati.

Osservazione 1.1.1: Ruolo del Sistema Operativo

Uno degli scopi primari nella progettazione degli algoritmi di un sistema operativo è proprio quello di mascherare ed erodere questi limiti fisici, offrendo ai programmi un'astrazione dell'hardware efficiente, omogenea e trasparente.

1.2 L'architettura di von Neumann (1945)

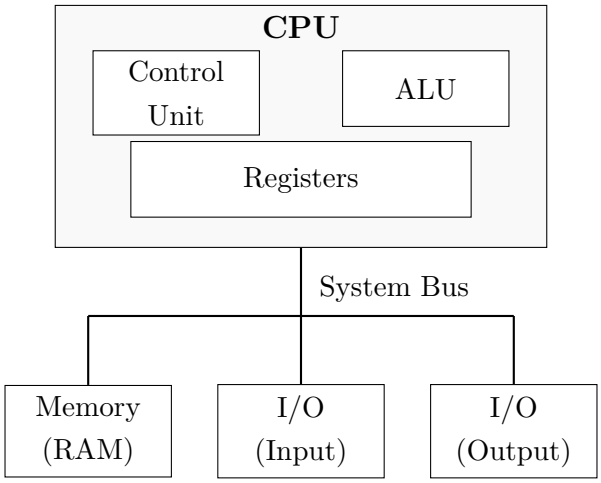
L'architettura di von Neumann rappresenta il fondamento teorico di quasi tutti i moderni calcolatori d'uso generale (*general-purpose*).

Tale modello si basa su due concetti chiave:

Concetto di programma memorizzato (*Stored-Program*): Le istruzioni e i dati condividono lo stesso spazio di memoria fisica (memorizzati in una memoria detta *Memoria Centrale*).

Esecuzione sequenziale: Le istruzioni vengono prelevate ed estratte una alla volta dalla memoria ed eseguite sequenzialmente dalla CPU. Non è previsto alcun tipo di concorrenza da questa architettura.

Di seguito forniamo uno schema grafico rappresentante la struttura del modello:



L'architettura si articola nei seguenti elementi funzionali:

Unità Centrale di Elaborazione (CPU): È il «cervello» del calcolatore, responsabile del coordinamento e dell'esecuzione di tutte le istruzioni. Al suo interno comprende:

- **Control Unit (Unità di Controllo):** Preleva le istruzioni dalla memoria, le decodifica e genera i segnali di controllo necessari al loro completamento.
- **ALU (Arithmetic Logic Unit):** Esegue le operazioni aritmetiche di base e le operazioni logiche elementari sui dati.
- **Registers (Registri):** Celle di memoria ad altissima velocità situate all'interno della CPU, impiegate per contenere temporaneamente dati, indirizzi di memoria o stati di esecuzione. Tra questi registri abbiamo, ad esempio:
  - **Program Counter (PC)**, che si occupa di memorizzare l'indirizzo della prossima istruzione che deve essere eseguita dalla CPU.
  - **Instruction Register (IR)**, che contiene il codice binario dell'effettiva istruzione puntata dal Program Counter.

Bus di Sistema (*System Bus*): È la struttura di collegamento condivisa (costituita da linee elettriche) attraverso la quale la CPU scambia indirizzi, dati e segnali di controllo con la memoria e le periferiche.

Memoria Principale (RAM): Contiene sia il codice dei programmi in esecuzione sia i dati necessari al funzionamento delle applicazioni. Nel modello di von Neumann, istruzioni e dati condividono lo stesso spazio di memoria ed i medesimi bus.

Interfacce di I/O (Input/Output): Moduli necessari a permettere l'interazione tra l'elaboratore e il mondo esterno, distinti in dispositivi di ingresso (*Input*, come tastiera e mouse) e di uscita (*Output*, come monitor e stampanti).

Tuttavia, come visto in precedenza, il principale svantaggio dell'architettura di von Neumann è proprio il fatto che le istruzioni e i dati condividono lo stesso spazio di memoria. Da ciò si origina il cosiddetto *von Neumann Bottleneck*, cioè un effettivo limite fisico in quanto una memoria non può essere acceduta da più dispositivi contemporaneamente. Un'alternativa a questo modello è infatti il cosiddetto **Modello Harvard**, che prevede spazi di memoria e bus fisicamente separati per istruzioni e dati.

1.2.1 Il Bus di Sistema

Entriamo più nel dettaglio riguardo ai bus di sistema, che hanno l'importante compito di traspostare qualsiasi tipo di dato tra una componente e l'altra.

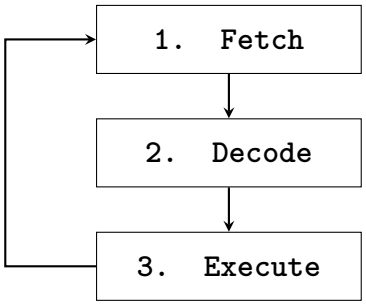
Bus Indirizzi (*Address Bus*): trasporta gli indirizzi di memoria dalla CPU verso i componenti di destinazione. Questo bus è **unidirezionale**, in quanto solo la CPU deve poter inviare l'indirizzo di memoria su cui lavorare. Inoltre, l'Address Bus ha una proprietà detta **Bus Width** (che fisicamente rappresenta quanti fili elettrici ci sono per quel bus) e che determina qual è il massimo numero di locazioni indirizzabili. La formula è molto semplice: se abbiamo width $= N$, il numero di locazioni indirizzabili è pari a 2^N.

Bus Dati (*Data Bus*): gestisce il trasferimento dei byte di dati effettivi tra i vari componenti. Rappresenta il canale sul quale viaggia fisicamente l'istruzione da eseguire oppure il dato da elaborare. Poiché i dati devono poter essere scritti e letti, la trasmissione lungo il Data Bus è di tipo *bidirezionale*. Anche questo bus possiede la proprietà di **Bus Width**, con significato analogo a quella dell'Address Bus, e questo valore in genere coincide con la Word Size del processore (dunque più è ampio il bus, maggiore è la quantità di dati che possono essere trasmessi simultaneamente).

Bus di Controllo (*Control Bus*): veicola i segnali di comando e di sincronizzazione del sistema, come le operazioni di lettura (*read*), scrittura (*write*), segnali di *reset* o richieste di *interrupt*. In sostanza, è il mezzo tramite cui la Control Unit (CU) definisce cosa fare con il dato e l'indirizzo selezionati.

1.2.2 Il ciclo di istruzione

Il **ciclo di istruzione** (o *fetch-decode-execute cycle*) rappresenta il processo fondamentale alla base del funzionamento di qualsiasi elaboratore. Ogni programma esegue questo ciclo in maniera continua.



Il ciclo si compone di tre fasi sequenziali gestite direttamente dalla CPU:

1. **Fetch (Prelevamento):** il contenuto del PC viene trasferito sull'Address Bus per poi prelevare l'effettiva istruzione puntata da quell'indirizzo. Alla fine di questa fase il valore del PC viene incrementato in modo di far passare all'istruzione successiva.
2. **Decode (Decodifica):** il processore analizza e interpreta il significato della sequenza di bit che compone l'istruzione. Questa operazione viene svolta in base alla specifica codifica scelta dall'architettura dell'elaboratore (ossia l'*Instruction Set Architecture* o ISA).
3. **Execute (Esecuzione):** viene svolta l'operazione richiesta dall'istruzione. Questa fase può comportare un calcolo eseguito dall'Unità Aritmetico-Logica (ALU) oppure un'operazione di lettura/scrittura in memoria.

Al termine della fase di esecuzione, il ciclo riparte immediatamente dalla fase di *fetch* per l'istruzione successiva.

1.2.3 L'Unità Centrale di Elaborazione (CPU)

La **Central Processing Unit** (**CPU**) rappresenta il "cervello" del calcolatore, essendo l'organo fondamentale preposto all'esecuzione delle istruzioni memorizzate in memoria.

L'architettura interna della CPU si articola su tre elementi costruttivi principali:

1. **Datapath (Unità Dati):** include l'**Arithmetic Logic Unit** (**ALU**), preposti alla manipolazione e al calcolo effettivo sui dati e i registri General-Purpose (adibiti al contenimento di risultati intermedi per le operazioni della ALU).
2. **Control Unit** (**CU**): opera come coordinatore dell'intero microprocessore. Il suo compito è prelevare le istruzioni dalla memoria (*fetch*), decodificarle e pilotare opportunamente il datapath per consentirne l'esecuzione.
3. **Bus Interno:** insieme di canali e linee di interconnessione interne che mettono in comunicazione diretta i registri, l'ALU e la Control Unit.

1.2.4 Registri della CPU

Vediamo ora una carrellata veloce dei registri di sistema e delle loro funzionalità:

1. **Registri visibili all'utente (General-Purpose):** sono direttamente indirizzabili dalle istruzioni in linguaggio assembly per la manipolazione dei dati e l'esecuzione delle operazioni.
2. **Registri di controllo e di stato (Special-Purpose):** sono utilizzati dalla *Control Unit* (CU) per gestire lo stato di esecuzione del sistema e del processore. Tra i più rilevanti rientrano:
   - **Program Counter (PC):** mantiene l'indirizzo della prossima istruzione da eseguire.
   - **Instruction Register (IR):** contiene l'istruzione attualmente in fase di decodifica o esecuzione.
   - **Processor Status Register (PSR):** memorizza le informazioni sullo stato corrente del processore (flag di condizione, livello di privilegio, ecc.).
   - **Stack Pointer (SP):** contiene l'indirizzo di memoria della cima dello stack delle chiamate (ad esempio di chiamate delle funzioni).
   - **Memory Address Register (MAR):** contiene l'indirizzo che sta per essere utilizzato dalla RAM. E' direttamente collegato all'Address Bus.
   - **Memory Data Register (MDR):** contiene i byte di dati letti dalla RAM, ed è direttamente collegato al Data Bus.

1.3 Pipelining delle istruzioni

Nei processori più semplici, il ciclo di esecuzione delle istruzioni segue un approccio strettamente sequenziale. Come previsto dal modello classico dell'architettura di Von Neumann, il processo avviene in modo sequenziale: la CPU deve completare interamente le fasi di prelievo (*fetch*), decodifica (*decode*) ed esecuzione (*execute*) di una singola istruzione prima di poter passare a quella successiva. Questo schema, pur essendo semplice, comporta un notevole spreco di risorse hardware, poiché molte unità funzionali della CPU rimangono inattive durante le fasi in cui non sono direttamente coinvolte.

Per superare questa limitazione e migliorare le prestazioni del sistema, si introduce una tecnica di ottimizzazione fondamentale:

Definizione 1.3.1: Pipelining

Il **pipelining** è una tecnica di ottimizzazione hardware che consente di sovrapporre l'esecuzione di più istruzioni nel tempo, parallelizzando le diverse fasi del loro ciclo di vita.

Tale tecnica, almeno per le architetture di tipo RISC, si articola in 5 stadi:

1. **IF (Instruction Fetch):** prelievo dell'istruzione dalla cache di primo livello (L1 Cache) o dalla memoria principale (RAM).
2. **ID (Instruction Decode):** decodifica dell'istruzione appena prelevata e lettura dei registri sorgente.
3. **EX (Execute):** esecuzione del calcolo tramite l'unità aritmetico-logica (ALU) o calcolo dell'indirizzo di memoria.
4. **MEM (Memory Access):** accesso alla memoria dati, che comporta un'operazione di lettura o scrittura verso la cache dati.
5. **WB (Writeback):** scrittura del risultato finale all'interno del registro di destinazione.

Grazie a questa suddivisione, ogni stadio opera su un'istruzione differente in modo simultaneo durante ciascun ciclo di clock, consentendo un'elevata sovrapposizione delle operazioni e migliorando l'efficienza del processore.

1.3.1 Hazard della pipeline

In condizioni ideali, le architetture con pipeline consentono di raggiungere il massimo incremento prestazionale. Tuttavia, nella realtà possono verificarsi delle situazioni anomale note come **hazard** (o criticità), che impediscono all'istruzione successiva di essere eseguita nel ciclo di clock prestabilito.

Tali criticità si dividono principalmente in tre grandi categorie:

Hazard strutturali (*Structural Hazards*): Si verificano in presenza di un conflitto di risorse, ovvero quando due o più stadi della pipeline richiedono contemporaneamente l'accesso allo stesso componente hardware fisico.

Hazard sui dati (*Data Hazards*): Si manifestano quando un'istruzione dipende strettamente dal risultato di un'istruzione precedente che non ha ancora completato il suo percorso all'interno della pipeline.

Hazard di controllo (*Control Hazards*): Sono causati dalle istruzioni di salto condizionato o incondizionato (*branch*), le quali modificano il valore del *Program Counter* (PC). Ciò rende invalide le istruzioni che sono già state caricate (*fetched*) preventivamente nella pipeline.

Per risolvere tali criticità, le CPU moderne impiegano dei predittori di salto (**branch predictors**) per indovinare l'esito di un'istruzione di salto prima che essa venga effettivamente completata. Tuttavia, tali metodologie non verranno trattate.

1.4 Instruction Set Architectures (ISA)

Nel mondo dell'informatica sono stabiliti due standard principali riguardo al linguaggio adottato dalle istruzioni macchina:

CISC (Complex Instruction Set Computer): ogni istruzione macchina può comprenderne altre (sono infatti a lunghezza variabile), in modo da "semplificare" la scrittura del software per gli utenti.

RISC (Reduced Instruction Set Computer): mantiene solo le istruzioni principali del codice macchina (tutte hanno la stessa lunghezza).

1.5 Processori Multi-Core

Gli elaboratori di oggi aggirano i limiti di potenza e velocità dei processori collocando più processori (detti appunto *core*) su un singolo macro-processore. La caratteristica principale è proprio che ogni *core* è come se fosse "autonomo", in quanto possiede i suoi registri e le sue unità di controllo. A questo punto deve essere il Sistema Operativo a saper gestire l'architettura Multi-Core: deve saper far comunicare efficientemente i vari core per la condivisione di risorse senza che si verifichino problemi di sincronizzazione delle variabili. Discuteremo di queste tecniche quando parleremo di *concorrenza* e *multi-threading*.

1.5.1 Prestazioni della CPU: Velocità di Clock e Cicli

Il funzionamento della CPU è sincronizzato da un **clock di sistema** (*SystemClock*) oscillante, che scandisce il ritmo delle operazioni eseguite dal processore. Per comprendere le prestazioni di una CPU, è fondamentale analizzare due grandezze fondamentali legate a questo segnale:

- **Periodo di clock (T):** rappresenta la durata temporale di un singolo ciclo di clock (ad esempio 0.25 nanosecondi).

- **Frequenza di clock (f):** rappresenta il numero di cicli di clock al secondo, misurata in Hertz (Hz).

Queste due grandezze sono inversamente proporzionali tra loro e legate dalla relazione:

$$f = \frac{1}{T}$$

A titolo d'esempio, una CPU a 4.0 GHz è in grado di completare 4×10^9 cicli di clock in un solo secondo.

Tuttavia, non tutte le istruzioni macchina richiedono lo stesso tempo per essere eseguite: le istruzioni più semplici possono essere completate in un solo ciclo di clock, mentre le operazioni più complesse (come la divisione tra numeri in virgola mobile o gli accessi alla memoria principale) richiedono un numero elevato di cicli per poter essere portate a termine.

Per valutare e confrontare l'efficienza dei processori, risulta fondamentale definire delle metriche standard. Una delle domande chiave nell'architettura dei calcolatori è: quante istruzioni riesce ad eseguire una CPU per ogni ciclo di clock? Per rispondere a questo interrogativo, introduciamo due parametri fondamentali:

- **CPI (Cycles Per Instruction):** rappresenta il numero medio di cicli di clock necessari per eseguire una singola istruzione.
- **IPC (Instructions Per Cycle):** rappresenta l'inverso del CPI, ovvero il numero di istruzioni eseguite in un singolo ciclo di clock. La relazione matematica è la seguente:

$$\text{IPC} = \frac{1}{\text{CPI}}$$

Per calcolare il tempo totale necessario per eseguire un determinato programma, si utilizza la formula fondamentale delle prestazioni della CPU, che mette in relazione il numero di istruzioni, l'efficienza architetturale e la frequenza di clock:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Dove con *CPU Time* si intende il tempo totale necessario per eseguire tutte le istruzioni del programma. Analizzando i singoli componenti della formula:

- **Instruction Count:** il numero totale di istruzioni eseguite dal programma.
- **CPI:** il numero medio di cicli per istruzione.
- **Clock Cycle Time:** la durata di un singolo ciclo di clock (l'inverso della frequenza di clock).

1.6 La gerarchia di memoria

La velocità dei processori, nel corso degli ultimi anni, è cresciuta molto più velocemente (si potrebbe dire in maniera quasi esponenziale) rispetto alla velocità di accesso alle memorie RAM. Per continuare a velocizzare l'accesso alle risorse, gli architetti dei computer fanno riferimento ad una *gerarchia delle memorie*, una struttura piramidale che organizza le memorie dall'alto verso il basso, dove quella più in **cima** sono i **registri**, memorie *estremamente veloci*, situate all'interno della CPU, ma altrettanto costose; alla base di questa piramide si collocano i dispositivi di archiviazione di massa, come *SSD* e *HDD*, che sono più economiche ma decisamente più lente. Dunque, il compito di un bravo architetto di computer è quello di trovare il giusto compromesso a livello di prezzo e velocità, e il compito di un buon sistema operativo è quello di saper sfruttare queste risorse nel modo più efficiente possibile.

Possiamo riassumere il comportamento della struttura con una regola fondamentale: **scendendo nella gerarchia, la velocità di accesso diminuisce, la capacità aumenta e il costo per byte diminuisce**.

Livello	**Componente**	**Capacità tipica**	**Latenza di accesso**
L0	CPU Registers	< 1 KiB	< 0.5 ns
L1	L1 Cache (per core)	$32 - 64$ KiB	$1 - 2$ ns
L2	L2 Cache (per core)	$256 - 512$ KiB	$3 - 5$ ns
L3	L3 Cache (shared)	$8 - 64$ MiB	$10 - 20$ ns
L4	Main Memory (DRAM)	$8 - 64$ GiB	$60 - 100$ ns
L5	SSD (NAND Flash)	256 GiB - 2 TiB	$10 - 100$ μs
L6	HDD (Magnetic Disk)	$1 - 10$ TiB	$5 - 10$ ms

La memoria principale è strutturata come una griglia monodimensionale di indirizzi di byte (*byte addresses*). Ogni cella possiede un indirizzo univoco a cui corrisponde un determinato contenuto memorizzato.

1.6.1 Il principio di località

L'efficacia della gerarchia di memoria (*memory hierarchy*) si basa sul "principio di località", che definisce una serie di principi secondo i quali lavora la CPU per ottimizzare l'accesso

alle informazioni. I programmi in esecuzione non accedono infatti alla memoria in modo casuale, ma presentano due forme fondamentali di località:

Località temporale (*Temporal Locality*): se una determinata locazione di memoria viene acceduta una volta, è molto probabile che la stessa locazione venga acceduta nuovamente nel futuro prossimo. Ad esempio, in un ciclo `for` in cui si accede alla variabile `i` per ogni iterazione del ciclo, la CPU dovrà accedere sempre alle stesse variabili e dunque terrà memorizzato in qualche registro vicino alla CPU l'indirizzo di memoria di quelle variabili.

Località spaziale (*Spatial Locality*): se viene acceduta una specifica locazione di memoria, è molto probabile che anche le locazioni ad essa vicine o adiacenti vengano accedute nel prossimo futuro. Tenendo sempre l'esempio del ciclo `for`, vedendo che tali locazioni di memoria vengono usate spesso, la CPU tende a memorizzare quei valori nei registri più vicini ad essa, in modo da renderne l'accesso quasi immediato.

1.6.2 Architettura della Cache Memory

Le memorie cache non gestiscono i dati a livello di singoli byte, ma operano su blocchi di dimensione fissa denominati **cache line** (la cui dimensione tipica è di 64 byte).

Ogni entry all'interno di una cache line è composta da tre elementi fondamentali:

Valid Bit: Un bit di controllo che indica se i dati presenti nella linea di cache sono validi ed aggiornati. Questo flag permette di verificare se il dato cercato è effettivamente presente ed utilizzabile all'interno della cache.

Tag Bit: Identifica l'indirizzo di memoria originale (in RAM) da cui sono stati reperiti i dati. Agisce in modo analogo alla chiave di un dizionario: permette di stabilire in modo univoco a quale posizione della memoria principale appartenga il blocco.

Data Block: Contiene l'effettivo blocco di dati copiato dalla memoria RAM (tipicamente 64 byte), ovvero il contenuto informativo richiesto situato a quell'indirizzo.

Nel momento in cui la CPU richiede un determinato indirizzo di memoria (*memory address*), si possono verificare due situazioni distinte:

Cache Hit (Dato trovato) Si ha un *cache hit* quando la CPU individua con successo l'indirizzo e il dato cercato direttamente all'interno della memoria cache.

Cache Miss (Dato non trovato) Si ha un *cache miss* quando la CPU non trova l'indirizzo richiesto all'interno della memoria cache, rendendo necessario il recupero del dato da memorie più lente.

1.6.3 Politiche di scrittura della Cache

Nel momento in cui la CPU scrive i dati in un cache block, possono essere applicate due politiche per mantenere aggiornata la RAM:

Write-Through: la CPU scrive contemporaneamente sia nella cache che nella memoria centrale. Il vantaggio è ovviamente una sincronizzazione costante dei dati; lo svantaggio è che una semplice operazione di scrittura di un dato sarà sempre lenta in quanto si dovrà riportare anche in memoria centrale, e dunque elimina il vantaggio di utilizzo della cache.

Write-Back: la CPU scrive solo nella cache line e la marca come "modificata". L'aggiornamento effettivo nella memoria centrale si verifica solo quando tale cache line viene espulsa dalla cache per fare spazio ad altri dati. Vantaggioso in quanto viene mantenuta la velocità di memorizzazione di dati all'interno della cache, ma rimane il problema possibile della sincronizzazione.

1.6.4 Politiche di mapping della Cache Line

Una questione fondamentale nell'organizzazione delle architetture dei calcolatori riguarda il criterio secondo cui un blocco della memoria principale (*memory block*) viene posizionato all'interno di uno slot della memoria cache (*cache slot*).

Per gestire questa mappatura ed individuare la posizione specifica in cui collocare i dati, si adottano principalmente tre schemi di progettazione:

Direct Mapped (Mappatura Diretta): Ogni blocco di memoria principale viene mappato in modo univoco ed esatto in un solo ed unico *cache slot*. Vantaggioso in quanto siamo sicuri di dove ritrovare i blocchi di memoria della cache. Possono tuttavia verificarsi conflitti di indirizzi in caso di mappatura nella stessa locazione.

Fully Associative (Completamente Associativa): Ogni blocco di memoria principale può essere collocato in un qualsiasi *cache slot* libero all'interno della cache. Tale approccio elimina i problemi di conflitto di mappatura, ma potendo posizionare un blocco in qualsiasi punto, dovremo cercare in qualsiasi punto possibile.

Set Associative (Associativa a Insiemi): Rappresenta un compromesso tra le due politiche precedenti. La memoria cache viene suddivisa in un determinato numero di insiemi (*set*), ciascuno composto da un numero fisso di slot. Un blocco di memoria viene mappato in modo rigido a uno specifico *set*, ma all'interno di quel set può risiedere in un qualsiasi slot libero.

1.7 Il sistema Input/Output (I/O)

Affinché un calcolatore sia effettivamente utile ed utilizzabile, esso deve necessariamente comunicare con l'ambiente esterno mediante opportuni dispositivi. È importante sottolineare che i dispositivi di Input/Output non si limitano unicamente alle periferiche di interazione diretta con l'utente (come quelle adibite all'inserimento di testo o alla visualizzazione grafica), ma comprendono un'ampia varietà di categorie:

- **Dispositivi di memorizzazione secondaria (Storage Devices):** memorie di massa persistenti quali SSD, HDD e unità USB Flash.
- **Dispositivi di rete (Network Devices):** schede di rete (NIC), sia cablate (Ethernet) che wireless (Wi-Fi), per la trasmissione e ricezione di dati attraverso la rete.
- **Interfacce utente (User Interfaces):** periferiche standard di interazione uomo-macchina, tra cui tastiera, mouse e schermo.

Tutti i trasferimenti di dati da CPU a dispositivi di I/O sono gestiti da appositi sistemi hardware o software specializzati nel controllo di quel specifico dispositivo o di quella specifica categoria di dispositivi.

Device Controller (Hardware) Consiste in un effettivo microchip integrato sulla scheda madre o alloggiato su una scheda di espansione che controlla direttamente il dispositivo fisico.

Device Driver (Software) Si tratta di un modulo del sistema operativo che conosce la sintassi e la logica per comunicare correttamente con il dispositivo di I/O.

Una questione fondamentale nella gestione delle periferiche riguarda il meccanismo con cui il processore interagisce con l'hardware: *come legge e scrive la CPU nei registri del device controller?*

Per rispondere a questa domanda si distinguono principalmente due approcci architetturali:

1. **Port-Mapped I/O (PMIO)**: Prevede l'utilizzo di **istruzioni CPU dedicate** (come ad esempio le istruzioni `IN` e `OUT` nelle architetture x86). Questo approccio nasce dal fatto che, a livello di *control bus*, le consuete operazioni di `LOAD` e `STORE` possono essere utilizzate solo se l'indirizzo di destinazione fa riferimento a una locazione di memoria principale. Si rende quindi necessaria la definizione di uno spazio di indirizzamento separato e di istruzioni specifiche riservate alla comunicazione con i dispositivi di I/O.

2. **Memory-Mapped I/O (MMIO)**: Rappresenta la soluzione standard e moderna, che consiste nella predisposizione di uno spazio di memoria e dei registri interamente dedicati a questi dispositivi.
   Un controller espone tipicamente tre classi fondamentali di registri:

   - **Status register**: registro di sola lettura, riporta lo stato corrente del dispositivo (BUSY, READY, ...).
   - **Command register**: sola scrittura, indica al dispositivo *cosa* fare.
   - **Data register**: lettura e scrittura, contiene i dati inviati/ricevuti al/dal dispositivo.

1.7.1 Protocolli di trasferimento I/O

Una delle problematiche fondamentali dell'architettura di un elaboratore riguarda la modalità con cui la CPU coordina e gestisce il trasferimento dei dati da e verso un dispositivo di Input/Output. Per rispondere a questa esigenza, si adottano principalmente tre protocolli:

1. **Programmed I/O (Polling)**: la CPU controlla ciclicamente lo stato del dispositivo per verificare se è pronto a trasferire i dati. Tale approccio può avere senso ed essere efficiente quando si devono trasferire pochissimi dati, in quanto evita l'overhead legato alla gestione delle interruzioni.

2. **Interrupt-Driven I/O**: come già accennato durante l'analisi del *control bus*, in questo schema è il dispositivo a notificare alla CPU l'avvenuto completamento di un'operazione tramite un segnale di interruzione. Risulta una modalità nettamente meno onerosa rispetto al polling quando si devono trasmettere quantitativi di dati più consistenti. In particolare, appena la CPU riceve questo segnale di interruzione, effettua una "fotografia" dello stato corrente dei registri e passa all'esecuzione di una particolare *routine* (**Interrupt Service Routine (ISR)**). Nello specifico, tutte le routine sono memorizzate all'interno di una struttura dati detta **Interrupt Vector Table** a cui la CPU ha accesso e che contiene i puntatori ad ogni specifica ISR.

3. **Direct Memory Access (DMA)**: un modulo hardware dedicato si occupa di trasferire i dati direttamente tra la memoria principale e il dispositivo di I/O, senza che la CPU debba intervenire per ogni singolo byte, riducendo drasticamente il carico di lavoro del processore.

2

Introduzione ai Sistemi Operativi

2.1 Cos'è un sistema operativo?

Definizione 2.1.1: Sistema Operativo (OS)

Un **sistema operativo** (Operating System, OS) è una collezione di software di sistema che gestisce le risorse hardware e fornisce servizi comuni per i programmi applicativi.

Il nucleo del sistema operativo è il primo software che si avvia all'avvio del computer.

2.2 I livelli di astrazione forniti dal SO

Il sistema operativo abbiamo capito essere come una sorta di "autorità" che gestisce tutte le risorse hardware e software per permettere all'utente di utilizzare la macchina nel modo più semplice e intuitivo possibile, avendo più o meno controllo sulle risorse (a seconda del sistema operativo che vogliamo implementare).

In generale, un buon sistema operativo deve:

Fornire l'astrazione adeguata , ovvero semplificare l'esecuzione di alcune operazioni, come ad esempio l'accesso al filesystem mediante una struttura organizzata in file e cartelle. Chiaramente questa struttura è puramente logica: se non venisse implementato questo tipo di astrazione, saremmo costretti ad accedere ai nostri file indicando il disco rigido e il settore in cui questo è contenuto.

Gestire in modo efficiente le risorse , decidendo quante e quali risorse allocare per ciascun processo in esecuzione alla CPU, e impedire che un processo prevalga sugli altri (*starvation*).

Controllare l'esecuzione delle applicazioni , impedendo ad esempio che un processo acceda (involontariamente) allo spazio di memoria riservato ad un altro processo, oppure impedendo che un programma generante un errore mandi in crash l'intero sistema.

Gestire l'esecuzione concorrente di più processi , suddividendo ogni processo in *thread* differenti (letteralmente dei sotto-programmi in esecuzione) e dando l'illusione che esistano contemporaneamente più CPU differenti che eseguono ognuna un processo (la magia del **timesharing**, di cui si parlerà più avanti).

2.3 Processi e Thread

Nella scorsa sezione abbiamo spesso citato i processi, senza mai darne una specifica definizione. La lasciamo di seguito.

Definizione 2.3.1: Processo

Un **processo** è un'**istanza di un programma in esecuzione**. Quando noi scriviamo un codice python, stiamo scrivendo un *programma*, qualcosa che può essere eseguito (ma possiamo pensare analogamente con tutti i software che installiamo da internet).
Una volta che manderemo in esecuzione tale codice o tale programma, per la CPU quello diventerà un processo.

Sempre come anticipato prima, diamo anche una definizione di Thread.

Definizione 2.3.2: Thread

Un **thread** è la più piccola unità di esecuzione indirizzabile dal sistema operativo, e consiste sostanzialmente in un *flusso di esecuzione* di un singolo processo. Detto con parole più semplici, possiamo suddividere un singolo processo in più thread, ognuno che si occupa di eseguire una parte di quel processo.

Da questa definizione si evincono due importanti caratteristiche dei thread:

- Possono essere creati **più thread diversi** relativi allo stesso processo.
- Più thread dello stesso processo condividono lo **stesso spazio di memorie per codice e variabili**.
- Dato che ogni thread esegue una *porzione diversa* dello stesso processo, ovviamente non possono essere condivisi registri come il Program Counter e i registri di calcolo.

2.4 La virtualizzazione della memoria

Fisicamente, la memoria centrale RAM è una griglia di celle in cui, per ogni cella, possiamo memorizzare un dato di una certa lunghezza (tipicamente equivalente alla *lunghezza di word* dell'architettura). Nell'esecuzione dei programmi, ogni programma può memorizzare le proprie variabili in memoria, tuttavia il SO si occupa di **virtualizzare** tale processo, assegnando ad ogni processo un indirizzo di memoria "finto", che verrà poi mappato ad un indirizzo reale in RAM. Ad esempio:

- L'indirizzo viruale `0x1000` nel Processo A viene mappato all'indirizzo fisico `0x7fff000`.
- Lo stesso indirizzo logico `0x1000` nel Processo B può essere invece tranquillamente mappato all'indirizzo fisico `0x8aaa000`.

A garantire che due indirizzi virtuali uguali non mappino allo stesso indirizzo fisico in memoria ci pensano il SO e la **Memory Management Unit (MMU)**.

2.5 Perché il Sistema Operativo necessita di supporto hardware

Il Sistema Operativo è, in fin dei conti, semplicemente un software. Se venisse eseguito sulla CPU esattamente allo stesso modo di un'applicazione utente, nulla impedirebbe a un programma utente di:

- Scrivere codice che disabiliti la protezione della memoria del Sistema Operativo;
- Leggere file personali appartenenti a un altro utente;
- Disabilitare gli interrupt e monopolizzare la CPU a tempo indeterminato.

Risposta? Nulla. Di fatto, se un sistema operativo non sfruttasse a pieno le risorse hardware, un utente potrebbe tranquillamente fare tutte queste cose.

2.5.1 Esecuzione in due modalità

Da questo problema nasce dunque la necessità di creare una barriera logica su cosa può e cosa non può fare un programma utente. Nasce così la **Dual-Mode Execution**, che definisce una gamma di istruzioni riservate all'esecuzione solo da parte del Kernel del sistema operativo.

Come viene implementata questa problematica? Definendo uno speciale bit, detto **Mode Bit**, memorizzate all'interno di uno speciale registro della CPU (tipicamente il PSR, Program Status Register).

- Se tale bit assume valore 0, la CPU eseguirà l'istruzione in **Kernel Mode** (tutti i privilegi). Questa modalità in alcune notazioni viene anche detta **Ring 0**.
- Altrimenti, con bit valore 1, l'istruzione verrà eseguita in **User Mode**, dunque con privilegi limitati. Questa modalità in alcune notazioni viene anche detta **Ring 3**.

Osservazione 2.5.1: E' lo stesso dell'"Esegui come amministratore"?

Anche se si potrebbe pensare ad un'analogia con quando ci viene chiesta l'autorizzazione dell'amministratore del nostro computer per eseguire un programma, per tali situazioni ci troviamo già ad un livello di astrazione più alto, dove ci serve la conferma di **un altro utente** e **non** del *sistema operativo*.

In generale, la transizione Kernel Mode -> User Mode si articola secondo i seguenti passaggi:

1. Viene memorizzato lo stato attuale dei processi e dei loro registri.
2. Viene effettuata la transizione a Kernel Mode.
3. Viene eseguita l'istruzione di sistema apposita.
4. All'interno della codifica di ogni istruzione di sistema è presente anche un blocco di istruzioni che si occupa di **ripristinare lo stato precedente alla kernel mode**, **mettere ad 1 il Mode Bit** e **ritornare alla normale esecuzione del processo**.

Alcuni esempi di istruzioni riservate alla Kernel Mode sono ad esempio `HLT`, che "spegne" la CPU, oppure `CLI/STI` che disabilita l'invio dei segnali di interrupt da parte di dispositivi hardware esterni. Ma anche banalmente l'accesso ai file e alle directories è un'operazione da Kernel Mode. Ci chiediamo dunque: come è possibile che noi riusciamo a fare queste operazioni in modo così semplice, nonostante siano formalmente riservate al sistema?

Ciò avviene principalmente secondo tre modalità differenti.

2.5.1.1 Chiamate di sistema

Il sistema operativo offre una lista di funzionalità Kernel Mode a cui l'utente può fare richiesta per eseguire una determinata operazione. Prendendo il precedente esempio dell'accesso al filesystem, quando clicchiamo sull'icon dell'`Esplora file`, implicitamente stiamo effettuando una chiamata al sistema operativo per permetterci di accedere ai file sul disco (quelli a cui l'utente è autorizzato, chiaramente). Come già detto, sta al sistema operativo fornire queste funzionalità, e dunque ci potrebbero essere sistemi operativi più o meno permissivi in tal senso.

Nell'ottica delle chiamate di sistema, ogni sistema operativo definisce una array detto **System Call Translation Table**, dove ogni elemento consiste in un puntatore ad una specifica funzionalità di sistema.

Gli argomenti di una syscall (pensiamo all'esempio di prima dell'accesso ad un file mediante una funzione `open`: dobbiamo chiaramente passare il percorso del file come argomento) possono essere memorizzati mediante:

- Registri, in caso di argomenti molto piccoli.
- Un stack di memoria esclusivamente adibito a questa funzionalità.
- Lo stack dell'utente.

Per quanto utili e potenti, le chiamate di sistema sono ouno strumento estremamente oneroso da implementare, in quanto richiedono un salvataggio e ripristino dello stato dei registri per ogni chiamata. Ecco perché molti linguaggi di programmazione ottimizzano multiple chiamate alla stessa funzione accorpandole, quando possibile, in una sola.

Osservazione 2.5.2: Esempi di syscalls

Funzioni come il `printf` nel linguaggio C, oppure il `System.out.println` in Java di per sé non sono delle vere e proprie syscalls. Tuttavia, se andassimo a leggere il codice mediante le quali sono state implementate queste funzionalità, ci accorgeremo di come esse facciano a capo ad una opportuna funzionalità di sistema che permette di mandare a schermo una serie di caratteri.

2.5.1.2 Eccezioni

Sono gli errori che ci vengono lanciati da un programma e che ne causano il crash. Anziché mandare in crash tutta l'applicazione, il sistema operativo deve gestire tali anormalità per impedire l'interruzione del lavoro di tutta la macchina. Sono comportamenti prevedibili e dunque **sincroni**, in quanto provengono da determinate istruzioni di un programma. Per fare un esempio banale, se il compilatore vede che è presente un'operazione di casting di tipo, è molto probabile che se il tipo passato non sia conforme quest'operazione lanci un'errore. Dunque, per quanto spesso siano imprevedibili, il sistema operativo e più in particolare i compilatori dei linguaggi possono attuare delle strategie per "prevedere" tali comportamenti. Tuttavia non tutti i tipi di eccezioni sono correttibili: alcune, dette **aborts**, causano errori hardware o di sistema irrecuperabili.

2.5.1.3 Interruzioni hardware

Come visto precedentemente, si verificano quando un dispositivo esterno di I/O interrompe il processo che stava eseguendo, e dunque invia un segnale di `INTERRUPT` al sistema operativo. Questo tipo di modalità è **asincrona**, in quanto non si potrà prevedere in alcun modo quando un dispositivo esterno abbia completato la propria operazione (es: quando il pacchetto è arrivato a destinazione).

La gestione da parte della CPU degli eventi asincroni legati ad interruzioni ed eccezioni avviene mediante un'array chiamato **Interrupt Descriptor Table (IDT)**. Ogni evento asincrono viene è marcato dal sistema operativo con un numero da 0 a 255. Questa marcatura stabilisce una corrispondenza biunivoca tra l'evento in sé e l'indice della IDT in cui si trovano le istruzioni necessarie a processare quell'evento.

A volte potrebbe accadere che il Kernel abbia bisogno di eseguire codice senza venire interrotto. Ecco perché la CPU mette a disposizione istruzioni come `CLI` per ignorare i flag di interrupt e `STI` per riabilitarne la lettura. Tuttavia non tutti gli interrupt possono essere ignorati: da qui la divisione tra **Maskable Interrupts** (ignorabili) e **Non-Maskable Interrupts** (critici, non ignorabili).

2.5.1.4 Timer di interruzione

Un esempio classico di interruzione che non può in alcun modo essere ignorata riguarda il **Timer Interrupt**.
Come suggerisce il nome, si tratta di un effettivo dispositivo hardware che interrompe periodicamente la CPU in modo da impedire l'esecuzione infinita o prioritaria di alcuni processi. Senza tale dispositivo, un programma contenente ad esempio `while (true) ...` verrebbe eseguito all'infinito. Per fortuna non è così, e grazie a questo timer di interruzione alcuni programmi esauriscono il proprio "tempo a disposizione" e vengono definitivamente spenti.

2.6 Protezione degli accessi in memoria

Il sistema operativo ha anche il compito di garantire che ogni processo esegua le proprie istruzioni e memorizzi le proprie variabili all'interno di uno spazio di memoria ben definito. Oltre a garantire che non si vadano ad utilizzare aree di memoria riservate ad altri processi, è fondamentale impedire che un processo possa accedere ad aree di memoria riservate al Kernel.

Questa richiesta viene semplicemente risolta impostando degli effettivi limiti per gli indirizzi di memoria accessibili da un singolo processo.
Ad esempio, supponiamo di avere tre attori in gioco:

- Spazio di memoria kernel, che va da `0000` a `0999`
- Spazio di memoria Processo A, che va da `1000` a `1499`.
- Spazio di memoria processo B, che va da `1500` a `1999`.

Nel momento in cui il Processo A genererà un indirizzo in cui memorizzare una variabile o in cui accedere ad una variabile, la CPU dovrà fare un semplice controllo: se tale indirizzo non risiede nel limite di indirizzi riservati al Processo A, quell'operazione va bloccata.

2.6.1 Memory Management Unit (MMU)

La Memory Management Unit (MMU) è quella **componente hardware che si occupa di tradurre gli indirizzi virtuali assegnati ai processi in indirizzi fisici nella memoria centrale**. Sostanzialmente, è tutto grazie alla MMU se i nostri processi riescono a coesistere senza intaccare l'uno nell'altro. La MMU è il componente hardware, che però si occupa di leggere le varie corrispondenze contenute all'interno di una particolare struttura chiamata **Page Table**. Ogni valore della page table (**Page Table Entry (PTE)**) contiene vari campi, tra cui l'indirizzo di memoria della singola entrata, i permessi di accesso (lettura/scrittura), il livello di autorizzazione (kernel mode/user mode).

Osservazione 2.6.1: Come funziona con il DMA?

Precedentemente avevamo introdotto il DMA, ovvero quel dispositivo hardware che si occupa di gestire tutti i segnali di interruzione relativi ai dispositivi di I/O. La domanda sorge spontanea: dove sono memorizzati i registri di questo controller? L'utente avrebbe modo di accedervi?

I registri vengono mappati ad opportune porte di I/O, e il mapping di queste porte avviene sempre grazie alla MMU che imposta tali aree di memoria come "Kernel Mode", dunque inaccessibili all'utente.

3

Il linguaggio di programmazione C

Perché studiare il linguaggio di programmazione C in un corso di sistemi operativi? Oppure, più in generale, perché studiarlo proprio? La risposta è semplice: C è il linguaggio più utilizzato al mondo, quello con cui sono stati scritti Windows, Linux, e quasi tutti i sistemi operativi. Si tratta di un linguaggio strettamente a basso livello (non quanto l'assembly chiaramente), che permette al programmatore di gestire la memoria del suo programma, tramite allocazioni e deallocazioni manuali. Nelle seguenti sezioni, partiremo davvero dalle basi del linguaggio. Questo capitolo non vuole essere sostituitivo ad un eventuale libro dedicato, ma fornire solo le conoscenze di base per l'uso del linguaggio e di eventuali esempi futuri.

3.1 Le variabili in C

Come ormai dovrebbe essere familiare a molti, una **variabile** è una locazione di memoria a cui abbiamo attribuito un nome, e in cui possiamo memorizzare dei dati. Un esempio di dichiarazione di variabile nel linguaggio C è il seguente `int numero = 42;`.

Come vediamo, la sintassi è la stessa che in altri linguaggi: tipo della variabile, nome, operatore di assegnazione, valore effettivo (se vogliamo attribuirlo da subito).
Infatti, il processo di dichiarazione di una variabile può essere suddiviso in tre momenti:

- **Dichiarazione**: quando esplicitiamo semplicemente il tipo e il nome della variabile, senza darle alcun valore. Esempio: `int numero;`
- **Inizializzazione**: può avvenire solo dopo la dichiarazione, e consiste nell'attribuire un valore ad una variabile già esistente. Esempio: `numero = 4;`
- **Definizione**: combina i due passi precedenti, dichiarando una variabile e inizializzandola ad un certo valore. Esempio: `int numero = 0;`

3.1.1 Visibilità delle variabili

Nel linguaggio C, le variabili hanno due livelli di visibilità:

- **Visibilità locale**: quando una variabile viene definita all'interno di una funzione, può essere vista e utilizzata **solo** da comandi presenti all'interno di quella funzione.

```
1       void esempio() {
2         int x = 3;
3         printf("%d", x)
4       }
5
6       printf("%d", x) // ERRORE!
```

- **Visibilità globale**: si tratta di una pratica sconsigliata, in genere, ma consiste nel memorizzare le variabili fuori da ogni funzione in modo che queste siano accessibili da qualunque pezzo di codice all'interno del file.

```
1       int x = 0;
2
3       void esempio() {
4         printf("%d", x) // CORRETTO, definita globalmente.
5       }
```

I livelli di visibilità sono qualcosa di automatico, ma i programmatori possono decidere di consigliare al compilatore dove memorizzare una data variabile:

- `register`: consiglia al compilatore di memorizzare la variabile all'interno di un registro, in quanto (probabilmente) ne sarà fatto frequente uso.
- `volatile`: indica che il valore di questa variabile potrebbe cambiare per nature indipendenti al programma in cui è stata definita.

3.1.2 I tipi di variabili

Molto velocemente, di seguito si indicano i tipi di variabili fondamentali del linguaggio, cosa permettono di memorizzare e quanto spazio in memoria occupano.

Tipo	Byte occupati	Cosa memorizza
char	1	Caratteri codificati in ASCII
short	2	Numeri interi con segno nel range [-32.768, 32767]
int	4	Numeri interi con segno
unsigned int	4	Numeri interi senza segno
float	4	Numeri con la virgola con una precisione "ridotta"
double	8	Numeri con la virgola e ampia precisione
bool	1	Valori booleani che memorizzano valgono 0 o 1

Inoltre in C, come in altri linguaggi, è presente il **casting di tipo**, ovvero la conversione tra tipi diversi.

- **Casting implicito**, dove una variabile di un tipo viene convertita da sola in un altro. Ad esempio `double d = 30.4; int i = d;`
  In questo caso una variabile di tipo `double` viene messa come valore ad una variabile dichiarata come `int`, il che vuol dire che viene persa nella rappresentazione intera.
- **Casting esplicito**, dove diciamo esplicitamente al compilatore cosa convertire in cosa. Esempio: `double pi = 3.14; int ipi = (int) pi;`

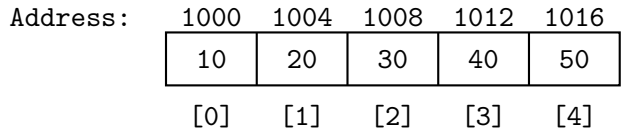
3.2 Gli Array

Un Array è una **sequenza di dati tutti dello stesso tipo** (omogenei tra loro). L'accesso agli elementi viene effettuato mediante `indici`: ogni indice punta ad un determinato elemento. Supponendo di avere un array di `n` elementi, gli indici vanno da `0` (primo elemento) a `n-1` (ultimo elemento). Un Array si può dichiarare come segue:

- `int array[5]`: definizione dell'array, bisogna obbligatoriamente indicare il **tipo di ogni elemento** e il **numero totale di elementi**.

- `int array[] = {1,2,3}`: dichiarazione dell'array con inizializzazione. Quando inizializziamo un array possiamo anche omettere la dimensione, in quanto il compilatore li va a contare.

In memoria, un Array è rappresentato come sequenza di locazioni di memoria, ognuna occupante un numero di byte corrispondente al tipo dell'array.



`sizeof(arr)` = 5 × 4 = 20 bytes

Giusto per accennarlo, si possono definire anche array bidimensionali, dove ogni elemento di un array bidimensionale è a sua volta un array. Un array bidimensionale viene spesso visto come una matrice di `n` righe ed `m` colonne

```
int matrice[2][3] = {{1,2,3}, {4,5,6}}
```

E come si può immaginare, per accedere ad una cella della matrice, si deve indicizzare sia per riga che per colonna. Ad esempio per ottenere l'elemento `4` devo scrivere `matrice[1][0]`.

3.3 Le stringhe

Al contrario di altri linguaggi, come ad esempio il Python o Java, le stringhe non sono un tipo nativo in C. Queste sono definite come un array di caratteri, che ha come ultimo carattere il carattere di escape `'\0'`, che indica il *fine parola*. La definizione di una stringa avviene dunque come segue:

```
char saluto[] = "Ciao";
```

E tale stringa sarà rappresentata in memoria esattamente come abbiamo visto rappresentare un'Array, con l'aggiunta del carattere '\0' citato prima. In questo caso avremo dunque

```
['C', 'i', 'a','o','\0']
```

3.3.1 Funzioni per le stringhe

Il linguaggio C mette a disposizione una libreria `string.h` che dispone anche di alcune funzioni utili nel lavorare con le stringhe.

- **Controllare la lunghezza di una stringa**: si utilizza `strlen` al contrario del classico metodo `sizeof` utilizzato per tutti gli altri tipi di variabile. Questo in quanto `sizeof` conta anche il carattere di fine riga.
- Per **copiare una stringa** all'interno di un'altra possiamo usare la funzione `strcpy`, che però non effettua alcun controllo sulla lunghezza della stringa di destinazione e su eventuali overflow. Il metodo consigliato è dunque `strncpy`, considerato più sicuro in quanto implementa tali controlli.
- Per **confrontare due stringhe** si utilizza il metodo `strcmp`, che prende come parametri due stringhe e restituisce `0 se le stringhe sono uguali`, `1 altrimenti`.

3.4 I puntatori

Uno dei punti di forza, se non *il* punto di forza del linguaggio C è proprio la flessibilità riguardo al controllo sulla memoria. Gran parte di questa gestione è garantita anche grazie ai **puntatori**, ovvero delle **variabili il cui valore è un indirizzo di memoria**. La dicharazione è molto semplice: se abbiamo una variabile di tipo `int` e vogliamo dichiarare un puntatore a quella variabile intera, lo faremo semplicemente aggiungendo un asterisco accanto al nome del tipo, dunque `int*`. Vale ovviamente per ogni tipo di variabile. Chiaramente bisogna mantenere **coerenza** tra il tipo della variabile che vogliamo puntare e il tipo del puntatore.

Strettamente legato al concetto di puntatore è il concetto di indirizzo. Per prendere l'indirizzo appartenente ad una variabile utilizziamo l'operatore `&`.

```
int x = 42;
int* p = $x
```

Inoltre, se utilizziamo di nuovo l'operatore `*` davanti ad un puntatore, faremo riferimento al **valore contenuto nell'indirizzo di memoria puntato da quel puntatore.**

```
int x = 42;
int* p = $x
printf("%d", *p) // Out: 42
```

Volendo, possiamo inizializzare un puntatore al valore speciale `NULL`, per indicare che non punta a nulla.

Osservazione 3.4.1: Puntatori `void*`

In alternativa a quanto visto prima, se vogliamo definire un puntatore che nel corso del programma può fare riferimento a tipi di variabili differenti (non è una good practice, però), potremmo definire un puntatore come `void*`. In questo modo il nostro puntatore potrà puntare all'indirizzo di memoria di qualsiasi variabile, tuttavia **non** potremmo usare l'operatore `*` per accedere al valore dell'indirizzo puntato da quel puntatore. Dovremo fare un casting esplicito al tipo della variabile puntata.

```
int x = 42;
void* p = &x
printf("%d", (int*)*p)
```

Possiam anche definire dei puntatori **costanti**, ovverosia che possono cambiare il valore (cioè l'indirizzo) memorizzato al proprio interno, ma **non può cambiare il valore** associato a quell'indirizzo.

3.4.1 Aritmetica dei puntatori

Di seguito alcune nozioni importanti su come funzionano i puntatori con gli Array e il tipo di operazioni aritmetiche che possonoe essere svolte sui puntatori.

- Un puntatore p ad un Array A punterà **sempre** al primo elemento di quell'array (dunque `int* p = arr`) è come dire `int* p = &arr[0]`.
- Quando incrementiamo un puntatore di un'unità, il valore dell'indirizzo aumenta della `sizeof` del tipo della variabile puntata. Ad esempio, se abbiamo un puntatore `double* p = &x` e facciamo `p++`, il valore dell'indirizzo di memoria andrà avanti della `sizeof(double)`, ovvero il numero di byte richiesti in memoria per memorizzare un double.
  Questo può essere un modo alternativo per iterare lungo gli elementi di un Array.
- Sottraendo il valore di due puntatori `p1, p2` (gli indirizzi di memoria, dunque), otteniamo come risultato **il numero di elementi compresi tra p1 e p2**.
  Attenzione! Non la dimensione complessiva di quegli elementi, ma proprio il numero di elementi in mezzo! Ad esempio:

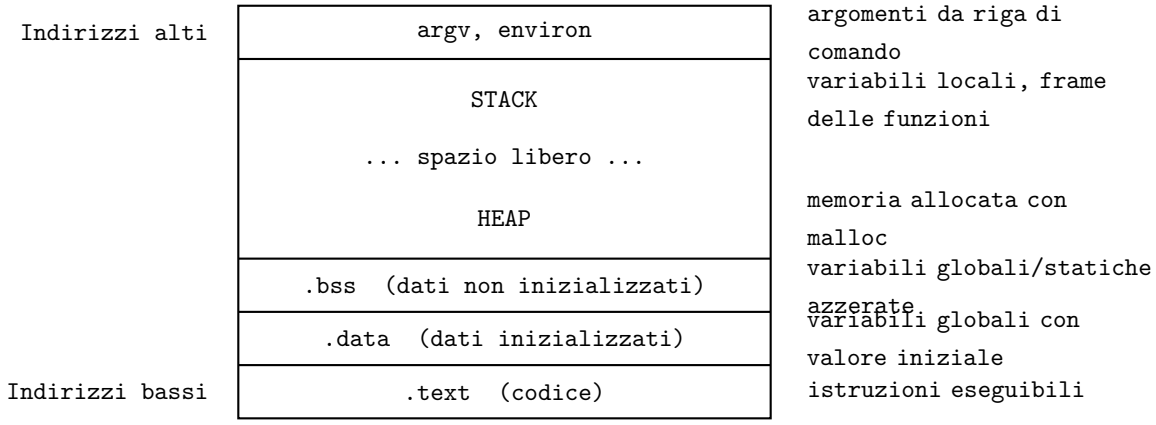
```
int arr[5] = 1,2,3,4,5
```

```
int* secondo = &arr[1]
 int* ultimo = &arr[4]
\\ ultimo - secondo = 2
```

- Possono essere effettuate operazioni di comparazioni tra puntatori, e funzionano come possiamo aspettarci.

3.5 La gestione della memoria

Ogni programma scritto in C organizza la propria memoria nel modo descritto dal diagramma sottostante.



In questa sezione andremo a discutere, più o meno nel dettaglio, il funzionamento di queste componenti.
Forniamo una breve panoramica sulle sezioni "meno interessanti":

- Il segmento **text** (o **code**) mantiene al suo interno le istruzioni del programma codificate nel linguaggio macchina. Ovviamente quest'area di memoria deve essere di sola lettura, in quanto un processo non deve poter modificare il codice mentre lo sta eseguendo.

- I segmenti **data** e **bss** contengono rispettivamente variabili globali/statiche **inizializzate** e **non inizializzate**.

- Lo stack cresce verso il basso, mentre lo heap cresce verso l'alto. Se queste due aree di memoria dovessero incontrarsi, nascerebbe un errore del tipo `Stack overflow Heap exhaustion`.

3.5.1 Lo Stack

Lo **Stack** è una regione di memoria utilizzato principalmente per la memorizzazione delle funzioni e delle variabili a loro associate (che siano essi parametri passati alle funzioni o variabili locali dichiarate al loro interno). Come suggerisce il nome, questa memoria opera in modalità **LIFO**, ovvero l'ultima funzione ad essere inserita sulla pila è anche la prima ad uscire (intuitivamente, se abbiamo una funzione A che chiama al suo interno una funzione B, la funzione B verrà terminata prima della sua chiamante, e dunque rimossa prima dalla pila).

Ogni chiamata di funzione crea un cosiddetto **stack frame**, ovvero un blocco di celle di memoria in cui vengono memorizzati dei dati riguardanti la funzione:

- **Gli argomenti passati alla funzione** che, se sono pochi, vengono memorizzati all'interno di speciali registri, altrimenti vengono inseriti nello stack frame.
- L'indirizzo della prossima istruzione da eseguire **dopo** la fine dell'esecuzione della funzione. Questo valore viene memorizzato in quanto anche la funzione chiamata dovrà seguire un suo valore di Program Counter.
- Un puntatore, detto **frame pointer** che punta all'inizio dello **stack frame della funzione chiamante**.
- **Variabili locali** alla funzione.

Inoltre, nella gestione della memoria Stack, vengono utilizzati due registri:

- **Stack pointer**, che punta alla cima dello stack.
- **Frame pointer** che, come già detto, punta ad uno specifico stack frame.

Come si potrebbe intuire, un eccessivo numero di chiamate di funzione può portare al **riempimento eccessivo dello stack**. In quel caso si parla di **Stack Overflow**, e può accadere ad esempio quando si hanno troppe chiamate ricorsive ad una funzione.

Osservazione 3.5.1: Il compilatore e lo Stack

Quando un programma viene compilato, il compilatore analizza ogni singola funzione, con i suoi parametri, variabili locali e tipi di ritorno e conosce già la dimensione dello stack frame associato a quella funzione.

3.5.2 Lo Heap

Lo **Heap** è una regione di memoria dedicata all'allocazione dinamica e **manuale** delle variabili da parte dell'utente. A differenza dello Stack, la dimensione dello Heap non è conosciuta a compilazione, ma rappresenta comunque un'area di memoria molto vasta.
Le variabili in C possono essere manualmente allocate tramite appositi comandi, che permetteranno alle variabili di essere memorizzate nell'Heap e non in altre aree di memoria.
I comandi predisposti sono i seguenti:

`malloc(n):` serve ad allocare manualmente un blocco di `n` bytes in memoria. Ad esempio, se volessimo allocare dinamicamente un array di 5 elementi potremmo scrivere `int* arr = malloc(5 * sizeof(int))`.
Un'importante osservazione riguarda il fatto che alla chiamata di `malloc(...)` noi non staremo allocando nulla di fatto: il processo si limita a chiedere al sistema operativo di tenere liberi nell'heap `size` byte di memoria. Questi verranno poi definitivamente allocati una volta che verrà richiesto l'accesso a quell'area (quando la andiamo ad utilizzare insomma).

`calloc(n, size):` serve ad allocare `n` byte, ognuno di dimensione `size` e di inizializzare tutti quei byte al valore di default per quel tipo di dato. Tenendoci sull'esempio dell'Array di prima, se facessimo `int* arr = calloc(5, sizeof(int))` e provassimo a mandare a schermo l'Array otterremmo `[0, 0, 0, 0, 0]`.

`realloc(ptr, new_size):` serve per riallocare un blocco di memoria precedentemente allocato. Supponendo di aver definito con `malloc` prima un array `arr` di 5 elementi interi, ma ci siamo resi conto che ci serve spazio per almeno 8 elementi, possiamo semplicemente fare `int* tmp = realloc(arr, 8 * sizeof(int))`.

`free(ptr):` fondamentale, serve a liberare un puntatore di memoria precedentemente allocato manualmente. Importantissimo è anche impostare il puntatore a `NULL` dopo averlo liberato.
In realtà, nel momento in cui chiamiamo il comando `free(ptr)` non stiamo liberando la memoria, bensì stiamo comunicando al sistema operativo di marcare quell'area di memoria come libera. Tuttavia l'effettiva deallocazione non avviene subito, in quanto potrebbe essere richiesta in breve tempo da un'altra `malloc`. La liberazione definitiva avviene solo al termine del processo.

Allocando e deallocando frequentemente nella memoria Heap si potrebbe verificare la cosiddetta **frammentazione della memoria**, ossia avere complessivamente molta memoria libera, ma a blocchi separati da memoria utilizzata. La seguente figura renderà bene il concetto:

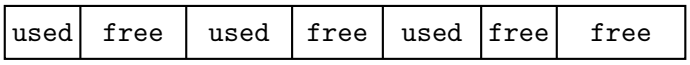


Figura 3.1: Frammentazione della memoria nell'Heap.

3.5.3 Errori di memoria

Spesso è proprio la libertà con la gestione della memoria conferitaci dal linguaggio ad essere causa della maggior parte degli errori che si incontrano nei programmi. Esempi degli errori più classici possono essere:

Perdita di memoria: si verifica quando ci dimentichiamo di liberare una locazione di memoria ormai inutilizzata.

Doppia chiamata di `free()`: si verifica quando chiamiamo `free()` due volte consecutive sullo stesso puntatore, senza averlo prima impostato a `NULL`.

Usare un puntatore dopo averlo liberato

Puntatori fantasma: si verifica quando ad un puntatore vengono restituiti valori di variabili locali a funzioni.

Puntatori non inizializzati: quando si dichiara un puntatore e non lo si inizializza a nessun valore, questo punta ad un indirizzo inutilizzato dal programma ma che potrebbe comunque servire in futuro. E' sempre buona pratica inizializzare i puntatori almeno a `NULL`.

Scrittura oltre i limiti di un Array: quando si prova ad accedere ad indici inesistenti di un Array (es: se `int arr[5]`, non esisterà l'elemento `arr[5]` in quanto il conteggio degli indici parte da 0).

Variabili inizializzate: una variabile non inizializzata contiene un qualsiasi valore presente precedentemente nello stack.

Esistono degli strumenti di compilazione come `valgrind` o `AddressSanitizer`, ma li citiamo solo per nozione e non per approfondirli.

3.6 La traduzione da sorgente ad eseguibile

Il processo di traduzione di un programma C nel suo eseguibile è un'operazione molto lunga e articolata in diverse fasi. In questa sezione vedremo quali sono queste sezioni e le discuteremo una per una. Di seguito un'immagine rappresentante la classica struttura di compilazione di un programma in C:

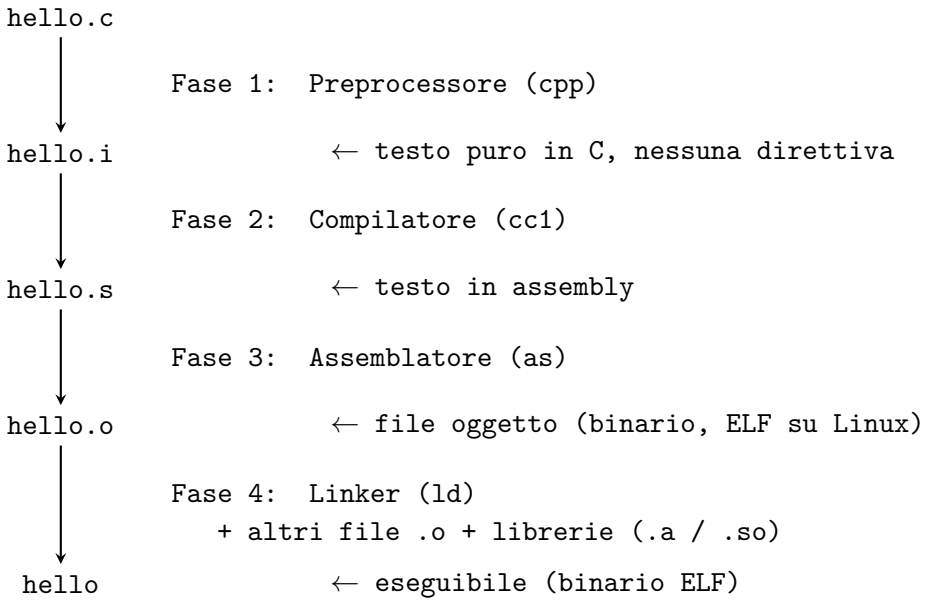


Figura 3.2: Pipeline di compilazione di un programma in C: dal sorgente (`.c`) all'eseguibile finale (`hello`) attraverso le quattro fasi fondamentali.

3.6.1 Il preprocessore

Si tratta del primissimo stadio dell'operazione di compilazione, e si occupa sostanzialmente di **manipolare il testo del programma** che abbiamo scritto. Parlando in modo più tecnico, si occupa di:

- Gestire tutti gli `#include`. Questo viene fatto **copiando e incollando** il codice del file incluso all'interno del nostro file.
- Sostituire tutti i `#define` con la corrispondente sostituzione scelta. Il comando `#define MAX 100` *alias* per una variabile all'interno del nostro programma. Nell'esempio fornito stiamo solo definendo un valore speciale `MAX` che vale 100, ma potremmo anche fare in modo che d'ora in poi nel nostro codice non dovremmo più dichiarare gli interi come `int` bensì come `intero` scrivendo semplicemente `#define intero int`.
- Rimozione dei commenti sostituendoli con un singolo spazio bianco.

Questa fase trasforma il file `hello.c -> hello.i`.

3.6.2 La compilazione

Si tratta della fase che si occupa di **tradurre il codice scritto in C nelle corrispettive direttive scritte in Assembly**. Anche se un progetto include più file, ogni file con

estensione .c viene compilato separatamente. Questa fase trasforma il file `hello.i -> hello.s/asm`

3.6.3 L'assemblaggio

Fase in cui entra in gioco il cosiddetto **assembler**, che si occupa di **convertire il testo scritto in Assembly in istruzioni macchina**. Non entreremo nel dettaglio di quale corrispondenza viene scelta a livello di bit, ma ciò dipende dal tipo di architettura della macchina (CISC/RISC come visto nella sezione 1.4). Questa fase genera il file `hello.s/asm -> hello.o`.

3.6.4 Il linking

Si tratta della fase finale del processo di traduzione, e si occupa sostanzialmente di **unire tutti i file oggetto del progetto in un singolo file eseguibile**. In particolare, vengono fatte queste operazioni intermedie:

- **Risoluzione dei simboli:** se ad esempio nel file `main.c` chiamiamo una funzione `calcola(a, b)`, il linker si occuperà di cercare in tutti i file oggetto una funzione con quel nome. In caso non dovesse trovarla, solleverà un errore tipo `undefined reference to 'calcola'`.
- **Riallocazione:** unisce tutti i file di codice in una singola sezione `.text` e tutte le sezioni dati in una sezione `.data`.

Alla fine di questo processo di traduzione viene finalmente **generato il file eseguibile**.

3.6.5 Librerie statiche e dinamiche

Durante l'operazione di linking, possono essere generati due tipi diversi di librerie.

Libreria statica: si tratta di un file con estensione `.a` (che sta appunto per *archivio*) che contiene una collezione di file oggetto. Durante il processo di linking, *tutto* il codice di questa libreria viene copiato all'interno del programma principale.

Libreria dinamica: si tratta di un file con estensione `.so` pensato per essere condiviso tra più file diversi. Si tratta di un'ottima scelta quando non si vuole far crescere la dimensione dell'eseguibile. Il grande vantaggio sta nel fatto che la copia delle funzioni utili non avviene a compilazione, bensì quando una funzione viene chiamata (il linker si occupa solo di inserire dei riferimenti per ogni funzione e il file in cui questa è definita).

4

Processi e API di sistema

Abbiamo già definito il concetto di *processo* nei capitoli precedenti, ma in questo capitolo ne vedremo meglio la natura, come creare processi, come eseguirli, ecc...

4.1 L'astrazione dei processi

Un processo è un'istanza di un programma in esecuzione, che comprende i seguenti parametri:

- Il **codice** che deve essere eseguito.
- Il valore corrente del **Program Counter**.
- I contenuti dei **registri della CPU**.
- Uno **spazio di memoria virtuale** a cui accedere (ovverosia il suo stack, il suo heap, ...)
- Gli identificativi ai file su cui il processo deve lavorare.
- Un **identificativo univoco di processo**, chiamato **ProcessID (PID)**.

In base alla filosofia secondo cui viene progettato il sistema operativo, questo può decidere di procedere secondo due strategie per allocare il processo in RAM:

- **Caricamento immediato**: appena il processo entra in esecuzione, tutti i suoi dati vengono caricati in memoria RAM. Si tratta di una soluzione facile ma poco efficiente, in quanto potrebbero venir caricate in memoria anche porzioni di codice mai utilizzate.

- **Caricamento pigro**: una sezione del processo viene caricata solo quando deve essere effettivamente utilizzata. Entreremo più nel tecnico di questa modalità quando parleremo di paging e memoria.

4.2 Gli stati di un processo

Affinché lo scheduler del sistema operativo possa decidere quando mandare un processo in esecuzione, quando fermarlo e quando farne partire un altro, si attribuisce ad ogni processo uno **stato**.

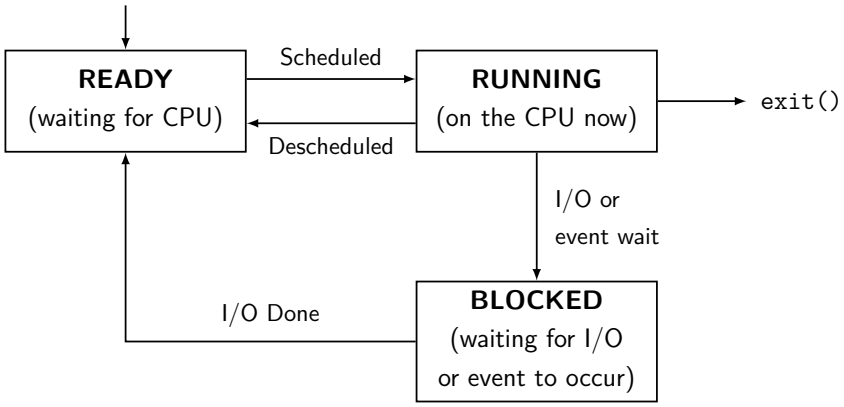


Figura 4.1: Diagramma a tre stati del ciclo di vita di un processo

Come si vede nella Figura 4.1 vi sono 3 possibili stati in cui un processo:

READY: i processi in questo stato sono pronti per essere eseguiti, e attendono solo che lo scheduler li mandi in esecuzione. Dire che un proceso è "pronto" per essere eseguito vuol dire che **possiede tutte le risorse necessarie per eseguire la prossima istruzione**.

RUNNING: è lo stato in cui si trova un **processo in esecuzione**. NOTA BENE: Ci può essere **solo un** processo in esecuzione alla volta su una singola CPU, in quanto è impossibile eseguire due processi simultaneamente. Se avessimo una CPU con n core, si potrebbero eseguire simultaneamente n processi, in quanto ogni core opererebbe come una CPU distinta con i suoi registri e i suoi stati.

BLOCKED: i processi in questo stato sono quelli che **attendono una risorsa da un dispositivo di I/O**, senza la quale non potrebbero continuare l'esecuzione del programma (es: un input da tastiera). Viene definita una categoria a parte per quando si richiedono risorse esterne in quanto abbiamo visto come l'interfacciamento con dispositivi esterni sia un'operazione molto lenta (soprattutto rispetto ai cicli di clock della CPU).

Osservazione 4.2.1: Transizione RUNNING → READY

Affinché un processo passi da READY a RUNNING è sufficiente che lo scheduler decida che "è arrivato il suo momento di eseguire".

Ma per il contrario? Se l'esecuzione di un processo fila liscia, senza neanche richiesta di interfacciarsi con dispositivi esterni, entra il gioco quello che nei capitoli precedenti avevamo chiamato *Timer Interrupt*, ovvero un timer di sistema che assegna ad ogni processo un certo intervallo di tempo per eseguire le sue istruzioni, dopodiché lo fa terminare per lasciare spazio ad altri processi.

4.3 Il Process Control Block (PCB)

Affinché il sistema operativo possa tenere traccia di tutti i dati relativi ad un processo, per ognuno di questi viene associata una particolare struttura dati chiamata **Process Control Block (PCB)**.
Questa struttura contiene informazioni fondamentali riguardo al processo, come ad esempio:

- Il suo **PID**.
- Il suo **stato corrente**, secondo la logica vista poco fa.
- Il **puntatore alla pagina di memoria riservata al process** (**pagetable**).
- Lo **stato e il contenuto dei registri della CPU** (**trapframe**).
  Siccome abbiamo una sola CPU che può eseguire un processo alla volta, quando un processo passa dalla fase di esecuzione ad una qualsiasi altra fase, dovrà memorizzare i valori dei registri nel momento in cui l'esecuzione è stata interrotta, in modo che quando venga ripresa questa non riparta dall'inizio ma dall'ultimo "checkpoint".

Tutti i PCB dei processi inizializzati vengono memorizzati all'interno di una struttura dati chiamata **Process Table** che **risiede in memoria kernel** (in quanto deve essere lo scheduler del sistema operativo a decidere chi eseguire). Lo scheduler andrà poi a scansionare questa tabella in ricerca di processi con stato READY da mandare in esecuzione.

4.4 La gerarchia dei processi

Nei sistemi UNIX, i processi formano un albero in cui la radice è il processo **systemd** con PID = 1. Questo processo non può essere killato neanche volendo, in quanto è strettamente legato al funzionamento del kernel e del sistema operativo.

La gerarchia dei processi è organizzata ad albero in quanto ogni processo può creare altri processi figli (con il comando `fork()`) e dunque ogni processo ha anche un padre (il cui valore si chiama **Parent ProcessID (PPID)**).

4.5 La funzione fork()

La funzione `fork()` è forse una delle funzioni più utili quando si va a lavorare con i processi, in quanto permette di **creare un nuovo processo**.
Più precisamente, ci permette di **duplicare il PCB del processo padre**, ovvero il PCB del processo chiamante. Quest'operazione ci permetterà di allocare un PCB (dunque senza bisogno di conoscere la natura della struttura dati, astrazione!) e poi di modificarlo quasi a piacimento.

Vediamo un esempio di utilizzo di `fork()` all'interno di un codice e commentiamolo:

```
1   int main() {
2     int pid = fork();
3
4     if (pid < 0) {
5       perror("fork failed");
6     } else if (pid == 0) {
7       printf("Processo figlio con PID: %d", getpid());
8     } else {
9       printf("Processo padre con PID: %d", getpid());
10    }
11  }
```

Analizziamo ogni singola istruzione di questo codice che, seppur breve, ci dovrebbe aiutare a capire il funzionamento di `fork()`.

1. **Chiamata alla funzione fork()**: come abbiamo già detto, avendo mandato il programma in esecuzione, quello sarà un processo, supponiamo con PID = 1234. Chiamando `fork()` all'interno di questo processo, stiamo dicendo al sistema di creare un nuovo processo (dunque un nuovo PCB) che sia sostanzialmente una **copia** del PCB del processo chiamante (che sarà il padre di questo nuovo processo).

2. **Cosa viene copiato nel PCB**: il PCB padre e PCB figlio avranno in comune l'area di memoria in cui è presente il codice binario da eseguire, ossia entrambi punteranno **allo stesso indirizzo di memoria**.
   Oltre a questo, non hanno altro in comune: avranno sì gli stessi valori delle variabili,

ma solo perché i valori dei registri saranno stati copiati e incollati anche nel PCB figlio. Il processo figlio avrà le proprie aree di memoria stack e heap, dunque se provassero a modificare la stessa variabile non riuscirebbero ad agire sullo stesso valore in quanto sarebbero logicamente indipendenti. E' molto importante sapere che al processo figlio, ovviamente, viene assegnato un nuovo PID. Supponiamo 1235 per il nostro esempio.

3. A questo punto, avviene una sezione critica: avendo copiato nel processo figlio lo stato di **tutti** i registri, sarà stato copiato anche il valore del Program Counter, e dunque processo padre e figlio riprenderanno **dallo stesso identico punto**.

4. Discutiamo ora il valore di ritorno della funzione `fork()`: prima di restituire un valore, viene eseguita la funzione e dunque creato il processo figlio. A questo punto, come abbiamo detto, i due processi condivideranno la stessa area di memoria per il codice ma non la stessa per le variabili. Qui il sistema fa una cosa interessante:

   - All'interno del processo figlio, il valore di ritorno della funzione sarà **categoricamente 0**. Si tratta di una convenzione stabilita per indicare che il processo corrente è figlio di uno già esistente.
   - All'interno del processo padre, la funzione `fork()` restituirà il PID del processo appena creato.

   Ma come è possibile che una singola funzione restituisca un valore diverso a due processi diversi?
   Questo avviene in quanto, dopo aver creato le aree di memoria dedicate al processo figlio, il valore di ritorno della funzione verrà messo per ogni processo all'interno di uno speciale registro della CPU. Avendo due registri diversi, uno per il padre e uno per il figlio, è possibile creare questa "voluta discrepanza" tra i valori di ritorno.

5. Una volta che la chiamata a `fork()` (supponiamo) è andata a buon fine, quindi, i possibili valori assumibili dalla nostra variabile intera sono **0 per il processo figlio** e $>$ **0 per il processo padre.**
   Questa è la ragione per cui inseriamo quei controlli all'interno del codice: **non avendo garanzia di chi partirà per primo con l'esecuzione**, e magari volendo separare tra le operazioni che può compiere il figlio e quelle del padre, l'unico modo è necessariamente controllare il valore restituito da `fork()`.

Come ci si può aspettare, la funzione di libreria C `fork()` nasconde sottobanco una chiamata di sistema alla funzione `sys_fork()`. Viene effettuata una chiamata di sistema in quanto abbiamo già detto che la Process Table in cui sono memorizzati i PCB dei vari

processi risiede in memoria kernel, e dunque è strettamente vietato far accedere quell'area di memoria da parte di un'istruzione utente.

Inoltre, come è stato già detto nella descrizione del flusso di esecuzione del processo `fork()`, è impossibile predirre chi, tra il padre o il figlio, verrà eseguito prima come processo. Tutto dipende dallo scheduler e dalla priorità attribuita ad ogni processo. E' dunque sconsigliato scrivere porzioni di codice *assumendo che un processo venga eseguito prima dell'altro.*

Un modo per forzare l'esecuzione del processo figlio prima del processo padre è quello di mettere "a dormire" il processo padre, usando la funzione di libreria `sleep()`.

```
int main() {
  int pid = fork();

  if (pid < 0) {
    perror("errore nella fork");
  } else if (pid == 0) {
    // ESEGUO OPERAZIONI FIGLIO
  } else  {
    sleep(1);
    // ESEGUO OPERAZIONI PADRE
  }
}
```

In questo modo stiamo forzatamente interrompendo per 1 secondo l'esecuzione del processo padre, tempo che per la CPU è enorme dato che esegue gli switch tra un processo e l'altro nell'ordine di nanosecondi. Questo modo è chiaramente molto grossolano, ed è una **bad practice** in quanto mette a dormire il padre anche se il processo figlio è stato eseguito per primo.

Prima di concludere il discorso sul comando `fork()`, vediamo come si comportano multiple chiamate della funzione all'interno di un programma.
Supponiamo di avere questo codice:

```
fork(); (1)
fork(); (2)
fork(); (3)
printf("Hello, World!");
```

Ragioniamo: ad ogni chiamata di fork, il numero di processi viene duplicato, in quanto un padre crea un processo figlio.
Alla chiamata (1), diventano 2 processi. Siccome l'esecuzione del codice per tutti i processi riparte dallo stesso punto grazie al valore del Program Counter, ciascuno dei due processi eseguirà anche la `fork()` del punto (2). Dato che avremo 2 processi ad eseguire una `fork()` e ognuno di questi genera 2 processi figli, alla fine di (2) avremo un totale di 4 processi. Come si può immaginare, ognuno dei 4 processi eseguirà un `fork()` duplicando il numero di processi, arrivando così ad 8 processi. Vediamo dunque come il numero di processi finali è dato da 2^n dove n è il numero di `fork()` eseguite.

Dunque se volessimo mandare in crash il sistema operativo, ora sapremmo come fare:

```
1  while (1) {
2    fork();
3  }
```

Listing 4.1: Codice per implementare la cosiddetta "fork bomb".

Questo ciclo continua a creare processi finché non viene raggiunto il limite di processi memorizzabili all'interno della Process Table, e dunque il sistema operativo va in crash.
Una possibile soluzione a questo problema sarebbe definire un limite massimo di processi generabili da un singolo utente.

4.5.1 La filosofia Copy-on-Write

In modo molto ingenuo, il comando `fork()` copierebbe l'intero spazio di memoria di un processo (che potenzialmente potrebbero essere anche diversi GB di memoria) in una nuova area riservata al processo figlio.
I sistemi operativi moderni evitano questa pratica (proprio per evitare sprechi di memoria) implementando la cosiddetta filosofia **Copy-on-Write**:

- Alla creazione del nuovo processo, l'area di memoria a lui attribuita non è una copia di quella del processo padre, bensì gli viene attribuito lo **stesso indirizzo di memoria** della memory page del padre. Possiamo quindi dire che fino ad un certo momento condivideranno letteralmente la stessa risorsa, dato che viene passato l'indirizzo di questa.

- Quando uno dei due processi prova a modificare la suddetta area condivisa, viene sollevata una **Page Fault** dalla Memory Management Unit. Solo a questo punto viene creata la copia in memoria della pagina, che includerà le modifiche apportare

dal processo che le ha richieste.
A questo punto avremmo dunque due pagine di memoria separate:

- La pagina di memoria "originale", alla quale punta il processo che **non** ha provato a modificarla.
- La pagina di memoria *copiata e modificata* dal processo che **ha** provato a modificarla.

4.6 La sincronizzazione dei processi con wait()

Come abbiamo detto e come si vuole ripetere, è impossibile determinare chi, tra il processo padre e quello figlio, riprenda l'esecuzione in seguito ad una `fork()`.
Supponendo che il processo figlio termini prima di quello del padre, la memoria dedicata al processo figlio **non verrà eliminata**, bensì verrà mantenuta fino alla fine del processo padre. Il processo figlio verrà poi messo in uno stato detto **ZOMBIE**.
Per fare in modo che questo problema non si presenti, il sistema operaivo mette a disposizione la funzione `wait()` che ci permette di fermare l'esecuzione del processo padre in attesa che i processi figli siano terminati.

Vediamo di seguito la firma del metodo `wait()`, commentiamola, e poi vediamo un esempio di utilizzo nel codice.

```
int wait(int *wstatus);
```

- La funzione restituisce un valore intero che, se l'esecuzione di `wait()` è andata a buon fine, rappresenta il PID del processo figlio che è terminato.
- Il parametro `int *wstatus` è un modo intelligente per far restituire alla funzione un altro valore.
  Siccome non siamo su Python e non possiamo restituire tuple di valori, il segreto sta nel passare il puntatore ad una variabile non inizializzata e poi ci penserà la funzione `wait()` al suo interno a modificarla, memorizzandoci lo stato con cui si è conclusa l'esecuzione del processo figlio.

Vediamo ora un esempio concreto in un pezzo di codice.

```
1  int main() {
2    int pid = fork();
3    if (pid == 0) {
4      printf("Sta eseguendo il figlio...");
```

```
5           exit(42); // Un generico codice di uscita
6         }
7
8         // Adesso siamo nel parent e dobbiamo attendere che
              finisca il child
9         int stato;
10        int finito = wait(&stato);
11
12        if (child_pid == -1) {
13          perror("wait");
14        } else {
15          printf("Arrivo qui solo dopo che ho finito.")
16        }
17      }
```

Come vediamo, il valore assunto dalla variabile `finito` rappresenta il valore del processo figlio terminato. Nell'ottica di voler far eseguire prima il figlio del padre, questo è l'approccio che si deve attuare.
Esiste anche la funzione `waitpid(int child_pid)` che consente al processo padre di attendere la terminazione dell'esecuzione **solo del processo figlio che ha PID uguale a quello passato come parametro della funzione**.

4.7 La funzione exec()

La funzione `exec()` è una procedura fondamentale in quanto ci permette di sostituire il codice eseguito da un processo con un altro file binario contenente altro codice.
Si tratta di una funzionalità molto utile, in quanto non risulta avere particolarmente senso far eseguire sia un processo che i suoi figli lo stesso file binario.

Osservazione 4.7.1: Successo di exec()

Se la funzione `exec()` viene portata a termine con successo, il processo figlio non tornerà mai ad eseguire il processo padre in quanto avrà cambiato totalmente file binario.

Una variante di `exec()` è la funzione `execvp(const char* file, char* const argv[])`, che prende come input il file binario di cui eseguire il codice ed eventuali argomenti che servono a tale programma per completare la sua esecuzione.
Prendiamo ad esempio il seguente codice:

```
    char* args[] = {"ls", "-l" "-a", "\tmp", NULL};
    execvp("ls", args);
```

Listing 4.2: Il comando ls serve a listare il contenuto della directory "\tmp", e i vari argomenti che iniziano con '-' sono dei flag che compiono operazioni aggiuntive.

Il processo corrente inizierà ad eseguire il comando `ls` (preso in questo caso dalle variabili di ambiente, ma solo perché è uno dei comandi di default di Linux) con i vari argomenti dell'array `args`. Il valore di terminazione per smettere di leggere argomenti è il valore speciale `NULL` all'interno dell'Array degli argomenti.

Avendo visto il funzionamento sia per `fork()` che `exec()`, vediamo un esempio di programma che li combina.

```
    int main() {
      int pid = fork();
      if (pid == 0) {
        execvp("ls", {"\tmp", NULL})
      } else {
        printf("Processo padre.")
      }
    }
```

Listing 4.3: Esempio di uso combinato di fork() e execvp(). Si ignorano possibili errori per brevità.

Avendo visto e compreso il funzionamento di questi due comandi, possiamo avere anche una visione più chiara di come funzioni un'istanza di terminale (shell), in quanto si basa su un loop di istruzioni molto preciso:

1. Far scrivere all'utente un comando.
2. Leggere il comando e dividerlo in token.
3. Creare un processo figlio che si occupi di eseguire il comando (`fork()`).
4. Far eseguire il comando al processo figlio (`execvp()`).
5. Attendere che il processo figlio termini l'esecuzione del comando (`wait()`).
6. Restituire il controllo al processo padre e ritornare all'inizio del loop.

Osservazione 4.7.2: Reale funzionamento di una shell

Una vera shell in realtà consente anche all'utente di eseguire nuovi comandi mentre altri stanno continuando in background. Questo può avvenire inserendo il carattere `&` alla fine di un comando, il che indicherà al sistema operativo di restituire il controllo all'utente subito dopo aver mandato in esecuzione il comando precedente.

4.8 Altre API di sistema

Di seguito un veloce elenco di altre API di sistema importanti ma non fondamentali come quelle descritte precedentemente.

exit(): si occupa di terminare un processo. E' presente in due varianti:

- `exit()`, che termina il processo in modo "lento" ma pulendo tutti i buffer e pulendo tutti gli stream.
- `_exit()`, che termina il processo in modo veloce ma senza pulire buffer e stream.

kill(): per inviare un segnale al processo figlio e "ucciderlo".

nice(): per aggiornare il livello di priorità di un processo. Tale livello è un numero intero che va da **-20** (**priorità massima**, solo il kernel) a **+19** (**priorità minima**, programmi utente).

pipe(): crea un canale di comunicazione tra due o più processi per permettere di condividere l'output di un processo con l'input di un altro.

5

Scheduling della CPU

Nei capitoli precedenti abbiamo accennato a come i sistemi moderni creino l'illusione di eseguire simultaneamente più processi "semplicemente" effettuando dei rapidissimi scambi di stato tra i processi in esecuzione.
In questo capitolo vedremo più nel dettaglio il ruolo dello **scheduler**, ovverosia quel modulo del kernel che si occupa di decidere, mediante un accorto **algoritmo di scheduling**, l'ordine dei processi da mandare in esecuzione.

L'obiettivo non sarà tuttavia fornire direttamente la soluzione adottata oggi dai sistemi moderni, bensì di guidare l'apprendimento imponendo dei vincoli e man mano rimuovendoli, per trovare standard sempre più efficaci.

5.1 Assunzioni sul carico di lavoro e metriche

Affinché lo scheduler possa organizzare i processi, bisogna definire il carico di lavoro e delle metriche secondo le quale stabilire questi confronti.

Definizione 5.1.1: Carico di lavoro

Il carico di lavoro descrive:

- Quanti processi stanno attualmente venendo eseguiti dal sistema.
- L'istante di tempo in cui viene creato un nuovo PCB.
- **Quanto tempo** è richiesto per ogni processo.
- **Quando** il processo dovrà fare operazioni di I/O.

Osservazione 5.1.1: Natura asincrona dei processi

E' doveroso fare un appunto sulla definizione soprastante.
Per lo scheduler è **impossibile** predirre gli ultimi due punti, in quanto i processi sono di per sè un'unità asincrona in cui alcune cose non sono deterimistiche.

Al fine di costruire il nostro percorso in maniera graduale verso un algoritmo di scheduling efficiente che funzioni per ogni sistema, dobbiamo partire letteralmente *dalle basi*, facendo delle assunzioni **molto comode** sulla natura del workload:

1. Ogni processo richiede la **stessa quantità di tempo** per essere eseguito.
2. Tutti i processi **arrivano allo stesso momento** ($t = 0$).
3. Prima di cambiare processo deve essere **completamente terminato il corrente** (nessuna interruzione del processo).
4. **Nessun** processo ha necessità di eseguire operazioni di **I/O**.
5. Il **tempo di esecuzione** di ogni processo è **noto a priori**.

Come vediamo sono delle assunzioni molto comode, che però piano piano andremo a calare per migliorare progressivamente l'algoritmo di scheduling che vogliamo progettare.

Oltre a queste assunzioni, che si permetteranno di definire l'algoritmo, dobbiamo anche avere dei criteri per permetterci di stabilire se un algoritmo performa meglio di un altro. Abbiamo principalmente due metriche:

Tempo di completamento (Turnaround Time): indica il tempo totale trascorso da quando il processo arriva al sistema a quando viene finito di eseguire.
La formula per il calcolo di questo parametro è la seguente:

$$T_{\text{turnaround}} = T_{\text{fine_processo}} - T_{\text{arrivo}}$$

Prendendo la media del tempo di completamento di tutti i processi schedulati, otteniamo l'importantissimo parametro di **Tempo di completamento medio**.

Tempo di risposta (Response Time): rappresenta il tempo da quando un processo arriva al sistema a quando arriva ad essere eseguito per la prima volta. Sostanzialmente indica quanto tempo un processo rimane in coda prima di essere considerato.
La formula per questo parametro è la seguente:

$$T_{\text{response}} = T_{\text{prima_esecuzione}} - T_{\text{arrivo}}$$

Questi appena descritti sono i parametri fondamentali nel calcolo dell'**equità** e delle **performance** di un algoritmo di scheduling.
In particolare:

Equità (Fairness): rappresenta in che misura il **tempo della CPU viene distribuito tra tutti i processi**. Un algoritmo è "**fair**" se garantisce a ogni processo di fare progressi regolari, evitando che qualche task rimanga bloccato ad aspettare per un tempo indefinito (starvation).

Prestazioni (Performance): rappresenta la **capacità di minimizzazione del turnaround time o del response time**.

Si può facilmente intuire come queste due filosofie siano contrapposte, pertanto si deve scegliere cosa si deve prediligere maggiormente in un algoritmo, oppure trovare l'equilibrio più vantaggioso tra le due.

5.2 I primi passi: FIFO e SJF

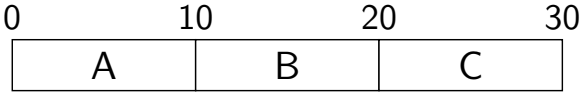
Avendo messo le basi per la nostra analisi degli algoritmi di scheduling, partiamo dalle basi, con due delle strategie più semplici.

5.2.1 FIFO - First In, First Out

E' forse la schedule più semplice a cui si potrebbe pensare: **i primi processi ad entrare sono anche i primi a venir eseguiti**.
Anche la sua implementazione risulta particolarmente semplice, in quanto consiste semplicemente nella gestione di una struttura dati *coda*. Vediamo un semplice esempio di questo algoritmo usando anche le nostre 5 assunzioni precedenti.

Supponiamo di avere tre job: **A**, **B** e **C** e che:

- Tutti arrivino allo stesso istante di tempo $t = 0$.
- Tutti abbiano la stessa durata di $10s$.



Andiamo adesso a calcolare il tempo di completamento per ogni processo e di conseguenza il tempo medio:

- A termina a $t = 10$. Tempo di completamento $= 10 - 0 = 10s$
- B termina a $t = 20$. Tempo di completamento $= 20 - 0 = 20s$
- C termina a $t = 30$. Tempo di completamento $= 30 - 0 = 30s$

Tempo di completamento medio: $\frac{10+20+30}{3} = 20s$.
Non è un brutto tempo, solo che stiamo lavorando con delle ipotesi molto comode.

Adesso, andiamo a rilassare l'Assunzione 1, quella sull'equa durata di tutti i processi.
Proviamo a ridisegnare lo schema precedente supponendo ora che $A = 100s, B = C = 10s$.



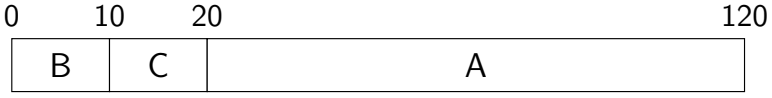
Se andassimo ora a calcolare il tempo di esecuzione minimo per ogni processo, il risultato sarebbe ben diverso:

- A termina a $t = 100$. Tempo di completamento $= 100 - 0 = 100s$
- B termina a $t = 110$. Tempo di completamento $= 110 - 0 = 110s$
- C termina a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$

Tempo di completamento medio: $\frac{100+110+120}{3} = 110s$.
Vediamo che la situazione è cambiata, e questo dato che i processi corti B e C sono bloccati dietro il processo lungo A e devono attenderne la fine. Nello studio degli algoritmi di scheduling, questo è detto **effetto convoglio**.

5.2.2 SJF - Shortest Job First

Per aggiustare questo effetto convoglio, evitando dunque che i procssi corti siano messi dopo uno più lungo di loro e rimangano bloccati, si attua la tecnica Shortest Job First (SJF) che, come suggerisce il nome, prevede di **eseguire per primo il processo più corto di tutti**.
Tenendo conto di questa nuova strategia, se andiamo a ridisegnare il diagramma precedente con $A = 100, B = C = 10$, vediamo come le cose cambino anche solo eseguendo per primi i processi corti.



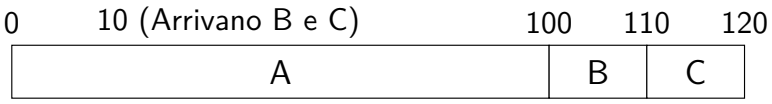
- B termina a $t = 10$. Tempo di completamento $= 10 - 0 = 10s$
- C termina a $t = 20$. Tempo di completamento $= 20 - 0 = 20s$
- A termina a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$

Facendo la media, otteniamo $\frac{10+20+120}{3} = 50s$ il che è un tempo inferiore per più della metà rispetto a prima!

Osservazione 5.2.1: Utilità del SJF

L'algoritmo SJF è **ottimale** per **minimizzare il tempo di completamento medio per processi che arrivano tutti allo stesso momento**.

Avendo lavorato sulla prima assunzione, riguardante la durata dei processi e avendo trovato un algoritmo che ci permette di rendere efficiente lo scheduling con quell'ipotesi rilassata, andiamo a rilassare anche a rimuovere la seconda assunzione, riguardante il fatto che **tutti i processi arrivano allo stesso tempo**.
Modifichiamo leggermente le variabili del nostro diagramma: $A = 100, t = 0; B = C = 10, t = 10$.



Analizzando ciò che accade:

1. Al tempo $t = 0$, l'unico processo presente è A e dunque lo scheduler deve necessariamente scegliere quello.
2. Al tempo $t = 10$ arrivano B e C, ma siccome il SJF non offre nessun meccanismo per gestire l'interruzione di un processo, questi dovranno attendere che A sia terminato prima di venir eseguiti.

Vediamo dunque che in questa casistica, avendo semplicemente rilassato la seconda ipotesi, **l'effetto convoglio è tornato**.

- A termina a $t = 100$. Tempo di completamento $= 100 - 0 = 100s$
- B termina a $t = 110$. Tempo di completamento $= 110 - 10 = 100s$
- C termina a $t = 120$. Tempo di completamento $= 120 - 10 = 110s$

Il tempo di completamento medio risulta essere $\frac{100+100+120}{3} = 103.3s$. Non ottimale.

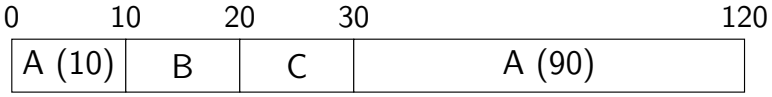
5.3 Interruzione dei processi e STCF

Per costruire un algoritmo di scheduling efficiente, dobbiamo rilassare la terza assunzione (i processi vanno avanti finché non vengono terminati).
Rilassare questa assunzione significa anche introdurre un meccanismo per memorizzare il *contesto* di un processo (ovvero i valori di tutti i registri utilizzati) per permettergli di ricominciare da dove si era interrotto. Come abbiamo già detto, però, questa abilità di interrompere è possibile grazie al Timer Interrupt, che è un dispositivo hardware.

Introduciamo così l'algoritmo **STCF - Shortest Time to Complete First** che opera nel seguente modo:

- Quando un processo entra nella coda dei processi, lo scheduler compara il tempo di esecuzione rimanente del processo corrente con quello totale del nuovo processo.
- Se il nuovo job ha un tempo di esecuzione inferiore rispetto al corrente, viene effettuato un cambio di contesto ed eseguito il nuovo processo fino a terminazione (o finché non ne arriva uno più corto).

Con questa nuova strategia, vediamo come si comporta la nostra schedule precedente: Andiamo ad illustrare nel dettaglio cosa accade:



1. Al tempo $t = 0$ è presente solo il processo A, dunque viene eseguito lui.
2. Al tempo $t = 10$ arrivano i processi B e C, entrambi della durata di $10s$.
    - Essendo arrivato un nuovo processo, viene comparato il tempo di esecuzione rimanente di $A(100 - 10 = 90s)$ con la durata del processo entrante $10s$.
    - Siccome il processo entrante è più corto, viene subito mandato in esecuzione.

Andiamo allora a vedere se è effettivamente migliorato qualcosa, a livello di tempi di completamento:

- A arriva a $t = 0$ e finisce a $t = 120$. Tempo di completamento $= 120 - 0 = 120s$.
- B arriva a $t = 10$ e finisce a $t = 20$. Tempo di completamento $= 20 - 10 = 10s$.
- C arriva a $t = 10$ e finisce a $t = 30$. Tempo di completamento $= 30 - 10 = 20s$.

La media tra queste grandezze è $\frac{120+10+20}{3} = 50s$. Decisamente un miglioramento pensando ai $103.3s$ precedenti.

Osservazione 5.3.1: Utilità del STFC

L'algoritmo STCF è **ottimale** quando si devono **eseguire processi di cui si conosce la lunghezza** ma che **possono arrivare in momenti diversi**.

Tuttavia, anche questo algoritmo non è sicuramente esente da difetti. Immaginiamo una situazione in cui il processo A di durata $100s$ inizia ad essere eseguito a tempo $t = 0$. Supponiamo che poi ogni 10 secondi arrivi un processo di durata inferiore ad A: verrebbe data priorità a lui, in quanto più corto. A lungo andare verrebbero eseguiti sempre processi di breve durata, ma quello più lungo rimarrebbe in coda ad attendere. Tale situazione è detta **Starvation**, che è un problema di **equità**.

5.4 Round Robin (RR)

Supponiamo di star eseguendo un processo su un sistema multiutente. L'utente A sta modificando un file di testo e l'utente B sta compilando un programma.
Supponendo che la compilazione richieda $10s$, gli algoritmi SJB e STCF darebbero la precedenza a questo processo. Ciò vuol dire però che all'utente A in quei $10s$ verranno ignorate le pressioni di tasti. Ci rendiamo conto di come, in una situazione reale, tali algoritmi non siano molto efficienti.

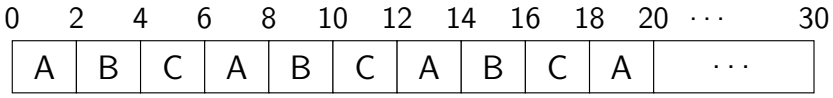
Ecco perché introduciamo l'algoritmo **Round Robin (RR)**, che si occupa di schedulare i processi seguendo un loop circolare:

- Anziché eseguire un processo fino alla fine, lo esegue per un breve lasso di tempo.
- Alla fine di questo lasso di tempo, il processo viene spostato alla fine della coda.
- Viene eseguito il processo attualmente in testa, e così via finché i processi non sono esauriti...

Il lasso di tempo entro il cui un processo viene eseguito è detto **Time Slice** o **Scheduling Quantum**, il quale deve rispettare le seguenti caratteristiche:

- Deve essere abbastanza lungo da ammortizzare i costi del context switch.
- Deve essere abbastanza corto da creare l'illusione dell'esecuzione simultanea.

Vediamo un esempio pratico, come fatto prima per i precedenti algoritmi: supponiamo di avere i processi A, B, C, tutti che arrivano a $t = 0$ e dalla durata di $10s$ ciascuno; il time-slice è impostato a $2s$. Stavolta andiamo a calcolare i **tempi di risposta** (che ricordiamo



essere la differenza tra quando un processo arriva in coda e quando viene eseguito per la prima volta).

- A arriva a $t = 0$ e viene eseguito per la prima volta a $t = 0$. Tempo di risposta $0 - 0 = 0s$.
- B arriva a $t = 0$ e viene eseguito per la prima volta a $t = 2$. Tempo di risposta $2 - 0 = 2s$.
- C arriva a $t = 0$ e viene eseguito per la prima volta a $t = 4$. Tempo di risposta $4 - 0 = 4s$.

Otteniamo come il tempo medio di risposta sia $\frac{0+2+4}{3} = 2s$, il che è un tempo di attesa **significativamente basso**. Il lettore può provare da solo a calcolare tale grandezza in riferimento ad un algoritmo come SJF.

Osservazione 5.4.1: Utilità del Round Robin

Come abbiamo visto, l'algoritmo Round Robin è **ottimale** per **minimizzare i tempi di risposta** (cioè per garantire che i processi vengano tutti "serviti" in un tempo ragionevole), tuttavia risulta terribile per quanto riguarda il tempo di completamento.

Considerando sempre l'esempio precedente, abbiamo che A termina a $t = 26$, B termina a $t = 28$, C termina a $t = 30$. Vediamo come il tempo di completamento minimo sia di $\frac{26+28+30}{3} = 28s$, il che è un tempo anche peggiore degli algoritmi SFJ e STCF.

Possiamo dunque fare una considerazione generale per quanto riguarda i tempi di risposta e di completamento.

Osservazione 5.4.2: Trade-off tra risposta e completamento

- Ogni politica che **ottimizza il tempo di risposta avrà un downgrade sul tempo di completamento**.

- Ogni politica che **ottimizza il tempo di completamento avrà un downgrade sul tempo di risposta**.

5.5 Incorporazione dell'I/O

Purtroppo, è arrivato anche il momento di rilassare la quarta assunzione, riguardante l'assenza di operazioni di I/O da completare, anche perché queste sono molto più frequenti di quanto ci si possa immaginare.
Come sappiamo avendo studiato gli stati di un processo, quando il job deve svolgere un'operazione di I/O si deve:

1. Mettere il processo in stato `BLOCKED`.
2. Attendere che l'operazione sia completata, e intanto eseguire altri processi.
3. Una volta completata l'operazione, il processo viene rimesso nella coda dei processi `READY`.

Il punto di un buono scheduler è proprio garantire che venga eseguito un altro processo nel frattempo che il corrente è impegnato in un'operazione di I/O.
La sezione di tempo in cui un processo viene eseguito è detta **CPU Burst**, mentre quella in cui il processo attende l'I/O si chiama **I/O Wait**.

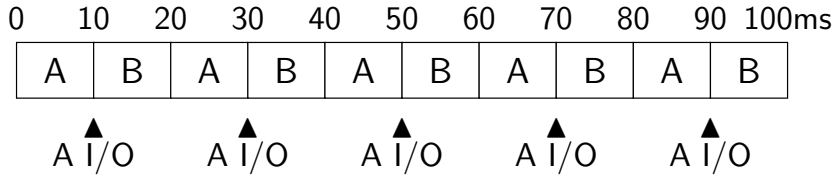
Tutto bello ma...come lo implementiamo nei nostri algoritmi di scheduling?
Il trucco è trattare ogni CPU Burst come un sotto-processo indipendente, che viene eseguito e schedulato per conto suo.

- Se il processo A ha bisogno di un totale di 50ms di CPU, dividiamo questo tempo in 5 CPU Burst ognuno da 10ms, separati ognuno da operazioni di I/O.
- Ogni CPU Burst da 10ms viene schedulato come se fosse un processo indipendente.

Vediamo un esempio di questa implementazione con l'algoritmo STCF, con il seguente esempio:

- Il processo A ha 10ms di CPU Burst e 10ms di I/O wait (il tutto ripetuto per 5 volte).
- Il processo B ha 50ms di solo CPU Burst.

Ricordiamo che l'algoritmo di scheduling è il STCF, dunque il workflow è il seguente:



1. Il processo A viene schedulato per primo perché la sua durata (10ms, ricordiamo che A è stato suddiviso in sotto-processi indipendenti) è inferiore a quella del processo B (50ms).

2. Dopo 10ms, il processo A entra in I/O Wait per 10ms, tempo durante il quale verrà eseguito il processo B.

3. Avanti così finché uno dei due processi non termina...

5.6 Algoritmi di scheduling più realistici

Finora, analizzando gli algoritmi di scheduling quali FIFO, SJF e STCF ci siamo sempre basati sulle famose 5 assunzioni e semplificazioni fatte all'inizio del capitolo. Abbiamo già sottolineato quanto queste fossero irrealistiche, dato che è impossibile per la CPU conoscere a priori per quanto tempo andrà in esecuzione un processo o quando arriverà. Possiamo dividere i processi che possono arrivare ai sistemi operativi in due classi di processi:

Processi innterattivi: caratterizzati da piccoli momenti di CPU Burst (quindi breve utilizzo della CPU) seguiti da richieste di I/O (es: pensiamo alla tastiera).

Processi batch: richiedono solo ed esclusivamente l'uso della CPU, oppure qualche processo esterno ogni tanto.

Un buono scheduler deve essere in grado di gestire entrambe questa classi di processi.

5.6.1 Multi-Level Feedback Queue (MLFQ)

Nasce così una nuova idea di organizzazione: l'algoritmo **Multi-Level Feedback Queue (MLFQ)**. Questo algoritmo lavora secondo questo ragionamento: se un processo sembra apparentemente comportarsi come un processo interattivo/batch, trattiamolo come tale. Nel concreto, questo risultato si ottiene implementando non una sola coda bensì **varie code**, ognuna con una priorità diversa: la coda più in alto sarà anche quella con priorità maggiore.

La struttura di un generico algoritmo MLFQ è la seguente:



Figura 5.1: Esempio di organizzazione delle code di un algoritmo MLFQ.

Bisogna dunque definire le regole per questa struttura:

1. Ogni processo entrante viene messo nella coda con più altra priorità (Q0).
2. Ad ogni coda è assegnata una priorità, ma tutti i processi appartenenti ad una coda godono della stessa priorità. Dunque tutti i processi in priorità$_{Q0}$ > priorità$_{Q1}$ > priorità$_{Q2}$ > priorità$_{Q3}$.
   Fatta quest'opportuna osservazione, definiamo alcune regole per la scelta del processo da eseguire:
   (a) Viene sempre eseguito il processo con la priorità più alta, dunque se Q2 ha all'interno dei processi ma nel frattempo ce ne sono ancora in Q1, si dovrà attendere che terminino i processi dentro Q1 prima di iniziare ad eseguire quelli di Q2.
   (b) Se due processi hanno la stessa priorità, significa che appartengono alla stessa coda e dunque vengono eseguiti secondo l'algoritmo Round Robin.
3. Per ogni coda sono definiti due parametri:
   - Il quanto temporale (**time-slice**) per cui ogni processo può essere eseguito consecutivamente.
   - Un tempo massimo di permanente di un processo all'interno della coda (**time allotment**). Una volta che un processo esaurisce questo tempo di permanenza, viene declassato nella coda sottostante. Questo tempo **deve** essere **cumulativo**, ovverosia bisogna tenere traccia per ogni processo di quanto tempo ha usato la CPU mentre si trovava in una determinata coda, in modo da poterlo eventualmente declassare.

4. Dopo un certo intervallo di tempo S tutti i processi vengono messi nella coda con più alta priorità, per impedire lo *starvation*: dovendo eseguire sempre i processi nella coda con più alta priorità, c'è il rischio che se ad esempio Q1 non si svuotasse mai, i processi in Q2 e Q3 non verrebbero mai eseguiti. Come ogni grandezza che ci viene richiesto di scegliere, dobbiamo trovarne il giusto equilibrio tra né troppo lunga e né troppo corta.

Osservazione 5.6.1: Time-allotment e operazioni di I/O

Il concetto del time-allotment è fondamentale, e previene che un processo ad alta priorità monopolizzi la coda in cui si trova.
In particolare, il conteggio del time-allotment fa riferimento al **tempo di utilizzo della CPU da parte del processo**. Dunque nel conteggio non viene (giustamente) contato eventuale tempo in cui il processo attendeva risposte da dispositivi di I/O.

Osservazione 5.6.2: Il boost di priorità funziona?

Il concetto di boost di priorità S è fondamentale per impedire la starvation.
Anche se può sembrare che facendo risalire i processi di coda non cambi nulla, questo non è vero: i processi scendono di priorità quando esauriscono il tempo disponibile in quella coda. Dunque affinché un processo che si trovava giù possa tornarci, dovrà usare almeno un po' la CPU, e dunque vi saranno degli effettivi progressi.

Vediamo degli esempi di come vengono gestiti i processi interattivi e quelli CPU bound.

Un solo processo interattivo

Supponiamo di avere una struttura MLFQ a 3 code.
Diamo adesso i vari valori di time allotment e time slice:

- Q0: time-allotment = 1ms, time-slice = 1ms.
- Q1: time-allotment = 4ms, time-slice = 1ms.
- Q2: FIFO

E supponiamo anche di avere un processo A che ha 0.5ms di CPU Burst e poi attende 4.5ms per una risposta I/O.

Tempo:	0	0.5	5	5.5	10	10.5	15	15.5	···
Q0:	[A]	[I/O]	[A]						
Q1:				[I/O]	[A]	[I/O]	[A]	···	
Q2:									

Figura 5.2: Diagramma descrittivo dell'esempio fornito su un processo interattivo.

Come vediamo, inizialmente il processo A esegue 0.5ms nella coda Q0. Per la natura del processo (che abbiamo descritto solo per semplificare la comprensione dell'esempio) vi saranno in seguito 4.5ms di I/O. Terminati questi, saremo a t=5s e, dato che il processo A ha eseguito solo 0.5ms degli 1.0ms totali disponibili, può eseguire un'altro ciclo di CPU. Adesso con questi ulteriori 0.5ms ha ufficialmente esaurito il tempo nella coda Q0 e viene dunque declassato nella coda Q1. Da qui il processo riprenderà, ma A potrà "rimanerci di più" grazie al maggiore time allotment.

Un solo processo Batch

Stesso setup dell'esempio precedente, solo che stavolta abbiamo un processo B che deve eseguire un totale di 20ms di lavoro CPU.

$t = 0$–1 ms: Q0: B viene eseguito per 1 ms → quota esaurita → retrocesso a Q1
$t = 1$–5 ms: Q1: B viene eseguito per 4 ms → quota esaurita → retrocesso a Q2
$t = 5$–20 ms: Q2: B viene eseguito per i rimanenti 15 ms fino al completamento

Si omette l'esempio riguardante 2 o più processi in quanto risulterebbe confusionario. Tuttavia come ci si può immaginare, due processi nella stessa coda vengono eseguiti con l'algoritmo Round Robin, e dunque il flusso è facilmente analizzabile.

Osservazione 5.6.3: Perché i processi interattivi in alto?

Una domanda potrebbe sorgere spontanea: perché i processi interattivi, che richiedono di bloccare "spesso" la loro esecuzione mediante istruzioni di I/O vengono messi nella coda con più priorità?

Questo avviene proprio per la quantità di istruzioni I/O che possiedono: se avessimo messo nella coda Q0 un processo batch, quello avrebbe utilizzato in modo continuo tutto il suo tempo nella coda, per poi terminare o venir declassato. Se invece facciamo usare questo tempo ai processi interattivi, ci aspettiamo che questi interrompano l'esecuzione e dunque escano dalla coda più tardi. E questo è un *bene*, dato che durante le operazioni di I/O la CPU non viene bloccata, bensì può eseguire altri processi.

Esempio 5.6.1: Come funzionava l'algoritmo Linux O(1)

Nelle versioni successive alla 2.6, il kernel di Linux ha iniziato ad implementare una procedura di scheduling molto creativa, chiamata ($O(1)$) in quanto era in grado di scegliere il prossimo processo da eseguire pressoché in tempo costante.

Il tutto è stato possibile grazie ad una rivisitazione dell'algoritmo MLFQ, utilizzando 140 code. Normalmente, per controllare da quale coda eseguire un processo si dovrebbe partire da quella di priorità più alta, controllare se è vuota e in tal caso scendere, ..., finché non se ne trova una non vuota di cui eseguire i processi.
Chiaramente si tratta di un'operazione lineare nel numero di processi in esecuzione. Per ottimizzare questa ricerca, Linux ha implementato il concetto di **bitmask**, ovvero una sequenza di 140 bit che fungevano da flag uno per ogni singola coda:

- Se il bit i-esimo == 0, allora la coda è vuota.
- Se il bit i-esimo == 1, allora la coda ha almeno un processo.

Dunque, per trovare una coda con almeno un processo era sufficiente scansionare questa sequenza di bit (un'operazione che richiede un tempo esiguo) e trovare il primo bit a valore 1: la coda associata a quel bit è la coda i cui processi verranno eseguiti.

L'algoritmo MLFQ rappresenta finora la scelta più bilanciata tra gli algoritmi che abbiamo visto: capace di gestire più processi, assegnando varie priorità, senza conoscere nulla a priori riguardo un processo.
L'unico problema è l'**equità tra processi**, cioè **garantire che tutti i processi godano di almeno una frazione di CPU**. Insomma, vogliamo distribuire il lavoro all'interno della CPU in base a quanta importanza ha un processo (es: un server DBMS che gestisce il nostro database è sicuramente più importante del programma in C che calcola la somma di due numeri).
Si utilizzano dunque due approcci standard.

5.6.2 Lottery Scheuduling

Come suggerisce il nome di questo algoritmo, il suo funzionamento è molto simile a quello di una lotteria: definiamo un numero totale di **biglietti** e li assegniamo in modo proporzionato ai processi.

Osservazione 5.6.4: Cosa vuol dire proporzionato?

Distribuire i biglietti in modo proporzionato vuol dire che la distribuzione dei biglietti **non avviene in modo casuale**, bensì assegniamo un numero maggiore di biglietti ai processi con più importanza, quelli che vogliamo abbiano più rilevanza per la CPU.

Adesso arriva la parte casuale: lo scheduler estrae un numero casuale, come nella lotteria. Il processo che avrà il biglietto corrispondente a quel numero verrà schedulato.

Processo A:	75 biglietti	(75% della CPU)
Processo B:	25 biglietti	(25% della CPU)
Totale:	100 biglietti	

Figura 5.3: Esempio di distrbuzione dei biglietti tra processi.

Dall'esempio in figura, possiamo assumere che il processo A debba eseguire un'operazione più importante e dunque gli attribuiamo più biglietti. NOTA: Attribuire più biglietti aumenta la probabilità che il ticket di A venga estratto, ma nulla vieta che potrebbe anche essere estratto quello di B, e dunque vada in esecuzione lui.

Senza entrare nello specifico del calcolo statistico e probabilistico, per la legge dei grandi numeri sappiamo che più è grande il numero delle estrazioni, più la probabilità dei processi viene distribuita equamente. Sempre per la legge dei grandi numeri, affinché ciò sia possibile serve che $n \longrightarrow \infty$ e dunque per una piccola quantità di processi da schedulare la disparità si fa sempre più ampia, non risolvendo di fatto il problema di equità presentato all'inizio.

5.6.3 Stride Scheduling

Per garantire l'equità tra i processi, abbiamo bisogno di un approccio più deterministico, cioè che non si basi su supposizioni o estrazioni casuali, ma su calcoli effettivi: deve essere la macchina ad imparare, in qualche modo, quale processo è più importante e di conseguenza adattarsi.

Per fare tutto ciò non servono complesse tecniche di Machine Learning: basta usare un po' di matematica, mischiata ai concetti visti già in precedenza per il lottery.

Ad ogni processo i assegniamo due valori:

$$\text{stride}_i = \frac{W}{\text{tickets}_i}$$

$$\text{pass}_i = 0(\text{inizialmente})$$

dove W è una costante grande decisa arbitrariamente da noi.

Facciamo delle importanti osservazioni:

1. Come vediamo, il concetto di *ticket* del lottery non è stato abbandonato, in quanto la distribuzione dei biglietti era l'unico aspetto deterministico di quell'algoritmo. Sostanzialmente ci permette ancora di attribuire una specie di "importanza" ad ogni processo.
2. Maggiore è il numero di biglietti di un processo, maggiore sarà la sua importanza e dunque minore sarà lo stride. Questo è molto importante per una formula che vedremo a breve.
3. Il *pass* rappresenta una specie di **priorità**.

Le regole fondamentali di questo algoritmo di scheduling sono:

1. Eseguire sempre il processo con il **minimo valore di pass**.
2. Dopo aver terminato l'esecuzione, aggiorniamo il valore di pass come $\text{pass}_i = \text{pass}_i + \text{stride}_i$.

Osservazione 5.6.5: L'importanza dei ticket

Come vediamo, nonostante l'algoritmo di Stride scheduling non si basi esclusivamente sui ticket e non includa alcun tipo di estrazione, avere tanti ticket è importante per un processo: avendo tanti ticket, il valore di stride_i sarà un valore piccolo e dunque ad ogni esecuzione il valore di pass si aggiornerà di molto poco, permettendo al processo di essere eseguito più spesso.

Maggiore importanza -> maggiore uso della CPU, come avevamo richiesto.

Esempio 5.6.2: Utlizzo concreto dell'algoritmo di Stride Scheduling

Supponiamo di aver scelto W = 10,000 e di avere 3 processi con le seguenti caratteristiche:

- A: 100 ticket, $\text{stride}_A = 100$. Utilizzo atteso della CPU = 25%.
- B: 50 ticket, $\text{stride}_B = 200$. Utilizzo atteso della CPU = 12.5%.
- A: 250 ticket, $\text{stride}_A = 40$. Utilizzo atteso della CPU = 62.5%.

Tick CPU	Vincitore	pass_A	pass_B	pass_C
–	(inizio)	0	0	0
1	A (parità; A scelto per primo)	**100**	0	0
2	B (parità B/C; B scelto)	100	**200**	0
3	C	100	200	**40**
4	C	100	200	**80**
5	C	100	200	**120**
6	A	200	200	120
7	C	200	200	**160**
8	C	200	200	**200**
9	A (parità a tre)	**300**	200	200
10	B	300	**400**	200

Dalla tabella vediamo che, anche su un piccolo numero di esecuzioni (10 in totale) la probabilità associata ad ogni processo è pressocché rimasta (basta effettuare un breve calcolo).

5.6.4 Linux CFS

Nelle sezioni precedenti abbiamo parlato in un esempio del funzionamento del sistema di scheduling di Linux, il cosiddetto $O(1)$. Per quanto tale mecceniasmo fosse a dir poco **geniale**, presentava criticità soprattutto per quanto riguardava la semplicità implemen-

tativa (bisognava gestire 140 code, e si avevano due array ognuno contenente entrambe le code) e l'effettiva iniquità con un grande numero di processi.
Per questo nella versione del kernel 2.26.23 tale algoritmo di scheduling è stato sostituito con un algoritmo detto **Completely Fair Scheduler (CFS)**.

L'idea di fondo di questo algoritmo è semplice, e il suo nome diventa dunque particolarmente intuitivo: fare in modo che ogni processo abbia la stessa importanza per la CPU. Dunque, se abbiamo n processi, ad ognuno deve essere garantito un tempo pari a $\frac{1}{n}$ sulla CPU.
Dato che non possiamo eseguire tutti i processi simultaneamente, l'idea è quella di tenere traccia di quanto tempo di CPU ogni processo ha utilizzato finora, ed eseguire sempre quello che ne ha impiegato meno degli altri. Questo è possibile grazie ad una grandezza chiamata `vruntime` (virual runtime) che indica appunto quanto tempo un determinato processo ha passato sulla CPU.
Tale grandezza si ottiene con la seguente formula:

$$\text{vruntime}_i += \Delta t \cdot \frac{W_0}{w_i}$$

dove:

- Δt indica per quanto tempo il processo finora è stato effettivamente in esecuzione.
- W_0 indica il peso del processo che ha valore nice $= 0$. (ricordiamo la niceness essere il sistema con cui Linux misura la priorità di un processo). Si tratta di un parametro associato in modo univoco al valore di nice del processo.
- w_i rappresenta il peso del processo corrente.

Come abbiamo detto prima, il concetto di base è che ogni volta che un processo viene eseguito dalla CPU, in base alla formula precedente calcoliamo per quanto tempo è stato eseguito. Dato che vogliamo mantenere l'equità tra i processi, volendo dunque garantire che ognuno goda dello stesso amount di CPU degli altri nel complesso, ogni volta che dobbiamo scegliere il processo da eseguire prendiamo quello con il `vruntime` più basso, poiché sarà quello che ha usato il quantitativo minore di CPU rispetto agli altri.

Osservazione 5.6.6: Come viene effettivamente implementato?

Ci accorgiamo che dunque bisogni mantenere un ordinamento tra i vari processi in esecuzione, in base al loro valore di `vruntime`. Per fare ciò, non ci conviene utilizzare una struttura dati come un Array (in quanto sappiamo che gli algoritmi di sorting sono particolarmente costosi) bensì una struttura dati ad albero, come ad esempio

gli AVL oppure, nello specifico caso dell'algoritmo CFS gli **alberi rosso-nero**. Non entreremo nel dettaglio della loro implementazione, ma basti sapere che questi sono un particolare tipo di Albero Binario di Ricerca (ABR) **bilanciato**, garantendo dunque **operazioni di modifica e ricerca in tempo logaritmico** rispetto al numero dei nodi.

Infine, per l'algoritmo CFS vengono definiti due ulteriori parametri:

Target Scheduling Latency: ovvero un limite sul tempo di risposta, cioè un parametro che indica entro quanto tempo ogni processo deve essere eseguito almeno una volta (default 6ms).

Minimum Granularity: parametro che indica il minimo quanto di tempo che hanno i processi per essere eseguiti prima di un context switch (default 0.75ms).

6

La virtualizzazione della memoria

Avendo trattato le varie politiche di scheduling, riguardanti dunque la virtualizzazione della CPU facendo credere ai processi di avere tutto per sé e dando l'illusione all'utente del parallelismo, è giunto il momento che il sistema operativo si occupi di gestire la memoria. L'insieme delle operazioni di allocazione, deallocazione e tutte le politiche decisionali secondo queste avvengono fanno parte del macro-argomento della **virtualizzazione della memoria**.

Nei primi tempi in cui nacquero gli elaboratori il problema di virtualizzare la memoria non si poneva: si aveva un solo processore che poteva eseguire solo un processo alla volta e nessuno scheduling. Quel processo poteva dunque accedere a tutte le aree di memoria che voleva. Anche a quelle kernel, se non si stava attenti.
Insomma, ci rendiamo conto che questo oggi non sia e non debba più essere possibile per due motivi:

1. Potremmo avere multipli processi in esecuzione nella stessa coda, e bisogna dunque gestire il cambio di contesto tra questi processi in maniera corretta.

2. Bisogna garantire la protezione dei reciproci spazi di memoria: un processo A non dovrà **mai** poter scrivere sull'area del processo B (salvo ciò sia intenzionale), né tantomeno nell'area Kernel.

Il processo di virtualizzazione della memoria si occupa di mantenere dunque i seguenti principi:

Trasparenza: i processi devono essere agnostici. Un processo non deve sapere il suo indirizzo di memoria reale: per lui il suo spazio di indirizzamento inizia sempre a `0x00...0`.

Efficienza: la virtualizzazione deve essere veloce, in quanto è un'operazione interpellata praticamente sempre, ed efficiente in termini di spazio. Per questo motivo implementiamo il tutto anche mediante dei dispositivi hardware (es: Memory Management Unit).

Protezione: un processo non deve essere in grado di accedere in alcun modo ad aree di memoria che non lo concernono.

Osservazione 6.0.1: Gli indirizzi in C

Quando mandiamo a schermo l'indirizzo di una variabile mediante l'operatore `&`, entra in gioco la virtualizzazione: vediamo puramente l'indirizzo logico, in quanto il processo non deve poter avere la minima idea di dove è memorizzato fisicamente all'interno della RAM. A lui interessa che, almeno secondo la sua percezione, il suo spazio di indirizzamento sia contiguo.

Osservazione 6.0.2: Duplicazione di indirizzi logici

Dato che ogni processo virtualmente parte dall'indirizzo `0x00...0` è molto probabile che due processi distinti abbiano una variabile entrambi memorizzata allo stesso indirizzo `0x43617`. Si specifica solo per completezza che quello **non è lo stesso indirizzo fisico**, in quanto lo stesso indirizzo logico può tranquillamente essere mappato a due indirizzi fisici diversi.

6.1 Un primo approccio: l'allocazione statica

Prima di passare a tecniche più complesse come il base and bounds e l'introduzione di dispositivi hardware per assistere il processo di virtualizzazione, la tecnica era molto più terra-terra: prendere ogni indirizzo logico, applicargli un **offset** ed ecco ottenuto il nostro indirizzo fisico definitivo. Inoltre, l'offset in sé non è niente di complesso: rappresenta banalmente l'indirizzo fisico in cui **inizia** il processo.
Dunque, se un processo inizia ad indirizzo `0x3000`, ad ogni indirizzo logico (che sia di istruzione, variabile, ... insomma appartenente alla gerarchia che avevamo visto col C) bisogna aggiungere `0x3000`.

Ci rendiamo conto che come modalità sia estremamente semplice da implementare, e dunque vantaggiosa secondo questo punto di vista, tuttavia soffre delle seguenti criticità:

1. **Nessuna protezione sugli indirizzi acceduti**: il processo A potrebbe tranquillamente venir erroneamente mappato ad un'area di memoria riservata al processo

B e non vi sarebbe alcun controllo in merito.

2. Spostare un processo da un'area di memoria all'altra è semplicemente un inferno: bisognerebbe copiare tutta l'area di memoria virtuale, copiare ogni singolo indirizzo fisico associato ad ogni variabile/istruzione, copiare gli indirizzi dei registri della CPU e poi spostare tutto nella nuova locazione. E' un processo così complesso che tanto vale non farlo.

6.2 Un approccio migliorato: il Base & Bounds

Questo nuovo approccio non si basa su una traduzione diretta degli indirizzi (quindi non basta applicare semplicemente un offset) bensì si affida anche ad un componente hardware chiamato **Memory Management Unit (MMU)** nella sua versione ancora "embrionale" (capiremo nelle sezioni future il perché di questo aggettivo). L'affidarsi ad un componente hardware è una soluzione ottimale in quanto garantisce la **separazione delle responsabilità**: la CPU non deve effettuare controlli ma delega tutto il lavoro alla MMU che si occuperà semplicemente di mandargli il risultato.

Per quest'implementazione, è sufficiente definire due registri all'interno della MMU:

- Il registro **base**, che contiene al suo interno l'**indirizzo al quale inizia l'area di memoria riservata al processo** (simile all'offset di prima dunque).
- Il registro **bounds**, che contiene al proprio interno la **dimensione dello spazio di indirizzamento relativo al processo**. Se un processo avesse uno spazio di indirizzamento virtuale pari a `16KB`, all'interno del bounds register sarebbe riportato tale valore.

Osservazione 6.2.1: Le assunzioni teoriche

E' doveroso specificare che per modellare questa soluzione stiamo ancora facendo due assunzioni molto importanti e poco realistiche:

1. Lo spazio di indirizzamento virtuale è minore di quello di indirizzamento fisico. Nei sistemi moderni, questo è spesso e volentieri falso, in quanto lo spazio di indirizzamento virtuale corrisponde a $2^{\text{dimensione_parola}}$ byte (che nei sistemi moderni può arrivare ad essere anche un $2^6$4 byte), mentre la memoria RAM ha una dimensione di ormai 34-64GB. Ci rendiamo conto esserci una sostanziale differenza.
   Un buon sistema operativo deve modellare anche questa casistica, ma lo vedremo in futuro mediante la tecnica del paging.

2. Lo spazio di indirizzamento relativo ad un processo deve essere contiguo anche in memoria RAM. Questa è un'assunzione forte, in quanto ripetiamo essere pressoché impossibile a causa delle dimensioni dello spazio di indirizzamento rispetto a quello effettivo. Inoltre potrebbero verificarsi casi in cui un processo **per intero** non riesce ad entrare in memoria centrale, ma spezzettandolo si riesce. Anche questo verrà risolto col paging.

La formula di traduzione risulta dunque immediata:

$$\text{indirizzo_fisico} = \text{indirizzo_virtuale} + \text{base}$$

e bisogna poi controllare che indirizzo_virtuale $<$ *bounds*, per verificare che l'indirizzo generato dal processo rimanga dentro i limiti imposti dalla MMU.

Facciamo un esempio di un processo con base = 32 KB, limite = 16 KB:

```
1  // Codice C (semplificato):
2  int x = 3000;        // memorizzato all'indirizzo virtuale
       15 KB (stack)
3  x = x + 3;           // carica x, aggiunge 3, memorizza x
```

Compilato in tre istruzioni x86 agli indirizzi virtuali 128, 132, 135:

1. Prelievo dell'istruzione @ virtuale 128:
   PC = 128 → fisico = 128 + 32.768 = 32.896 ✓→ prelevata dalla RAM
2. Esecuzione: `movl 0x0(%ebx), %eax` (caricamento da virtuale 15 KB = 15.360):
   15.360 < 16.384 ✓→ fisico = 15.360 + 32.768 = 48.128 → lettura da RAM
3. Esecuzione: `addl $0x03, %eax` (operazione pura su registri – nessun accesso alla memoria)
4. Esecuzione: `movl %eax, 0x0(%ebx)` (scrittura su virtuale 15 KB = 15.360):
   15.360 < 16.384 ✓→ fisico = 15.360 + 32.768 = 48.128 → scrittura in RAM

Osservazione 6.2.2: Il valore del registro bounds

Alcune interpretazioni vogliono che all'interno del registro bound sia contenuto il valore dell'indirizzo finale al quale può accedere il processo (insomma quello che per la nostra implementazione attuale sarebbe il risultato della somma `base + bounds`).

E' inutile dire che la decisione del valore dei registri base e bounds spetta al sistema operativo, dunque tutte le operazioni di questo genere sono riservate alla Kernel mode (Ring 0).

Osservazione 6.2.3: Come funziona per multipli processi

Con questa implementazione, ad ogni processo sono associati i rispettivi valori dei registri base e bounds. Così facendo, stiamo aggiungendo un altro parametro da memorizzare all'interno del PCB per permettere di poter ripristinare anche la corretta area di lavoro del processo a seguito del context switch.

Con questa implementazione, anche il problema di spostare un processo da un'area di memoria all'altra è molto più semplice: basta copiare i contenuti dello spazio di indirizzamento e cambiare semplicemente il valore del registro base (dato che il bounds nella nostra implementazione rappresenta l'ampiezza dello spazio di memorizzazione, dunque rimane quello).
Affinché ciò sia possibile, dobbiamo spostare il processo in un'area di memoria libera. A tal proposito, il sistema operativo mantiene memorizzato un array di aree di memoria, in cui per ogni area di memoria libera viene memorizzato a che indirizzo inizia e per quanti indirizzi dura (è come il concetto base & bounds ma applicato alle aree di memoria libere).

Con l'implementazione del meccanismo Base & Bounds, il sistema operativo deve gestire 4 scenari chiave.

6.2.1 Creazione di un processo

Quando un nuovo processo viene creato, il sistema operativo deve fare in modo che questo venga allocato correttamente (e contiguamente, per le nostre assunzioni attuali) in memoria. Lo spazio da allocare in memoria, bene o male, può essere determinato dalla compilazione e dall'analisi del codice macchina del processo.
A quel punto, bisogna cercare uno slot libero in cui posizionare un processo. Si possono seguire 3 politiche nella ricerca e posizionamento di tale processo:

First-fit: si posiziona il processo nella prima area di memoria che riesce a contenere il suo spazio di indirizzamento.

Best-fit: si posiziona il processo nell'area più piccola capace di contenerlo.

Worst-fit: si posiziona il processo nell'area di memoria libera più grande.

Dopo aver deciso dove posizionare il processo in memoria, il SO si occupa di copiare tutti i dati relativi allo spazio di indirizzamento virtuale nell'area di memoria selezionata. Successivamente dovrà valorizzare anche i registri base e bounds.

6.2.2 Terminazione di un processo

Una volta che il processo termina la propria esecuzione, deve essere:

1. Rimosso dalla coda dei processi pronti.
2. Impostare l'area di memoria occupata dal processo come libera. Nel pratico questo viene fatto leggendo i valori dei registri base e bound e inserendo tali valori nell'Array delle aree di memoria libere.
   Facendo un esempio, supponiamo di avere un processo con base = 32KB e bounds = 16KB. Una volta terminato, il sistema si occuperà di fare `aree_memoria_libere.add(start=32KB, size=16KB)`.
3. Controlla se la regione appena inserita è adiacente ad altre regioni libere: in tal caso viene fatta la cosiddetta **fusione** delle aree di memoria, utile per segnalare tutta la sezione come un'unica area libera e prevenire un minimo la frammentazione.

6.2.3 Cambio di contesto tra processi

Riprendendo brevemente quanto già detto prima, quando la CPU deve spostare l'esecuzione da un processo A ad un processo B, bisogna memorizzare nel PCB del processo A anche i valori dei suoi registri *base* e *bounds*. Fatta questa memorizzazione (insieme a tutti gli altri dati relativi al processo, ovviamente) verrà eseguito il processo B "sovrascrivendo" i precedenti valori dei registri base&bounds del processo A con i suoi.

6.2.4 Spostare un processo da un'area di memoria all'altra

Come detto prima, bisogna semplicemente modificare il valore del registro *base* e cercare una locazione di memoria contigua di *bounds* bytes all'interno dell'array `aree_memoria_libere`.

6.2.5 La frammentazione: un problema critico

In una situazione realistica, avendo processi di dimensione variabile, è probabile che a seguito di molte allocazioni e de-allocazioni si creino piccole aree di memoria libere in cui non possiamo memorizzare i processi in quanto troppo grandi per quell'area (e sempre per l'assunzione di mantenere la contiguità). Il problema della **frammentazione** nasce dal fatto che **potremmo avere tante piccole aree di memoria libere che nel complesso**

consistono in tanta memoria, che risulta dunque sprecata.
La seguente figura renderà il concetto:

SO 16 KB	LIBERO 8 KB	Proc A 16 KB	LIBERO 4 KB	Proc B 16 KB	LIBERO 20 KB

Figura 6.1: Frammentazione esterna della memoria fisica.

Memoria libera totale: $8 + 4 + 20 = 32$ KB
Se adesso dovesse entrare un processo richiedente 24 KB di memoria, complessivamente avremmo lo spazio, ma non contiguamente, e dunque non potremmo posizionarlo.

Oltre a questo problema della *frammentazione esterna*, vi è anche il problema della *frammentazione interna*. Ora come ora, noi per ogni processo allochiamo una quantità di spazio prefissato calcolata dalla compilazione del programma, e dunque allochiamo una quantità di memoria ad esempio per lo heap, che **inizialmente sarà vuoto**. Ci rendiamo conto che questo spazio di memoria libero potrebbe risultare utile per altri processi, tuttavia appartenendo al "blocco" del processo corrente non è possibile rendere quella sezione di memoria indirizzabile (almeno ad ora).

Processo A: dimensione spazio di indirizzamento = 16 KB

– Codice:	2 KB	(utilizzato attivamente)
– Dati statici:	1 KB	(utilizzato attivamente)
– Heap:	0 KB	(attualmente vuoto – crescerà successivamente)
– Stack:	1 KB	(attualmente utilizzato)
– Spazio libero:	12 KB	(tra la cima dell'heap e il fondo dello stack)

Figura 6.2: Ripartizione dello spazio di indirizzamento per il Processo A.

Osservazione 6.2.4: La condivisione di aree di memoria

E' utile notare come, con questa tecnica del base & bounds, far condividere un'area di memoria a due processi sia pressoché impossibile: dato che ad ogni processo è associata un'area di memoria, quando si prova ad accedere a qualcosa fuori da quello *scope* risulta un errore. Non viene effettuato alcun controllo sulla natura dell'area di memoria, se questa sia condivisa o meno (pensiamo a quando creiamo un sottoprocesso con `fork()`: la sezione di memoria `code` è inizialmente condivisa tra i due processi, dunque questo con il base & bounds classico non sarebbe possibile).

6.3 La segmentazione

Nella sezione precedente abbiamo discusso dei vantaggi di una tecnica come il Base & Bounds, ma anche e soprattutto dei suoi svantaggi (o meglio, delle sue limitazioni). Tra queste, quella forse più evidente, è la *frammentazione interna della memoria*, causata dal fatto che stiamo allocando un singolo blocco monolitico per tutto lo spazio di memoria virtuale riservato al processo.
L'idea della segmentazione nasce proprio per scollegarsi da quest'idea di mantenere un blocco unico per tutto il processo, e suddividere il tutto in regioni indipendenti, chiamate appunto **segmenti**. In riferimento all'organizzazione effettiva della memoria virtuale di un processo, avremo dunque **un segmento dati per la memoria heap**, **uno per lo stack** e **uno per il codice** (omettiamo per semplicità i segmenti dei dati, ma il concetto è uguale).

Avendo dunque un segmento separato per ogni sezione di memoria, dovremo tenere traccia della lunghezza di questi segmenti: ognuno di questi dovrà avere un registro *base* e un registro *bounds*, il cui funzionamento è sempre lo stesso: mantenere l'indirizzo di memoria del segmento e la sua dimensione. A tal proposito, per ogni processo non ci sarà più solo una coppia di registri, bensì dovremmo memorizzare per ogni segmento i valori dei suoi registri. Questo si ottiene grazie alla **Segment Table**, una tabella di valori memorizzata nel PCB del processo che contiene, per ogni entrata (che rappresenta un segmento), i dati relativi a quella singola area di memoria.

Esempio 6.3.1: Esempio di Segment Table

Seg	Base	Bounds	Protection
Code	32,768	2,048	R-X
Heap	36,864	3,072	RW-
Stack	28,672	2,048	RW-

Per ogni segmento, vengono memorizzati il base, il bounds e il livello di accesso consentito.

- **Read (R)**: accesso consentito in lettura.
- **Write (W)**: accesso consentito in scrittura.
- **Execute (X)**: è possibile eseguire questo blocco.

Intuitivamente, come prima durante il context switch venivano copiati i valori dei due registri base e bounds, stavolta verrà copiata tutta la Segment Table.

Osservazione 6.3.1: Condivisione di aree di memoria

Siccome adesso non abbiamo più un unico blocco monolitico, bensì una sezione di memoria dedicata ad ogni singola area del processo, anche la condivisione del codice tra due processi sarà semplicissima. Supponiamo che il processo B abbia lo stesso codice del processo A: sarà sufficiente copiare i valori di base e bounds del segmento `code` di A all'interno del segmento `code` di B.

Questa tecnica risolve pienamente il problema della segmentazione interna: se con il base & bounds classico avevamo un unico blocco allocato, e dunque il *gap* tra heap e stack rimaneva di fatto memoria inutilizzata, stavolta abbiamo delle sezioni ad-hoc per queste aree, e dunque nessun problema di frammentazione interna.

6.3.1 Traduzione degli indirizzi con la segmentazione

Quando un indirizzo virtuale viene generato all'interno del processo, ricordiamo appunto che questo è "locale" all'istanza del processo stesso, dunque dovrà essere tradotto in un apposito indirizzo fisico da riferire. Avendo adesso però segmenti diversi, bisogna capire **a quale segmento stiamo facendo riferimento** con quell'area di memoria.
La soluzione? Avendo 3 segmenti distinti, **utilizziamo i primi 2 bit più significativi dell'indirizzo** (per rappresentare il numero 3 in binario sono richiesti 2 bit) **per selezionare il segmento a cui ci stiamo riferendo**, e i seguenti bit saranno l'offset dell'effettivo indirizzo da memorizzare all'interno di quel segmento.

Esempio 6.3.2: Traduzione degli indirizzi con segmentazione

Indirizzo virtuale (14 bit):

Segmento	Offset
2 bits	12 bit

Codifica dei segmenti:

00 → Code segment
01 → Heap segment
11 → Stack segment
10 → (unused / invalid)

(La codifica dei segmenti ovviamente è scelta da noi, non ci sono regole specifiche)

Esempio: VA = 01000000001100 = 0x100C

Bit identificanti il segmento = 01 → Heap
Offset = 0x00C = 12
→ Andiamo alla base dello heap e aggiungiamo "offset" bytes.

Osservazione 6.3.2: Stack e segmentazione

Ricordiamo, come già detto in precedenza, che la memoria stack cresce "verso il basso" (ossia non si muove verso indirizzi più grandi bensì verso quelli più piccoli). Dunque non possiamo applicare la regoletta classica del base & bounds di sommare l'offset, bensì in questo caso dovremmo sottrarlo. Si tratta di una piccolezza, e a livello implementativo non è neanche così complesso, solo che sembrava corretto fare questo appunto.

Esempio 6.3.3: Algoritmo di traduzione VA -> PA

```
1       function translate(VA, access_type):
2         seg    = extract_segment_bits(VA)
3         offset = extract_offset_bits(VA)
4
5         entry = segment_table[seg]
6
7         if entry.valid == 0:
8           raise SegmentNotPresentFault
9
```

```
        if entry.direction == NEGATIVE:
          neg_offset = offset - max_segment_size
          if abs(neg_offset) > entry.bounds:
            raise SegmentBoundsFault
          PA = entry.base + neg_offset

        else:  // Crescita positiva
          if offset >= entry.bounds:
            raise SegmentBoundsFault
          PA = entry.base + offset

        if not check_protection(entry.prot, access_type):
          raise ProtectionFault

        return PA
```

6.3.2 Doveri del SO con la segmentazione

Con questo nuovo "metodo" di traduzione degli indirizzi, dobbiamo definire anche il comportamento del Sistema Operativo nei confronti di alcune operazioni fondamentali che riguardano il ciclo di vita di ogni processo.

Context-Switch: come accennato prima, stavolta non si hanno più un singolo registro base e un singolo registro bounds per ogni processo, bensì ognuno di essi ha una Segment Table. Dunque il PCB di ogni processo dovrà prevedere un campo `SegmentTable` all'interno del quale memorizzare i dati relativi ad ogni processo.

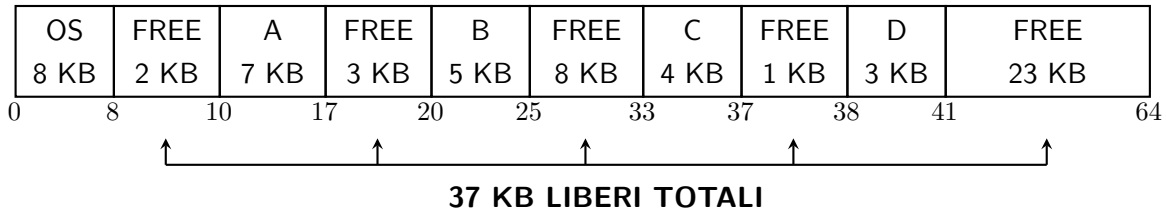
Gestire la crescita dei segmenti: questo può verificarsi soprattutto con segmenti di memoria dinamici come lo heap. Quando un processo tenta di allocare memoria con `malloc()` ma lo heap è pieno, bisogna cercare in tutti i modi di allocare quella memoria. Il flusso che si segue è dunque questo.

1. Viene chiamata implicitamente la funzione `sbrk(increment)` che serve a richiedere un incremento di `increment` bytes alla memoria heap. Ovviamente sottobanco questa funzione nasconde una chiamata di sistema.
2. Il SO controlla se esiste memoria adiacente alla fine corrente dello heap:
   - Se si, espando lo heap e ho risolto la richiesta.

- Altrimenti, si deve cercare un blocco di memoria della dimensione `dim_heap + incremento` (dato che siamo sempre sotto l'assunzione che i singoli segmenti debbano essere contigui in memoria). Se tale blocco viene trovato, lo spazio attualmente occupato dallo heap viene liberato e questo si sposta nella nuova area di memoria; se invece non viene trovato, viene lanciata un eccezione `OutOfMemory`.

6.3.3 La frammentazione esterna

Abbiamo discusso di come l'approccio tramite segmenti risolva il problema della segmentazione interna della memoria, eliminando il *gap* inutilizzato tra memoria heap e memoria stack. Il problema rimane uno già discusso precedentemente: avendo aree di memoria **di dimensione variabile**, ci ritroveremo in una situazione come quella descritta nella figura sottostante, ovvero in cui **complessivamente** abbiamo molto spazio libero, ma è impossibile allocare un segmento in quanto quello spazio libero **non è contiguo**.



L'unica soluzione che il sistema operativo può attuare per garantire l'allocazione è la **compattazione** (o **deframmentazione**) della memoria, che consiste nel unire una dopo l'altra tutte le aree di memoria occupate e lasciare alla fine un unico blocco libero contiguo.

6.4 La paginazione della memoria (paging)

Il problema della frammentazione esterna è dato da un semplice fatto: i segmenti di memoria relativi ad ogni area hanno dimensione diversa. Ed è **matematicamente impossibile** trovare una soluzione che non causi questi buchi in memoria, sotto queste circostanze. Bisogna dunque trovare una soluzione matematicamente compatibile: è così che nasce la tecnica del **paging**, cioè la suddivisione sia della memoria virtuale sia della memoria fisica in blocchi contigui detti **pagine** (per la memoria virtuale) oppure **frame** (per la memoria fisica), che avranno la stessa dimensione per permettere ad ogni pagina di finire dentro ogni frame.

Osservazione 6.4.1: Perché questo risolve il problema?

Suddividere la memoria virtuale e fisica in tante piccole pagine tutte della stessa dimensione garantisce che anche eventuali locazioni di memoria lasciate libere siano sempre della stessa dimensione delle pagine. La frammentazione esterna è così **impossibile**.

A questo punto, intuitivamente, nel PCB di ogni processo non sarà più necessario memorizzare una tabella contenente tutti i segmenti, bensì una tabella contenente tutte le pagine, appunto detta **Page Table**.
In particolare, ogni **pagina virtuale** è identificata da un **Virtual Page Number (VPN)**, mentre una **frame** da un **Physical Frame Number (PFN)**. In ogni entry della tabella viene dunque memorizzata la corrispondenza tra un VPN e il suo PFN.



Figura 6.3: Struttura di un indirizzo virtuale.

La figura rappresenta la struttura di un indirizzo virtuale (VA) secondo la tecnica del paging. Come si può vedere, questo è composto da due campi:

- La **VPN**, dunque l'**identificativo univoco della pagina in memoria virtuale**. Questo valore servirà come chiave per consultare la Page Table e ottenere il corrispondente frame associato in memoria fisica.
- Una volta trovata la corrispondente pagina in memoria, l'**indirizzo all'interno della pagina a cui il VA effettivamente punta** è dato dall'**offset**, ossia di quanto si deve spostare dall'inizio della pagina.

Esempio 6.4.1: Esempio di traduzione VA -> PA

Ad esempio, se avessimo `VPN = 1` e `offset = 10` dovremo:

1. Prendere in memoria il valore del frame associato a `PageTable[1]`.
2. Una volta trovato tale frame, dobbiamo prendere l'indirizzo a cui questo inizia e aggiungere `offset`. Così avremo convertito completamente da indirizzo virtuale a fisico.

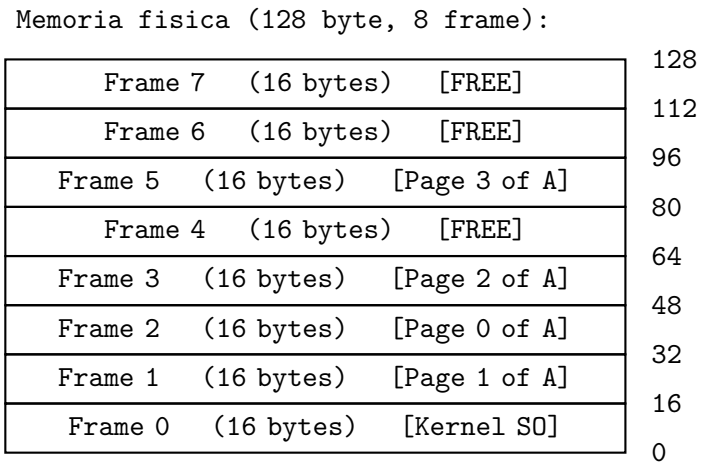


Figura 6.4: Struttura della memoria fisica suddivisa in frame.

Come vediamo, non è neanche richiesta continuità in quanto i frame del processo A possono essere collocati in posizioni completamente staccate o disordinate l'uno dall'altro.

Esempio 6.4.2: Dimensione delle pagine

Supponiamo di avere a disposizione 4GB di memoria RAM, che vogliamo suddividere in pagine da 4KB l'una.
Convertendo tutto in potenze di 2, abbiamo che:

- Capienza RAM: 2^32 byte.
- Dimensione pagina: 2^12 byte.

Per calcolare *quante* pagine possiamo ottenere basta fare una semplice divisione: $2^{32} : 2^{12} = 2^{20}$. Ciò vuol dire che potremo avere fino a 2^{20} pagine distinte. E questo valore, o meglio l'esponente di questo valore (20) rappresenta la lunghezza del campo VPN del Virtual Address, in quanto dovremo garantire che ogni pagina sia univocamente identificata. Abbiamo inoltre già detto che ogni pagina deve essere di 2^{12} byte, e dunque avremo un totale di 32 byte di cui 20 riservati alla VPN e 12 all'offset.

Per quanto questa tecnica sia rivoluzionaria sotto molti punti di vista rispetto alle precedenti bisogna premettere che, almeno per come l'abbiamo definita finora, non è esente da criticità:

1. L'esempio presentato poco fa ci mostra come anche con pagine di 4KB possiamo fare fino a $2^{20} \approx 1$ milione di pagine. Se ogni pagina è fatta da 32 bit questo vuol

dire che la page table potrà costare un massimo di $2^{20} \cdot 2^2 = 2^{22} \approx 4$MB per ogni processo. Se andiamo a scalarlo per un largo numero di processi, questo numero si può alzare, ma se scegliamo pagine di 4KB è un valore anche ragionevole. Se invece fossimo in un sistema a 64 bit e volessimo mantenere comunque pagine da 4KB l'una, risulterebbe in circa 512GB per ogni processo, *solo per la page table*! Ci rendiamo conto essere una cifra esorbitante ed irrealistica anche ad oggi.

2. Per scrivere/leggere un indirizzo dalla memoria servono due accessi consecutivi a questa: uno per trovare il frame fisico associato alla pagina logica, l'altro per effettuare effettivamente l'operazione in memoria (lettura o scrittura che sia). Rispetto al base & bounds o alla segmentazione, questo doppio accesso rappresenta un rallentamento di 2 volte rispetto a prima!

Per risolvere queste due criticità e costruire finalmente la versione definitiva della virtualizzazione della memoria, gli ingegneri del software hanno trovato due soluzioni di cui discutiamo separatamente nelle seguenti sezioni.

6.4.1 Il Translation Lookaside Buffer (TLB)

Al fine di risolvere la prima criticità, legata al numero massivo di accessi in memoria, si è deciso di prendere una strada già vista in precedenza, che ha sempre dato soddisfazioni: aggiungere un componente hardware dedicato all'interno della MMU. In questo caso, tale componente è il **Translation Lookaside Buffer (TLB)**, che non è altro che una memoria cache il cui obiettivo è memorizzare tutte le conversioni recenti VPN -> PFN.

Essendo una memoria cache, il TLB può contenere solitamente poche entrate, ma l'accesso in scrittura e lettura sarà praticamente immediato.
Ogni volta che viene utilizzata una nuova pagina dalla memoria, la coppia VPN -> PFN, assieme ad altri valori, viene inserita all'interno della TLB.
Alla richiesta di un frame di memoria, questo viene prima cercato nella TLB:

- Se tale frame viene trovato, allora si procede con l'operazione da eseguire in memoria.
- Altrimenti, è necessario cercare tale frame nella memoria RAM, che come sappiamo richiederà un tempo più lungo. Tuttavia, una volta trovato questo frame, potremo memorizzare la corrispondenza all'interno della cache.

Proposizione 6.4.1: Address Space Identifier (ASID)

All'interno di una TLB Entry vengono memorizzati vari dati, oltre alla corrispondenza tra pagina virtuale e frame fisico. Uno di questi dati è il cosiddetto **Address**

Space Identifier (ASID), che serve ad identificare quella pagina a quale processo appartiene.
Questo parametro risulta particolarmente utile perché in questo modo durante il context switch non ci sarà bisogno di cancellare tutte le entry della TLB, bensì di mantenerle tutte e controllare solo quelle il cui ASID è uguale all'ASID del processo corrente.

La soluzione mediante TLB è **estremamente efficiente**, in quanto risulta quasi sempre in un hit e quasi mai in un miss. Vediamo con il seguente esempio come questi concetti si legano al principio della località temporale visto nel primo capitolo.

Esempio 6.4.3: Tasso di hit del TLB

```
int sum = 0;
for (int i = 0; i < 1000; i++) {
  sum += array[i];     // sum si trova sullo stack,
     sempre nella stessa pagina
}
```

- Variabile `sum`: sullo stack, vi si accede 1000 volte → stesso VPN → caricata nel TLB una volta, 999 hit.
- `i`: stessa pagina dello stack → hit a ogni iterazione dopo il primo miss.
- `array[i]`: località spaziale — 1000 interi occupano 1-2 pagine → solo 1-2 miss del TLB per 1000 accessi.

Miss totali per questo ciclo: 3-4 (2 miss per lo stack + 1-2 miss per l'array).
Accessi totali: ∼3000 (fetch delle istruzioni + accessi ai dati), per dunque un Hit rate $> 99.9\%$.

Per sapere quanti dati relativi ad un processo un TLB può effettivamente mantenere, è sufficiente applicare la seguente formula:

$$\text{copertura_totale_TLB} = \text{numero_entrate_TLB} \times \text{dimensione_singola_pagina}$$

Se ci manteniamo sulle precedenti pagine da 4KB e supponiamo di avere un TLB da massimo 64 entrate, avremo un working set di circa 256KB, che di solito è abbastanza per contenere tutti i dati di un programma medio-piccolo, e dunque si avranno pochi TLB

Miss.
Tuttavia, se provassimo ad accedere ad un numero di pagine *maggiore* alla capienza massima del TLB, avremo un costante TLB Miss, dato che le pagine verrebbero periodicamente sostituite (dopo aver caricato 64 pagine, se vogliamo metterne un'altra dobbiamo rimuovere la prima). Per risolvere tale fenomeno, chiamato **TLB Trashing** una soluzione è quella di aumentare la dimensione della singola pagina: attenzione però al fatto che le pagine devono essere contigue in memoria, e dunque pagine più grandi significa anche possibilità di frammentazione interna maggiore.

Una formula utile per alcuni esercizi può essere quella per il calcolo dell'**Effective Memory Access Time**, che indica qual'è il tempo effettivo di accesso alla memoria in base ai parametri della cache TLB.

$$\text{EMAT} = h \cdot (T_{\text{TLB}} + T_{\text{mem}}) + (1 - h) \cdot (T_{\text{TLB}} + T_{\text{PT}} + T_{\text{mem}})$$

dove:

- T_{TLB} rappresenta il tempo necessario ad **accedere al TLB**.
- T_{PT} indica il tempo necessario ad **accedere la pagina nella Page Table in memoria RAM**.
- T_{mem} indica il tempo di **accesso dell'effettivo indirizzo in RAM**.
- h rappresenta il **TLB hit-rate** (dunque un valore compreso tra 0.0 e 1.0).

6.4.2 Multi-Level Page Tables

Per risolvere il secondo problema, quello del massivo numero di pagine ottenute a causa dello spazio virtuale praticamente infinito (agli occhi del singolo processo), si può operare stavolta a livello logico, modificando l'implementazione della Page Table.
Prima di procedere con la spiegazione, vediamo però un esempio:

Osservazione 6.4.2: Spreco di memoria per le pagine di un proceso

Processo a 32-bit, pagine da 4 KB $\rightarrow 2^{20} = 1.048.576$ voci PTE.
Ogni PTE = 4 byte $\rightarrow$ Page table = 4 MB
Ma quante pagine usa effettivamente un processo tipico?

Code:	∼500 KB	→	∼125 pagine
Data:	∼100 KB	→	∼25 pagine
Heap:	∼1 MB	→	∼256 pagine
Stack:	∼64 KB	→	∼16 pagine
TOTALE:	∼422 pagine utilizzate attivamente		

La tabella delle pagine piatta alloca 1.048.576 voci a fronte di sole 422 pagine utilizzate. Il 99.96% delle voci della tabella delle pagine risulta NON VALIDO (valid bit = 0). Il 99.96% di 4 MB sprecati per PTE non valide.

L'idea di fondo per risolvere questo problema è quella di **allocare solo le pagine che ci servono davvero**.
Nel pratico, questo viene fatto nel seguente modo:

1. Per ogni processo, si definisce un Array chiamato **Page Directory**.
2. Ogni elemento di questo Array è detto **Page Directory Entry (PDE)**. Ad ogni PDE è associata **una e una sola Page Table**, che contiene le pagine di quel processo contenute in un certo range.
3. Se il valore associato ad una PDE viene marcato come *non valido*, ci basta **non allocare le pagine di quel range**, e avremo risparmiato moltissima memoria!

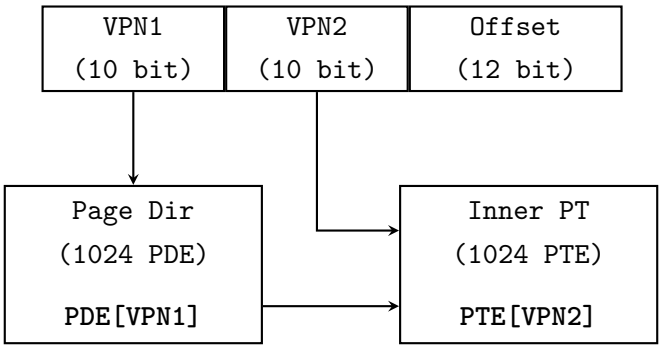


Figura 6.5: Traduzione dell'indirizzo con tabella delle pagine a due livelli.

Dato un indirizzo virtuale a 32 bit suddiviso in:

```
[ PDI (10 bit) | PTI (10 bit) | Offset (12 bit) ]
```

Il meccanismo di traduzione hardware (MMU) si divide ora nei seguenti passi:

1. **Accesso alla Page Directory (1° Livello):**

- La MMU estrae i primi 10 bit più significativi dell'indirizzo virtuale, che rappresentano l'indice della PDE che punta alla Page Table in cui è contenuta la pagina puntata dall'indirizzo.
- Calcola l'indirizzo della voce corrispondente:

$$\text{Indirizzo PDE} = \text{Indirizzo di base PageDirectory} + (\text{Indice Pagina} \times 4 \text{ byte})$$

- Legge la **PDE**:
  - Se il bit di validità è 0, la tabella di secondo livello non esiste in memoria → viene sollevata un'eccezione hardware (**Page Fault** / *Segmentation Fault*).
  - Se il bit di validità è 1, la PDE contiene il PFN (*Page Frame Number*) in cui risiede la Page Table di secondo livello.

2. **Accesso alla Page Table (2° Livello):**
   - La MMU estrae i successivi 10 bit dell'indirizzo virtuale.
   - Calcola l'indirizzo della **PTE** (*Page Table Entry*) all'interno della Page Table individuata al passo 1:

$$\text{Indirizzo PTE} = \text{Base Page Table} + (\text{PTI} \times 4 \text{ byte})$$

   - Legge la **PTE**:
     - Se il bit di presenza/validità è 0, la pagina dati non è caricata in RAM → scatta un **Page Fault** (il kernel caricherà la pagina dal disco).
     - Se il bit è 1, si verificano i permessi di accesso (R/W/X). Se validi, si estrae il PFN del frame fisico contenente i dati.
3. **Calcolo dell'Indirizzo Fisico Effettivo:**
   - La MMU combina il **PFN** estratto al passo 2 con i 12 bit di **Offset** originali (invariati rispetto all'indirizzo virtuale).
   - La CPU esegue la lettura o scrittura sul frame di memoria fisica.

In realtà, questa stessa codifica mediante soli 2 livelli risulta ancora non essere abbastanza per eliminare gli sprechi di memoria nei sistemi a 64 bit. Difatti, l'implementazione per tali architetture si basa sull'utilizzo di 4 livelli.

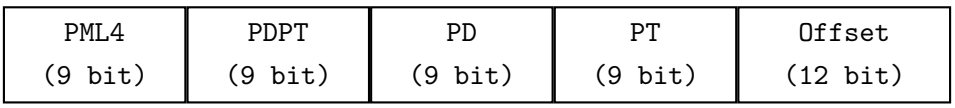


Figura 6.6: Esempio grafico dell'organizzazione del paging in sistemi a 64 bit.

In sistemi che usano Multi-Paging a 4 livelli, abbiamo i seguenti oggetti:

- Page Map Level 4, la radice degli indirizzi associati ad un processo.
- Page Directory Pointer Table, che punta alla page directory in cui si trova l'indirizzo.
- Page Directory, il secondo livello che infatti assume lo stesso ruolo di prima.
- Page Table, che contiene l'effettiva corrispondenza.

6.4.3 Le Page Faults

Come suggerisce il nome, una **Page Fault** è un'eccezione hardware generata dalla MMU quando la CPU tenta di accedere ad una pagina virtuale che non è stata ancora mappata ad un frame fisico in memoria. Quando la CPU tenta di accedere ad una pagina, ne controlla tutti i bit di stato: se il *valid bit* è pari a 0, vuol dire che la pagina non ha ancora una corrispondenza in memoria e dunque deve essere memorizzata (ad esempio nella TLB, come abbiamo già visto).
Quando si scatena una Page Fault:

1. La CPU blocca l'esecuzione corrente, va in Kernel Mode e invoca l'apposito *Page Fault Handler*.
2. Se l'accesso è lecito (dunque non è un problema di allocazione ma di permessi) si procede a cercare un'area di memoria libera in cui mappare la pagina.
3. Si carica la pagina in memoria e si continua l'esecuzione del processo. Durante le precedenti fasi di aggiornamento delle pagine, questo verrà messo in stato `WAITING`.

6.5 Lo Swapping

Abbiamo costruito un sistema della gestione della memoria impeccabile, grazie all'intuizione matematica di suddividerla in blocchi di uguale ampiezza detti *pagine* o *frame*. Tuttavia, abbiamo sempre lavorato sotto una potentissima assunzione: che la memoria centrale sia abbastanza grande per contenere tutte le pagine di tutti i processi in esecuzione al momento. Quest'assunzione è tanto forte quanto falsa, in quanto ad oggi gli spazi virtuali hanno dimensioni pari all'architettura della macchina (circa 2^{48} byte di indirizzamento) mentre le memorie RAM non sono affatto così capienti.
Anche se più processi venissero eseguiti "simultaneamente", cosa succederebbe se ne subentrasse uno più pesante degli altri? Per come siamo rimasti ora, il SO dovrebbe rifiutare nuovi processi oppure uccidere quelli già esistenti. Ma non si starebbe risolvendo il problema, bensì solo evitando.

La soluzione è dunque la seguente: si crea uno spazio in memoria di massa chiamato **Swap Space**, ovvero un'area del disco in cui vengono memorizzate le pagine di un processo che non hanno spazio per stare in memoria RAM. Nello Swap Space, le pagine possono entrare dopo che la RAM è piena, oppure quando deve essere effettuato uno *scambio (swap)* tra una pagina attualmente inutilizzata in memoria RAM e una richiesta in memoria di massa. Lo Swap Space è suddiviso in blocchi, ognuno uguale alla dimensione di una pagina, in modo che si possa stabilire anche qui una corrispondenza biunivoca tra il VPN e il blocco di Swap in tale pagina è contenuta.

Al fine di controllare dove si trovi la pagina in memoria, deve essere aggiunto un altro bit ad ogni Page Table Entry (PTE): il **Present bit** viene messo ad 1 se la pagina si trova in memoria RAM, a 0 se si trova nello Swap.

Osservazione 6.5.1: Ruolo del valid bit

Ricordiamo che nella PTE è presente anche un valid bit, che indica se la pagina è stata mai allocata. La presenza di questo valid bit è fondamentale, in quanto se la pagina non fosse **mai** stata allocata in memoria, verrebbe lanciata un'eccezione e l'OS handler delle Page Fault si occuperebbe di allocarla in qualche modo. Dunque se questo bit vale 1, vuol dire che necessariamente si trova in RAM oppure nello Swap.

Osservazione 6.5.2: Pagina memorizzata in Swap

Quando una pagina viene memorizzata nello Swap, sarà inutile mantenere all'interno della sua PTE l'indirizzo del frame fisico associato. Per questo motivo, quando una pagina viene spostata dalla RAM allo Swap quella sequenza di bit riservata al PFN viene sostituita con l'identificativo del blocco dello Swap in cui verrà messa (**Swap Entry**).

6.5.1 Da disco a RAM

Quando una pagina virtuale deve essere caricata in memoria RAM per poter accedere ai suoi dati, il SO procede nel seguente modo:

1. Si controlla il present bit della pagina: se questo è 0, vuol dire che si deve andare a cercarla nello Swap.
2. All'interno dello Swap si cerca il blocco associato alla pagina (valore recuperato dalla PTE) e si fa il fetch di tale pagina.

3. Si cerca ora un frame libero in RAM su cui memorizzare la pagina.

4. Nel frattempo, dato che in fin dei conti il caricamento di una pagina da memoria fisica è un'operazione di I/O, il processo viene messo in stato di `WAITING`.

Osservazione 6.5.3: Ottimizzazione con il dirty bit

Un ulteriore bit presente all'interno di una PTE è il cosiddetto dirty bit, che indica se quella pagina di memoria sia mai stata modificata o meno. Per ottimizzare il processo di "sfrattamento" di una pagina da RAM a Swap, si preferisce sempre sfrattare pagine con dirty bit = 0.
La ragione per ciò sta nel fatto che le pagine che sono state modificate potrebbero esserlo nuovamente, e dunque eventuali cambiamenti devono essere salvati in memoria. Swappare una pagina *dirty* costa una scrittura in memoria. Per le pagine *clean* invece non è così: se una pagina non è mai stata modificata, probabilmente è *read-only*, e dunque non c'è bisogno di prevedere o memorizzare alcun cambiamento. Swappando una pagina *clean* si risparmia il tempo di una scrittura in memoria, che ricordiamo essere un'operazione piuttosto onerosa, soprattutto in termini di cicli di clock CPU.

Definizione 6.5.1: Tipologie di Page Fault

Il sistema operativo distingue in due classi di Page Fault: **minor fault** e **major fault**. La sostanziale differenza è che le *minor faults* si verificano su pagine che sono già presenti in memoria RAM, e devono essere caricate. Le *major faults*, al contrario, sono quegli errori scatenati da pagine che si trovano nella memoria di Swap e devono essere riportate in RAM.

Come ci si può aspettare, le *major faults* sono più dispendiose (in quanto richiedono un accesso in memoria di massa), e dunque un buon sistema operativo deve minimizzare l'occorrenza di tali eventi con appositi algoritmi.

6.5.2 Politiche di rimpiazzo pagina

Avendo predisposto un'area di memoria fisica in cui memorizzare tutte le pagine che non entrano in memoria RAM, dobbiamo adesso decidere *quali pagine muovere da RAM a disco*.
Quando prendiamo una pagina dalla sua Page Table, significa che ci serve *ora*. Se nella RAM non c'è spazio, dobbiamo in qualche modo decidere quale pagina spostare dalla memoria centrale alla memoria di massa.

Osservazione 6.5.4: Ottimizzazione sui rimpiazzi

E' necessario trovare una buona politica di rimpiazzo per evitare frequenti accessi in memoria di massa, che rallenterebbero di molto l'esecuzione di ogni processo.

The Optimal Policy (OPT)

Una prima soluzione viene fornita dall'algoritmo **Optimal replacement policy**, il cui concetto è estremamente semplice: **rimpiazzare sempre la pagina che verrà utilizzata per ultima rispetto alle altre**. Supponendo di avere la sequenza delle pagine richieste "`7 0 1 2 0 3 0 4 2 3`" e un massimo di 3 frame in memoria, allo step 4 dovrà essere liberato un frame tra i 3 già allocati (`7 0 1`). Come decidiamo quale frame rimuovere dalla memoria? Semplicemente guardando in avanti e vedendo quali verranno utilizzati e quando:

- Lo 0 verrà utilizzato a breve e più di una volta, quindi sarebbe uno spreco liberarlo.
- I frame 7 e 1 non verranno più utilizzati in futuro, dunque possiamo scegliere tra uno dei due.

Chiaramente, questo approccio si fonda su un'assunzione potentissima: sapere a priori quali frame verranno utilizzati e quando. Ecco perché questo approccio risulta essere puramente teorico e funge solo come base per lo studio di approcci futuri e più concreti.

Politica FIFO

Come sempre, il primo approccio che si tenta ogni volta è quello **FIFO**: il SO tiene una coda in cui memorizzare passo passo le pagine richieste, e **quando la coda è piena si prende la pagina in testa e si memorizza in memoria di Swap**. Successivamente si carica in RAM la nuova pagina e si inserisce all'interno della coda. E via così...
Tuttavia, come accade quasi sempre con il metodo FIFO, per quanto intuitivo non è quasi mai la scelta migliore: non tenendo traccia di nulla ma solo di quando una pagina è entrata, non sappiamo se sia una pagina richiesta frequentemente (come ad esempio il codice eseguibile di un processo) oppure una pagina utilizzata una-tantum. Rischiamo dunque di generare tanti *major faults* che sarebbero altrimenti evitabili.

Least Recently Used (LRU)

La formalizzazione pratica della Optimal Policy è l'algoritmo **Least Recently Used (LRU)** che, come suggerisce il nome, decide di ***sfrattare* la pagina che non viene utilizzata da più tempo**. Il principio su cui si fonda l'intuizione alla base di questo algoritmo è quello di *località temporale* di cui abbiamo discusso nei primi capitoli: una

pagina acceduta a $t = 99$ è molto più probabile che sia acceduta nuovamente a $t = 100$ al posto di una acceduta inizialmente a $t = 1$. Un sistema (così come noi umani) non è in grado di prevedere il futuro per determinare quale pagina verrà utilizzata e quando. Tuttavia è pienamente in grado di analizzare comportamenti passati e tracciare statistiche per formulare previsioni.

Questo approccio è più pragmatico e meno utopistico della OPT, tuttavia presente alcune criticità legate all'implementazione hardware: per tale approccio, dovremmo memorizzare per ogni pagina il timestamp in cui è stata acceduta e poi quando la memoria risulta piena sfrattare la pagina con il timestamp più alto. Questo si traduce nell'associare ad ogni pagina un valore e dover scorrere ogni volta tale lista di valori, rimuovere il minimo e inserire un nuovo valore in cima: un classico approccio risolvibile in tempo $O(n)$ che però risulta in uno spreco di memoria sostanzioso.

Clock algorithm

Il **Clock algorithm** nasce per essere una versione più realistica dell'algoritmo LRU.
Il trucco sta nel memorizzare i frame della RAM all'interno di un'Array circolare, che si comporterà esattamente come un orologio. Si sfrutta poi l'*Accessed bit*, ovvero un bit presente nella PTE di ogni pagina, per determinare quale di queste dovrà essere sfrattata:

- Appena una pagina viene acceduta, viene impostato l'Accessed bit = 1.
- Quando si incontra una pagina con Accessed bit = 1, le si da una "*seconda possibilità*", non scartandola subito bensì impostando tale bit a 0.
- Quando si trova una pagina con bit = 0, quella viene **subito scartata** per fare spazio alla nuova pagina da caricare in memoria.

E' necessario fare un'annotazione importante: ogni volta che una pagina viene inserita/rimossa da questo Array circolare lo scorrimento della struttura dati non riprende da capo bensì dall'ultimo frame appena visitato. Le uniche informazioni di cui tenere traccia sono dunque un Array delle pagine (implementato in maniera diversa, ma pur sempre un array è) e un indice indicante la pagina corrente.

6.5.3 Criticità su sistemi reali

Tenendo a mente gli algoritmi visti finora, sorgono delle criticità che un bravo ingegnere del software deve prevedere e risolvere nel modo migliore possibile. Un problema può essere quello del **trashing**: quando ci sono **troppe pagine rispetto alla dimensione della RAM**. Abbiamo già visto che il tutto viene risolto tramite la memoria Swap. Ma mettiamoci nel caso in cui un processo A vada a *sfrattare una pagina riservata al processo*

B. E il processo B faccia lo stesso con il processo C.
Una soluzione a tale problema sarebbe attribuire ad ogni processo un numero massimo di pagine utilizzabili. Ma qui si torna al problema di spreco di memoria: e **se attribuissimo 20 pagine ad un processo che ne usa solo 5**? Bisogna scegliere accuratamente tale valore, ma come si può immaginare non è una semplice impresa.

7

La concorrenza

In questo capitolo caleremo un'altra semplificazione con la quale abbiamo lavorato finora: ogni singolo processo non sarà più una singola unità elaborativa, bensì un insieme di "unità lavorative" che ne condivideranno le risorse e opereranno in modo da rendere l'esecuzione del processo padre più veloce di quanto non possa essere quella sequenziale. Introduciamo così il concetto di **Thread**.

7.1 I thread

Anche se anticipato dall'introduzione di questo capitolo, diamo di seguito una definizione più formale di thread.

Definizione 7.1.1: Thread

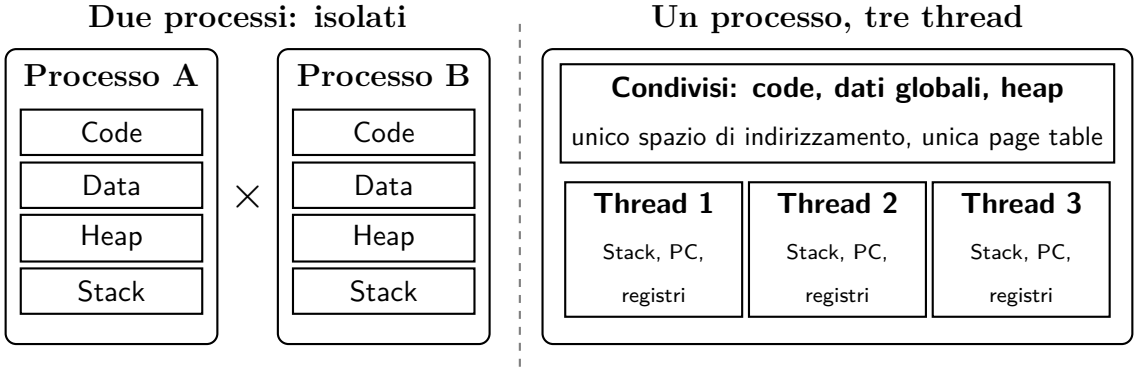
Un **thread** è l'unità di esecuzione più piccola possibile schedulabile dal Sistema Operativo. Un thread può esistere **solo e soltando** in funzione di un processo da cui origina (detto *processo padre* o *main thread*), e per questo ne condivide le risorse fondamentali come lo spazio di memoria riservato al codice eseguibile, le pagine e le variabili globali; possiede invece un proprio stack privato e i propri valori dei registri.

Avendo definito il concetto di thread, sorge spontanea una domanda: era davvero necessario? Dopotutto siamo già in grado di creare processi figli a partire da un processo padre con `fork()`, e siamo in grado di far eseguire al processo figlio un'altra porzione di codice con `execvp()`. A che pro dunque definiamo un'altra unità schedulabile?
Diamo adesso una risposta a tutte queste domande:

1. La creazione di un processo figlio è un'operazione estremamente onerosa dal punto di vista computazionale e di memoria, in quanto si deve creare un nuovo PCB per il

processo figlio, e si dovranno allocare aree di memoria *ex-novo* riservate al processo figlio.

2. I thread, per loro natura, condividono la maggior parte delle aree di memoria con il processo padre. Dunque l'operazione di *context-switch* tra due thread dello stesso processo padre risulta molto semplice in quanto bisognerà modificare un numero limitato di dati.



A questo punto, con l'introduzione dei thread, lo scheduler del SO non si occuperà più di schedulare interi processi bensì i singoli thread dei processi originali. Tuttavia, tutti i discorsi fatti finora sugli algoritmi di scheduling rimangono, cambia solo l'unità di lavoro che viene schedulata.
Come ricordiamo, per i processi avevamo definito i PCB (Processo Control Block), ovvero una struttura contenente tutti i dati utili a ripristinare lo stato di un processo dopo un cambio di contesto. Per i thread definiamo una struttura simile, chiamata però Thread Control Block (TCB), che avrà esattamente la stessa funzione ma conterrà dati leggermente diversi:

- Un identificatore univoco del thread, il **Thread ID (tid)**.
- Lo stato del thread (esattamente come lo stato di un processo: `READY, RUN, BLOCKED`).
- I valori dei registri privati del thread, dunque il proprio **stack pointer**, il **base pointer**, il proprio **program counter** e tutti gli altri registri di calcolo *general purpose*.
- Importantissimo, ogni thread contiene all'interno del suo TCB un puntatore alla struttura PCB del processo padre, in modo da poter accedere ai dati contenuti in quelle aree di memoria condivise tra i singoli thread e definite per il processo main.

Osservazione 7.1.1: Esistenza del PCB

Una domanda arguta potrebbe essere: che senso ha mantenere la struttura PCB per ogni processo se ormai vogliamo suddividerli in sotto-processi schedulabili in modo indipendente?
La risposta si articola in 3 punti:

1. Dobbiamo ancora considerare la situazione di un processo magari molto breve che non è possibile suddividere in thread. A quel punto non staremmo lavorando con un thread bensì con un "classico" processo.

2. Il concetto di thread nasce proprio come *sotto-unità di esecuzione* di un processo padre. Dunque, se non esistesse un riferimento a questo processo padre (il PCB per l'appunto), non avrebbe neanche senso parlare di thread.

3. Un thread può essere creato solo a partire dall'*entry point* di un processo padre (es: la funzione `main()` per quanto riguarda il linguaggio C). Ciò vuol dire che non avremo **mai** un thread che definisce delle variabili globali, poiché è impossibile dal punto di vista strutturale dato che queste vengono definite *prima* della funzione `main()`.

7.1.1 La libreria `pthread`

Il linguaggio C mette a disposizione la libreria `pthread` utile per la creazione e la gestione dei thread a partire da un processo. Vediamo di seguito le varie operazioni messe a disposizione da questa libreria.

7.1.1.1 Creazione di un thread

La creazione di un thread è possibile grazie alla funzione `pthread_create`. Questa funzione prende in ingresso diversi parametri, alcuni dei quali fondamentali per la definizione del thread:

- Il thread ID, a cui dovremo passare l'indirizzo di una variabile di tipo compatibile in cui verrà memorizzato il valore del `tid` attribuito dal SO al thread.
- La **start_routine**, ovvero la funzione che il thread deve eseguire appena verrà schedulato.
- Separatamente, vengono passati anche eventuali **argomenti della funzione start_routine**.

Una nota importante: al termine dell'esecuzione della funzione `create` il thread appena creato verrà subito messo in stato di `READY`, dunque pronto per essere eseguito. L'effettivo momento di esecuzione (`RUN`) dipenderà poi dallo scheduler.

Esempio 7.1.1: Esempio di creazione di un thread in C

```c
#include <stdio.h>
#include <pthread.h>

void *thread_function(void *arg) {
  int id = *(int *)arg;
  printf("Thread \%d: hello from a new thread!\\n",
     id);
  return NULL;
}

int main(void) {
  pthread_t tid;
  int arg = 1;

  pthread_create(&tid, NULL, thread_function, &arg);

  printf("Main thread: created thread \%lu\\n", (
     unsigned long)tid);

  printf("Main thread: thread has finished.\\n");
  return 0;
}
```

Visto l'esempio precedente e facendone una rapida analisi, si può notare un problema riscontrato anche nella creazione dei processi con `fork()`: l'output **non è deterministico**. Non sappiamo quale thread, se il main o il nuovo appena creato, verrà eseguito prima, e dunque non possiamo prevedere l'ordine delle istruzioni.
Tendenzialmente, queste situazioni sono preferibilmente evitabili nella scrittura di un programma, ed è proprio per questo che la libreria `pthread` ci mette a disposizione anche uno strumento per il controllo di queste situazioni.

7.1.1.2 Attesa della fine di un thread

Per risolvere problemi di natura simile a quelli visti sopra, la libreria ci mette a disposizione la funzione `pthread_join()`, che prende come argomento il **tid del thread di cui vogliamo aspettare la fine** e **blocca l'esecuzione del thread che ha chiamato questa funzione fino a quando non sarà terminato il thread specificato**. Questa funzione è molto simile a `waitpid()`, che avevamo visto essere utile per attendere la terminazione di un processo dato il suo Process ID.

Esempio 7.1.2: Esempio di attesa della fine di un thread

```c
#include <stdio.h>
#include <pthread.h>

void *thread_function(void *arg) {
  int id = *(int *)arg;
  printf("Thread \%d: hello from a new thread!\\n",
     id);
  return NULL;
}

int main(void) {
  pthread_t tid;
  int arg = 1;

  pthread_create(&tid, NULL, thread_function, &arg);

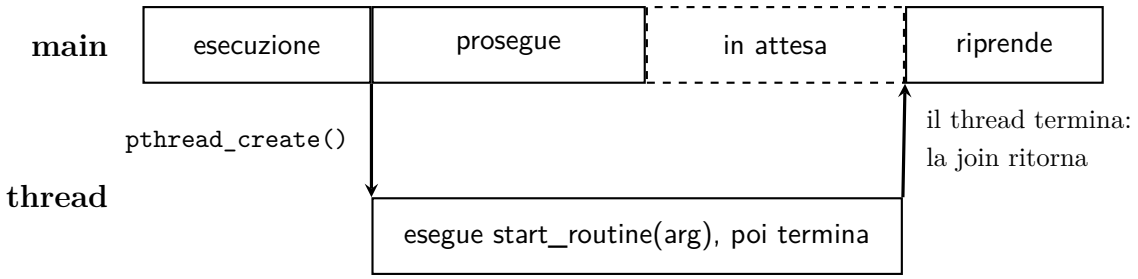
  printf("Main thread: created thread \%lu\\n", (
     unsigned long)tid);

  pthread_join(tid, NULL);

  printf("Main thread: thread has finished.\\n");
  return 0;
}
```

Abbiamo creato il thread all'interno del *flusso principale del programma* (detto anche *main thread*) e dopo di ciò vogliamo attendere che questofinisca prima di riprendere l'esecuzione del main. Otteniamo questo risultando posizionandoci nel

flusso che deve aspettare (in questo caso il main) e chiamando la funzione `pthread_join()`.



7.2 La condivisione della memoria

Purtroppo l'utilizzo dei thread non è tutto fiori e acqua di rose: si creano dei grandi problemi quando più thread tentano di accedere ad una variabile globale e modificarne il valore.
Prendiamo in esempio il seguente programma:

```
1   #include <stdio.h>
2   #include <pthread.h>
3   int counter = 0;
4
5   void *increment(void *arg) {
6     for (int i = 0; i < 1000000; i++) {
7       counter++;
8     }
9     return NULL;
10  }
11
12  int main(void) {
13    pthread_t t1, t2;
14    pthread_create(&t1, NULL, increment, NULL);
15    pthread_create(&t2, NULL, increment, NULL);
16    pthread_join(t1, NULL);
17    pthread_join(t2, NULL);
18    printf("Valore finale del counter: \%d\n", counter);
19  }
```

Ci aspetteremmo come risultato di counter 2000000, dato che abbiamo due thread ognuno dei quali incrementa di un milione. La realtà dei fatti è però deludente: il risultato sperato non viene raggiunto *quasi mai*.

Analizziamo però il *perché* di questo comportamento.
Anche un'operazione apparentemente semplice, come un incremento unario di una variabile, si scompone in 3 istruzioni in linguaggio macchina:

```
1    mov eax, counter
2    add eax, 1
3    mov counter, eax
```

Queste istruzioni si occupano rispettivamente di:

1. Copiare il valore della variabile `counter` all'interno del registro `eax`.
2. Incrementare il valore **nel registro** di 1.
3. Copiare il valore del registro nella variabile `counter`.

Supponiamo ora che il *time-slice* di un thread si esaurisca in mezzo alle ultime due operazioni. In che situazione ci troveremmo: il registro conterrebbe un valore **aggiornato** della variabile, mentre la variabile stessa conterrebbe **il valore vecchio, non aggiornato**. Questo disallineamento dei valori avviene in quanto l'operazione *non è atomica*. Un'operazione si definisce **atomica** quando può essere eseguita consecutivamente senza interruzioni.
Questo disallineamento dei valori, che accade per un singolo thread e dunque fa propagare l'errore anche al processo generale, viene definito come un problema di **race condition**, ovvero una condizione in cui più thread vanno a modificare la stessa variabile globale ottenendo un risultato **non deterministico, bensì dipendente da quale thread verrà schedulato prima e da quando terminerà il time-slice di un singolo thread**.

Definizione 7.2.1: Sezione critica

La parte di codice che può creare disallineamenti delle risorse in thread diversi è detta **sezione critica**.
Per garantire una corretta gestione della sezione critica, e dunque prevenire problemi di race condition, si devono garantire:

- **Esecuzione atomica**, dunque quando un thread entra per eseguire la sezione critica, deve eseguirla **tutta**.

- **Mutua esclusione**: se un thread$_i$ sta accedendo alla sezione critica, **nessun altro thread deve essere in grado di accedervi**.
- **Progresso**: non si può risolvere il problema tagliando la testa al toro e impedendo ad ogni thread di accedere. Staremo garantendo i due punti precedenti, ma staremo anche bloccando permanentemente l'esecuzione del processo.

Osservazione 7.2.1: Natura della sezione critica

Per il principio della mutua esclusione, alla sezione critica può accedere un singolo thread alla volta. Questo vale **anche quando quel thread viene deschedulato**: se per qualche ragione il *time-slice* terminasse all'interno della sezione critica e venisse schedulato un altro thread dello stesso processo, bisognerebbe impedire che quel nuovo thread eseguisse la sezione critica anche se il precedente è stato deschedulato.
Per questa ragione, è molto importante che la sezione critica sia **il più breve possibile**, in quanto bisogna minimizzare il fatto che i processi vadano in `BUSY WAITING`, ossia quando sono formalmente in stato di `READY` bensì stanno solo attendendo di poter eseguire la loro parte di codice.

7.2.1 I lock

Per ovviare questo problema, è stato definito il concetto di **lock**: un **lock** è una variabile condivisa tra thread dello stesso processo, per la quale però solo un thread alla volta avrà "*la chiave per aprirlo*": una volta aperto, il lucchetto sarà chiuso per gli altri ma aperto per il thread che ha usato la chiave. Si tratta ovviamente solo di un concetto astratto, che è stato però implementato in modo concreto dalla libreria `pthread` mediante il tipo di dato `pthread_mutex_t`.
Vediamo di seguito il codice precedente in cui però la sezione critica viene corretta mediante l'uso dei lock:

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  int counter = 0;
5  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
6
7  void *increment(void *arg) {
8    for (int i = 0; i < 1000000; i++) {
```

```
9           pthread_mutex_lock(&lock);
10          counter++;
11          pthread_mutex_unlock(&lock);
12        }
13        return NULL;
14      }
15
16      int main(void) {
17        pthread_t t1, t2;
18        pthread_create(&t1, NULL, increment, NULL);
19        pthread_create(&t2, NULL, increment, NULL);
20        pthread_join(t1, NULL);
21        pthread_join(t2, NULL);
22        printf("Valore finale del counter: \%d\n", counter);
23      }
```

Come vediamo, la soluzione risulta molto semplice ed altrettanto efficace:

1. Come primissima cosa, dobbiamo definire il nostro lock: usiamo il costrutto `PTHREAD_MUTEX_INITIALIZER`, una costante della libreria.

2. Individuiamo la sezione critica (nel nostro caso solo `counter++`) e la rinchiudiamo nei due costrutti astratti definiti prima: il thread che arriva prende la chiave e chiude il lucchetto (`pthread_mutex_lock(&lock)`), esegue **tutto** il codice della sezione critica e poi sblocca il lucchetto per permettere ad altri thread di eseguire la sezione critica (`pthread_mutex_unlock(&lock)`).

Osservazione 7.2.2: Ordine di esecuzione dei thread

Supponiamo che un thread_i prenda per primo il lock e blocchi la sezione critica per gli altri. Una volta liberato questo lock, la "chiave" del lucchetto verrà attribuita al primo thread che ha provato ad accedere alla risorsa dopo il thread_i. Insomma, la gestione dell'ordine per l'accesso dei thread alla risorsa viene fatta mediante una logica FIFO.

Negli esempi precedenti abbiamo visto come l'utilizzo di un lock risulti la soluzione corretta al problema della race condition. Non abbiamo però analizzato *come* costruire un lock: cosa c'è dietro alle istruzioni di `lock()` e `unlock()` che abbiamo utilizzato? Come può un Sistema Operativo implementare un lock in modo corretto ed efficiente? Vediamo di

seguito l'evoluzione delle linee di pensiero che hanno poi portato alla gestione corrente nei sistemi moderni.

Disabilitazione degli interrupt La prima e più banale idea per creare un lock è disattivare i segnali di interrupt del processore quando un thread entra nella sezione critica e riattivarli quando questo vi esce.
Per quanto semplice, questa soluzione nasconde delle insidie potentissime:

1. Viene dato *troppo* potere al codice utente: la disabilitazione dei segnali è chiaramente un'operazione riservata alla *Kernel mode*. Disabilitando i segnali di interruzione stiamo lasciando che il singolo thread **monopolizzi la CPU** finché non verrà terminata la sezione critica. I problemi? Il thread potrebbe bloccarsi e dunque bloccare totalmente la CPU, più non è mai una buona soluzione far monopolizzare la CPU da un processo, qualunque che esso sia.

2. In un'architettura multicore, a bloccarsi sarebbe il singolo core all'interno del quale il primo thread stava lavorando, ma ciò non impedisce affatto a thread in esecuzione su altri core di entrare nella sezione critica.

Come vediamo, dunque, questa non è certo la soluzione che cerchiamo.

Tentativi puramente software

Si potrebbe pensare di utilizzare una variabile globale che agisce da flag condiviso tra i vari thread: il thread che accede la sezione critica imposta il flag a 1 e tutti gli altri verranno bloccati da un `while (flag == 1)` , un ciclo while apparentemente infinito utile solo a bloccare i vari thread dal proseguimento del codice.
Vediamo però una somiglianza tra questa soluzione e il problema della variabile `counter` precedentemente analizzato: chi ci garantisce che nell'istante preciso in un thread imposta il flag a 1 nessun altro thread lo stia già facendo? E che tale operazione non possa essere *logicamente* interrotta dal time-slice dello scheduler?
Come si è già visto più volte in questo corso, la soluzione spesso risiede nell'utilizzo di componenti hardware dedicate. In particolare, questa volta vogliamo trovare un modo per ottenere un'**operazione di lettura e scrittura che venga interpretata dalla CPU come atomica**, dunque eseguibile in un blocco solo. Queste istruzioni vengono dette **Read-Modify-Write (RMW)**, sono fornite dall'hardware e possono essere programmate in diversi modi che vediamo di seguito.

Test-and-Set (TAS)

L'implementazione della logica **Test-and-Set** avviene nel seguente modo:

```
int TestAndSet(int* indirizzo, int nuovo_valore) {
  int vecchio = *ptr;
  *ptr = nuovo_valore;
  return vecchio;
}
```

Questo algoritmo si occupa di scrivere *incondizionatamente* un nuovo valore all'interno dell'indirizzo di memoria fornito (`int* ptr` in questo caso) e di restituire il vecchio valore presente. E come può una strategia così semplice funzionare?
Supponiamo di impostare `flag = 0` per un lock libero e `flag = 1` per il lock occupato. Quando un thread inizialmente arriva e prende il lock, il nuovo valore del flag sarà chiaramente 1. Tuttavia la funzione restituirà 0, che era il valore precedente. Vedendo come valore di ritorno 0, il thread capisce che ha acquisito il lock e può eseguire la funzione critica. Qualsiasi altro thread che proverà a prendere il lock dovrà dunque eseguire la funzione `TestAndSet()` in loop, che continuerà ad impostare continuamente il valore ad 1 ma, finché il thread vedrà come valore restituito dalla funzione 1, capirà che anche precedentemente al suo arrivo il lock era già stato ottenuto e dunque non può procedere con la sezione critica.

Esempio 7.2.1: Esempio di uso reale di TAS

Una semplice implementazione di un lock che garantisce la mutua esclusione tra più thread dello stesso processo è la seguente:

```
int counter = 0;
int mutex = 0;

void lock_tas() {
  while (TestAndSet(&mutex, 1) == 1) {}
}

void unlock_tas() {
  mutex = 0;
}
```

Definiamo innanzitutto il nostro lock, che assumerà valore 0 quando libero e 1 quando occupato. Il primo thread che entrerà nella funzione `foo()` sarà anche

il primo ad acquisire il lock. L'acquisizione del lock si basa sul funzionamento dell'algoritmo TAS:

1. Il primo thread tenta di scrivere 1 all'interno del mutex (vedi il secondo parametro della funzione `TestAndSet`).
2. Tale funzione restituirà sempre il valore `old`, cioè quello precedente alla chiamata di funzione. Se tale valore sarà 0 (e questo è **certo** per il primo thread che accede alla risorsa) allora il valore viene correttamente impostato ad 1 e viene restituito 0.
3. Il thread chiamante, leggendo come valore di ritorno 0, non si bloccherà dentro al ciclo while, benì procederà con le istruzioni della sezione critica.
4. Se nel frattempo il *time-slice* di tale thread dovesse terminare e dovesse essere mandato in esecuzione un altro, questo dovrà a sua volta tentare di acquisire il lock per accedere alla sezione critica.
   Nel suo caso, però, chiamando la funzione `TestAndSet` si otterrà come valore di ritorno 1, poiché il lock era già stato acquisito, ma non liberato, dal thread precedente.
5. Al suo ritorno, quello che era il primo thread libererà il lock semplicemente impostandolo a 0. Nota che non è necessario eseguire un `TestAndSet(&mutex, 0)`, in quanto il thread corrente avrà già acquisito il lock e *dunque* sarà sicuramente l'unico in esecuzione per quell'istruzione.

Questa soluzione garantisce la **correttezza** del lock, bensì **non l'equità**, in quanto non viene memorizzata alcuna coda dei thread in ordine di come hanno richiesto il lock, e dunque la scelta del prossimo thread da eseguire è totalmente casuale (problema della **starvation**). Si tratta inoltre di un algoritmo oneroso anche dal punto di vista computazionale, in quanto i thread inattivi eseguono costantemente la funzione `TestAndSet()` fino all'arrivo del loro turno: ciò implica una lettura e una scrittura per ogni tick di esecuzione del thread, portando così ad un innecessario traffico dei bus di sistema per aggiornare la cache.

Compare-and-Swap (CAS)

Un'alternativa all'algoritmo TAS, che scrive costantemente il nuovo valore all'interno della variabile, è presentata dall'algoritmo **Compare-and-Swap (CAS)**, che aggiorna il valore del lock solo se questo corrisponde al valore atteso passato dal thread in esecuzione.

```
int CompareAndSwap(int* ptr, int atteso, int nuovo) {
  if (*ptr == atteso) {
    *ptr = nuovo;
    return 1;
  }

  return 0;
}
```

Molto banalmente, il funzionamento di CAS è il seguente:

1. Il thread chiamante vuole modificare una variabile, supponiamo incrementandola di una sola unità. Dato che l'incremento di una variabile condivisa da parte di più thread abbiamo visto essere un'operazione delicata, il thread chiama la funzione CAS con la seguente firma: `CompareAndSwap(&variabile, vecchio, vecchio + 1)`. Questo perché il singolo thread, non avendo modificato il valore della variabile nel mentre, si aspetta di trovare `atteso = vecchio`.

2. Nella realtà dei fatti, come abbiamo visto, non sempre questo accade: se si verificasse che un thread ha modificato la variabile condivisa, avremmo un disallineamento tra i due valori. A quel punto il controllo della funzione CAS fallirebbe e dunque il thread dovrà riprovare e aggiornare il valore della variabile da lui letto, ri-eseguendo il CAS: solo una volta che il controllo andrà a buon fine il thread potrà procedere.

Questo algoritmo è apparentemente ottimo, e permette di definire algoritmi potenti privi di lock (*lock-free*) e non soggetti a soli due stati (sarà utile per la costruzione dei futex, citati in seguito), tuttavia soffre dello stesso problema di consumo delle risorse del TAS (chiamate continue alla funzione di aggiornamento) e del cosiddetto **problema ABA**: se un valore cambia da A a B e poi ritorna ad A prima che il thread completi il CAS, questo non si accorge minimamente della manipolazione della variabile, portando potenzialmente a gravi bug come la corruzione della memoria. L'implementazione del CAS come mutex è analoga a quella vista nell'esempio precedente riguardante il CAS.

Fetch-and-Add (FAA) e Ticket-Lock

L'algoritmo **Fetch-and-Add (FAA)** si occupa di incrementare la variabile condivisa di un dato valore `d` (tipicamente 1) e restituisce il vecchio valore. Il principio è simile ai due algoritmi precedenti, ma questo viene spesso accoppiato all'algoritmo **Ticket-Lock**, che

prevede l'assegnazione di un *biglietto d'ingresso* ad ogni thread che prova ad accedere alla variabile. In particolare, il funzionamento di questo algoritmo è il seguente:

1. Vengono inizializzati due contatori condivisi tra i thread: `ticket`, che indica il **numero del prossimo biglietto da consegnare**; `turn`, che indica il numero del biglietto che può accedere alla risorsa in quel momento.

2. Quando il Thread A prova ad accedere alla risorsa condivisa, chiama la funzione `FetchAndAdd(&ticket, 1)`: riceve il biglietto con valore 0 e la variabile `ticket` avanza ad 1. Immediatamente dopo, il Thread B fa lo stesso: avendo incrementato i due contatori, stavolta B riceve il biglietto con valore 1 e il valore globale di `ticket` avanza a 2.

3. A questo punto entra in gioco il contatore `turn`: all'entrata della sezione critica viene posizionato un classico loop che mantiene i thread bloccati. La condizione di questo loop è `turn != thread_ticket`, dunque **blocca tutti i thread di cui non è il turno**. Il thread che riesce a superare questo ciclo può eseguire la sezione critica e, alla fine di questa, dovrà essere incrementato il valore di `turn` in modo atomico con `FetchAndAdd(&turn, 1)` per permettere al prossimo thread in coda di essere eseguito.

Con l'algoritmo Ticket-Lock viene implementata anche una specie di coda che tiene traccia dell'ordine dei thread che hanno richiesto la risorsa, in modo da prevenire completamente il problema della starvation (prima o poi tutti i thread accederanno alla risorsa, in un tempo abbastanza ragionevole).

Osservazione 7.2.3: Le operazioni atomiche

Avendo letto questi algoritmi si potrebbe pensare: ok, ma le operazioni di incremento delle variabili non sono soggette agli stessi problemi di esaurimento del time-slice che abbiamo discusso nel primo esempio di `counter`?
Per quanto le due situazioni sembrano simili, vengono separate da un'importante differenza: gli algoritmi descritti sopra prevedono l'implementazione di tali operazioni **a livello hardware**, dunque vengono codificate proprio dal produttore dell'elaboratore e rappresentano dunque per il sistema un'operazione unica, un unico comando.

Ad esempio, per le architetture x86-64, i comandi che descrivono le varie operazioni sono i seguenti:

- TAS: comando `xchg`.

- CAS: comando `cmpxchg`.
- FAA: comando `lock xadd`.

Thread in attesa

Quando un Thread A acquisisce con successo un lock, vengono messi in attesa tutti i thread che provano ad accedere alla risorsa. Questi hanno dunque due opzioni:

- **Spinning**: continuare a girare a vuoto nel loop che blocca l'accesso alla risorsa in attesa che questa venga liberata. Questo comportamento viene detto di `BUSY WAITING`. E' particolarmente efficace quando la sezione critica è molto corta, e dunque l'overhead del context-switch sarebbe troppo costoso rispetto alla semplice attesa.
- **Sleeping**: i thread che attendono il rilascio del lock vengono "messi a dormire", duqnue fatti passare dallo stato `RUNNING` a `BLOCKED`, venendo liberati solo al rilascio della risorsa. Si tratta di un'alternativa particolarmente efficace quando la sezione critica è molto lunga, in quanto altrimenti si sarebbe sprecato lavoro di CPU inutile per il singolo thread, magari fondamentale per altri processi.

La prima opzione è molto semplice, e non richiede particolari approfondimenti. Per quanto riguarda invece la seconda, è buono spenderci un paio di parole.
Il tutto viene realizzato secondo una logica ben precisa:

1. Il Thread A acquisisce per primo il lock, dovendo dunque effettuare una chiamata di sistema per gestire in modo corretto l'operazione di acquisizione del lock in modo che questa risulti atomica.
2. Quando un thread trova il lock già acquisito, è pronto per *mettersi a dormire*: innanzitutto viene aggiunto alla coda dei thread in attesa (`wait_queue`), in modo da prevenire la starvation e conservare un ordine tra i thread richiedenti; poi viene effettuata una chiamata di sistema mediante la funzione `park()`, che comunicherà allo scheduler di mettere in stato `BLOCKED` il thread corrente.
   NOTA: La funzione `park()` è diversa dalla funzione `sleep()` che abbiamo già visto, in quanto il processo viene messo in `BLOCKED` per un tempo indeterminato.
3. Una volta che il Thread A avrà completato la sezione critica, dovrà rilasciare anche il lock. L'operazione di rilascio del lock avviene assieme ad altre due operazioni:
   (a) Viene prelevato dalla `wait_queue` il prossimo thread da eseguire.

(b) Viene svegliato tale thread mediante la funzione `unpark()`, che anch'essa nasconde una chiamata di sistema, e che si occupa di far passare il thread dallo stato `BLOCKED` a `READY`, in modo che possa essere schedulato.

Un modo estremamente efficiente di gestire i lock viene fatto (ovviamente) dal sistema operativo Linux mediante i **Fast Userspace Mutex (futex)**. Questa strategia permette di ridurre al minimo le chiamate di sistema, onerosissime e caratteristiche dello sleeping dei thread descritto poco fa, e di effettuarle solo quando veramente necessario, ovvero quando ci sono dei thread in coda ad attendere l'accesso alla risorsa. Questo poiché nelle situazioni reali un thread trova il lock aperto il 90-95% delle volte, proprio per la necessità della sezione critica di essere il più breve possibile. Non avrebbe senso dunque effettuare chiamate di sistema costanti anche quando non necessarie.
Il funzionamento del futex è il seguente:

1. Il Thread A arriva per primo alla risorsa, trova il lock a 0 e lo acquisisce, impostandolo poi ad 1.
2. Se non arriva nessun thread nel frattempo, non viene effettuata nessuna chiamata di sistema.
3. Se invece arriva un nuovo thread, imposterà il valore del lock a 2, indicando dunque che c'è qualcuno in coda. A quel punto il thread andrà a dormire e verrà messo in una speciale coda di sistema. La corrispondente operazione è `futex(&lock_state, FUTEX_WAIT, 2)`.
4. Quando il Thread A avrà finito, controllerà il valore del lock: se questo sarà impostato a 2, significa che è arrivato un altro thread a richiedere la risorsa. Dunque il Thread A dovrà notificare il SO di aver terminato l'esecuzione della sezione critica, re-impostando il lock e facendo svegliare il primo thread arrivato. Questo viene fatto con la chiamata `futex(&lock_state, FUTEX_WAKE, 1)`.

Lock a due fasi

Come sempre, la soluzione più geniale consiste nell'implementare entrambi i comportamenti:

- **Spinning per un breve periodo di tempo**, in modo che se la sezione critica risulti essere abbastanza breve, il thread potrà venir mandato subito in esecuzione senza l'overhead caratteristico del context-switch.

- Se durante lo spinning il thread non è riuscito ad acquisire il lock, significa che la **sezione critica è abbastanza lunga**, e dunque **il thread viene mandato in `BLOCKED`**, per evitare di consumare inutilmente cicli di CPU.

7.3 Condition Variables

Fino ad ora abbiamo visto come l'unico modo che ha un thread (ad esempio il *main*) per attendere un altro thread sia che quest'ultimo termini la sua esecuzione. Questo comportamento è ottenibile mediante la funzione `pthread_join(&tid, ...)`. Tuttavia, in molti casi, vorremmo che la *condizione di sblocco* del thread che attende non sia *la fine dell'altro thread*, bensì una condizione arbitraria, come ad esempio il cambio di valore di una variabile. Una semplice implementazione può essere quella di bloccare il main thread con un ciclo while finché tale variabile di condizionamento (o espressione di condizionamento) non assume il valore atteso. Tuttavia, come si può vedere nell'esempio di seguito, si ritorna sempre al problema di spinning e di spreco del time-slice della CPU.

```
1   volatile int done = 0;
2
3   void *child(void *arg) {
4     printf("child: in esecuzione\n");
5     done = 1;
6     return NULL;
7   }
8
9   int main(void) {
10    pthread_t tid;
11    pthread_create(&tid, NULL, child, NULL);
12    while (done == 0) { }
13    printf("parent: child terminato\n");
14    return 0;
15  }
```

La soluzione proposta dalle condition variable è molto semplice ed intuitiva: quando ad un thread la condizione di controllo risulta falsa, questo va a dormire; si sveglierà soltanto mediante un apposito segnale, rilasciato dal processo occupante nel momento in cui terminerà la sezione critica.

Definizione 7.3.1: Condition Variable

Una **condition variable** non è altro che una struttura composta da vari elementi, che lavorando insieme garantiscono la mutua esclusione tra thread evitando però il busy-waiting:

- Una **condizione di controllo**, che di fatto si traduce in una variabile condivisa tra i vari thread (dunque *ovviamente* protetta da mutex) che rappresenta la condizione secondo la quale i thread possono accedere o meno alla sezione critica.
- Una **coda dei thread dormienti**, usata per tenere traccia dei thread che staranno effettivamente attendendo che si liberi il lock (ovvero la condizione di controllo diventi vera) in modo da accedere alla risorsa condivisa.

Le due operazioni fondamentali che caratterizzano le condition variable sono:

- `wait(cv, m)`, che si occupa di rilasciare il lock `m` e di aggiungere alla coda dei processi dormienti il thread che ha chiamato la funzione. Il processo di rilascio della lock + inserimento in coda + passaggio a `BLOCKED` viene visto dal sistema come una singola operazione atomica, in modo da impedire che alcuni thread vadano a dormire per sempre.
- `signal(cv)`, che si occupa di svegliare *casualmente* uno dei thread dormienti nella coda. Nota importante: in alcune implementazioni (discusse brevemente in seguito) la `signal()` si limita solo a svegliare un thread dormiente, senza però attribuirgli la lock `m`.

Il pattern universale dei thread che utilizzano una condition variable è il seguente:

```
1    pthread_mutex_lock(&m);
2    while (!condizione) {
3      pthread_cond_wait(cv, &m);
4    }
5    // fai operazioni
6    pthread_mutex_unlock(&m);
```

Listing 7.1: Pattern per il thread attendente

```
1    pthread_mutex_lock(&m);
```

```
2     // fai operazioni che renderanno vera la condizione
3     pthread_cond_signal(&cv);
4     pthread_mutex_unlock(&m);
```

Listing 7.2: Pattern per il thread eseguente

Osservazione 7.3.1: Ruolo della lock e della condizione

Una causa frequente di confusione riguardo alle condition variable è il seguente: a che serve definire *sia un lock che una condizione?* Giustamente, se definiamo un lock è impossibile che due thread eseguano contemporaneamente la sezione critica, in quanto la lock garantisce la *mutua esclusione*. Tuttavia, come abbiamo premesso introducendo l'argomento, la condizione serve proprio a limitare *l'accesso alla risorsa in un momento errato*, oppure "non propizio". Magari il secondo thread riuscirà ad acquisire la lock, però poi rimarrà bloccato nel ciclo while perché la condizione non sarà ancora diventata vera, e dunque dovrà atomicamente rilasciare la lock e mettersi a dormire (situazione non calzante per l'esempio precedente, ma verificabile in contesti reali).

7.3.1 Il problema dei Produttori-Consumatori

Presentiamo adesso uno dei problemi classici della programmazione concorrente: il problema dei Produttori e Consumatori. La traccia è semplice: si hanno `n` thread che hanno accesso ad un buffer di elementi; alcuni di questi thread, i **produttori**, si occupano di **inserire elementi all'interno del buffer**. La condizione che li limita dal farlo è ovviamente che il buffer sia pieno; il resto dei thread rappresenta i **consumatori**, che si occupano dell'operazione duale. Rimuovere una risorsa dal buffer, con l'ovvia limitazione di non poterlo fare quando questo è vuoto. Ovviamente, l'accesso al buffer è garantito solo in mutua esclusione, ma ormai siamo bravi a gestire queste casistiche e sappiamo di dover utilizzare un `mutex`.
Nel nostro esempio useremo come `buffer` una singola variabile (anche se il problema si può estendere anche a strutture dati più complesse) e una variabile di stato `count` che varrà `0` se il buffer è vuoto e `1` se il buffer è pieno.
Leggendo questa traccia, una prima soluzione che ci viene in mente è quella di usare un mutex per proteggere l'accesso al buffer e una Condition Variable che controlli:

- Per il produttore se il buffer è pieno (`count = 1`) e gli impedisca di procedere in tal caso.

- Per il consumatore se il buffer è vuoto (`count = 0`) e gli impedisca di procedere se tale condizione risulta vera.

Lo pseudocodice può essere dunque tradotto semplicemente nel seguente modo:

```
1   void produttore() {
2     pthread_mutex_lock(&m);
3     while (count == 1) {
4       pthread_cond_wait(&cv, &m);
5     }
6     // mette un elemento nel buffer
7     pthread_cond_signal(&cv);
8     pthread_mutex_unlock(&m);
9   }
10
11  void consumatore() {
12    pthread_mutex_lock(&m);
13    while (count == 0) {
14      pthread_cond_wait(&cv, &m);
15    }
16    // rimuove un elemento dal buffer
17    pthread_cond_signal(&cv);
18    pthread_mutex_unlock(&m);
19  }
```

Ad una prima occhiata sembra funzionare: avendo messo un mutex prima della sezione critica è vero che due thread non possono accedere contemporaneamente ad essa. E grazie alla condizione e alla `wait()` è anche vero che i thread dello stesso tipo vengono bloccati finché la condizione non risulta vero.
Ma mettiamoci nell'ottica di avere un produttore P e due consumatori, C1 e C2. Supponiamo che P esegua tranquillamente le sue operazioni, e nel mentre è arrivato C1. Esso viene dunque messo a dormire. Prima che P finisca, arriva anche C2. Al termine di P viene svegliato un dormiente casuale, sia questo C1: esegue la sezione critica, libera il buffer e rilascia il lock. Tutto corretto. Adesso nella "coda dei dormienti" abbiamo P, C2. Supponiamo che C1 svegli C2 invece di P: C2 vede il buffer vuoto e torna a dormire, senza svegliare nessuno (non ha completato l'operazione!). Tutti i thread dormono: **deadlock**.

Come abbiamo visto, il problema è dato dall'implementazione dell'operazione `signal()`, che sveglia un processo casuale all'interno della coda, ma **non garantisce che questo**

acquisisca la lock, né tantomento che questo venga eseguito! Per prevenire situazioni di stallo come quella descritta precedentemente, dobbiamo fare in modo che i produttori sveglino i consumatori e viceversa: ciò si ottiene implementando due CV, ognuna per coda. Di seguito l'implementazione *corretta* per risolvere problemi del tipo *Produttore-Consumatore*.

```
1  pthread_cond_t cv_produttori = ...
2  pthread_cond_t cv_consumatori = ...
3
4  void produttore() {
5    pthread_mutex_lock(&m);
6    while (count == 1) {
7      pthread_cond_wait(&cv_produttori, &m);
8    }
9    // mette un elemento nel buffer
10   pthread_cond_signal(&cv_consumatori);
11   pthread_mutex_unlock(&m);
12 }
13
14 void consumatore() {
15   pthread_mutex_lock(&m);
16   while (count == 0) {
17     pthread_cond_wait(&cv_consumatori, &m);
18   }
19   // rimuove un elemento dal buffer
20   pthread_cond_signal(&cv_produttori);
21   pthread_mutex_unlock(&m);
22 }
```

Con questa nuova implementazione, si ottiene il comportamento descritto: quando un produttore trova il buffer pieno, va a dormire*nella sua coda*; dopo aver eseguito le operazioni, non sveglia a caso rischiando di svegliare un altro produttore, bensì sveglia un consumatore dall'apposita coda. Il discorso è analogo per il ruolo duale.

7.3.2 Semantiche Mesa e Hoare

Brevemente, discutiamo un concetto accennato precedentemente riguardante diverse filosofie di implementazione della funzione `signal()`:

- **Semantica Hoare**: prevede che il segnale svegli un processo dalla coda, gli attri-

buisca la lock e lo faccia schedulare immediatamente. Utile per la maggior parte dei casi, ma include un ampio numero di chiamate di sistema (manipolazione dello scheduler, ad esempio) e include nel suo costo l'overhead di un context switch, dunque se usato troppo frequentemente risulta oneroso dal punto di vista del costo.

- **Semantica Mesa**: prevede che il segnale si occupi solo di svegliare casualmente un processo in coda. Si tratta della semantica adottata finora negli esempi, dunque se mal gestita può portare ai problemi di deadlock visti poc'anzi.

7.3.3 Broadcast

Ci sono casistiche in cui svegliare un singolo thread non è sufficiente. Supponiamo di voler implementare un allocatore di memoria. Abbiamo un processo A che richiede di allocare 100 byte e un processo B che ne richiede 50.
Supponiamo ora di avere 60 byte di memoria liberi, e di avere i processi A, B dormienti nella coda della Condition Variable: per la natura casuale della `signal()`, viene svegliato prima il thread A e viene effettuato il controllo di validità della condizione: 100 <= 60? No, dunque il processo A torna (correttamente) a dormire. Tuttavia non è stato svegliato nessun altro, dunque il processo B, anche se per lui la condizione sarà vera, non avrà mai la possibilità di procedere nella sezione critica in quanto non potrà essere svegliato da nessuno (salvo processi nuovi entranti, ma stiamo considerando un caso estremo).
Dato che i thread dormienti non sono fisicamente allocati, non abbiamo neanche modo di controllare se per loro la condizione sarebbe vera o meno, in modo da svegliarli. Quando non possiamo determinare quali processi svegliare, li svegliamo tutti: è proprio questo il ruolo della funzione `broadcast()`. Una volta svegliati, i thread si alterneranno per provare ad accedere alla sezione critica: solo uno potrà accedervi e dunque tutti gli altri torneranno a dormire, ma è un piccolo prezzo da pagare per non entrare in una situazione di stallo.

7.4 Semafori

Nelle precedenti sezioni abbiamo coperto il concetto di **lock**, come strumento per garantire l'accesso in mutua esclusione su una risorsa da parte di più thread, e di **Condition Variable** per limitare l'accesso alla risorsa solo ai thread per cui è rispettata una certa condizione. Per semplificare il tutto, il genio informatico *Edsger Djikstra* ha pensato di condensare entrambe queste strutture in una sola: i **semafori**.
Si può pensare ad un semaforo banalmente come un "distributore di biglietti", con cui ovviamente si può interagire in due modi:

- `wait()`: si prende un biglietto. Se il distributore è vuoto, si attende finché qualcun altro non avrà rimesso il suo biglietto a posto.
- `post()`: si mette un biglietto nel distributore. Nota che, seppur logicamente non sia così, quest'operazione ha senso anche se un semaforo viene inizializzato come "senza biglietti". Vedremo a breve perché.

Fatte le dovute premesse diciamo dunque che, essenzialmente, un semaforo non è altro che una struttura dati che mette a disposizione un intero `S` (che rappresenta il numero di biglietti), una coda dei thread in attesa (come nelle CV) e le due operazioni descritte poco fa, che sono banalmente implementabili con il seguente pseudocodice:

```
1  semaforo_wait(S):
2    while (S == 0):
3      metti a dormire il thread nella coda del semaforo
4    S = S - 1
5
6  semafoto_post(S):
7    S = S + 1
8    if coda non vuota:
9      sveglia un thread
```

Come vediamo, sono tutte operazioni molto intuitive se si sono capiti i concetti precedenti, in quanto non è altro che la consolidazione finale di tali strutture in una sola. Possiamo infatti vedere un'analogia: la `wait()` del semaforo è come se fosse la `lock()` del mutex, che blocca i thread che trovano S == 0; la `post()`, al contrario, può essere vista come l'analogo della `unlock()`, che incrementa S e dunque dà possibilità ad altri thread di accedere alla risorsa.
I semafori hanno però una caratteristica fondamentale che li distingue dalle due strutture precedenti: sono più generici e scalabili. Questo assume maggiore significato paragonandoli alle lock: in queste ultime, il valore del mutex poteva essere 0 oppure 1, per garantire la mutua esclusione. In un semaforo, il valore di S può tranquillamente essere > 0, in quanto dipende dalla situazione che si vuole gestire. Vediamo infatti di seguito le 3 casistiche per cui vengono utilizzati i semafori.

7.4.1 I semafori come mutex

Alla creazione del semaforo, inizializzando il valore di `S = 1` avremo che questo si comporterà esattamente come una lock: quando un thread chiamerà `semaforo_wait()` la funzione controllerà la condizione del while, ossia `S == 0`. Siccome questa risulterà falsa,

il thread passerà immediatamente all'istruzione successiva e decrementerà S, portandolo a 0. Se un secondo thread proverà ora ad accedere alla sezione critica, dovrà comunque fare `semaforo_wait()`. Nel suo caso, però, il valore di S risulterà 0 in quanto decrementato dal primo thread, e dunque non potrà procedere, dunque andrà a dormire. Terminata la sezione critica, il primo thread chiamerà `semaforo_post()`, riportando S al valore 1 e svegliando uno dei thread in attesa nella coda. E così via, conosciamo già il cinema...

7.4.2 I semafori come ordinamento

Supponiamo invece di voler far comportare un semaforo come una `pthread_join()`, cioè facendo in modo che un thread "padre" attenda la fine (oppure il verificarsi di una condizione, similmente alle CV) all'interno di un thread figlio.
Possiamo ottenere questo comportamento in modo molto semplice, inizializzando il semaforo a 0 e chiamando la funzione `semaforo_wait()` all'interno del thread padre. Nel thread figlio, invece, eseguiremo tutte le operazioni critiche e solo alla fine chiameremo `semaforo_post()`, in modo da "sbloccare" eventuali thread in attesa. Una semplice implementazione è la seguente:

```
1   void operazioni_figlio() {
2     // operazioni critiche
3     semaforo_post(&sem);
4   }
5
6   int main() {
7     sem_init(&sem, 0);
8     pthread_t tid;
9     pthread_create(&tid, operazioni_figlio);
10    semaforo_wait(&sem);
11    return 0;
12  }
```

7.4.3 Semafori come gestori di un pool di risorse

Immaginiamo infine di avere una situazione diversa dalle precedenti: abbiamo un server che può gestire un massimo di `N` richieste alla volta. Le richieste sono indipendenti l'una dall'altra, ma sulla macchina possono lavorare più thread contemporaneamente: l'importante è garantire che non venga sforato il numero `N` di thread in esecuzione contemporaneamente (ha senso parlare di contemporaneità nei sistemi multi-core). Tale operazione si può, di nuovo, gestire grazie ad un semaforo, inizializzando il valore di `S = N`. In questo

modo è come se il semaforo stesse dicendo: "Posso far passare un massimo di `N` thread alla volta, ma l'`N+1`-esimo troverà S=0 e dunque verrà bloccato finché uno dei thread non avrà rilasciato la risorsa".

Osservazione 7.4.1: Produttori-Consumatori coi semafori

Per risolvere il problema dei Produttori-Consumatori trattato in precedenza avevamo usato un mutex (per controllare l'accesso al buffer) e due CV il blocco e la memorizzazione separata dei produttori/consumatori.
Tale soluzione può essere ottenuta anche mediante i semafori, inizializzandone uno come `mutex, S=1`, uno come `waiter, S=0` (per bloccare i consumatori dal provare a consumare subito quando ancora non c'è nulla) e uno con `S = lunghezza_buffer` (per permettere ai produttori di aggiungere elementi al buffer).

7.4.4 Il problema dei filosofi a cena

Avendo ora definito la struttura *Semaforo*, discuteremo i problemi della programmazione concorrente utilizzando questo nuovo costrutto. Il problema che si vuole esaminare adesso è il famoso dilemma (che nasce in realtà come forma di presa in giro) dei *filosofi a cena*. Ci sono `N` filosofi a tavola e altrettante forchette. Un filosofo può compiere solo due azioni, alternate l'una dall'altra: prima pensa `think()` e poi mangia `eat()`. Per mangiare, però, ha bisogno sia della forchetta alla sua destra che di quella alla sua sinistra (insomma la forchetta destinata a lui e quella destinata ad un altro filosofo). Come coordinare una routine corretta ed efficiente per fare in modo che tutti i filosofi eseguano entrambe le azioni?
Una prima idea e intuitiva idea è quella di mettere un lock su ogni forchetta, in modo che due filosofi non possano avere contemporaneamente la stessa forchetta. Facciamo dunque prendere ad ogni filosofo la forchetta di sinistra, comportamento che viene rappresentato dal seguente codice:

```
1  sem_t forchette[N];
2
3  void filosofo(int id_filosofo) {
4    while (true) {
5      think();
6      // prova a prendere la forchetta a destra
7      semaforo_wait(&forchette[id_filosofo])
8
9      // prova a prendere la forchetta a sinistra
```

```
10          semaforo_wait(&forchette[(id_filosofo+1)\%N]);
11          eat();
12
13          // mette a posto entrambe le forchette
14          semaforo_push(&forchette[id_filosofo)
15          semaforo_push(&forchette[(id_filosofo+1)\%N]);
16        }
17      }
```

Sembra logico no? Ma per il ragionamento che stiamo seguendo, tutti i filosofi prendono, uno dopo l'altro, la forchetta di sinistra. Forchetta che *per un filosofo* sarà quella di sinistra, mentre per quello adiacente sarà quella di destra. Si otterrà dunque una fantastica situazione in cui tutti i filosofi terranno alzata la forchetta di sinistra e proveranno a prendere quella di destra, che sarà però occupata da un altro filosofo: **stallo, deadlock**.

Per problemi di questa natura la soluzione è pensare fuori dagli schemi: la posizione circolare dei filosofi al tavolo ci fa pensare di dover mantenere una simmetria, ma è proprio per questo che dobbiamo romperla: dobbiamo fare in modo che **almeno un filosofo rompa la simmetria prendendo in mano la forchetta di posizione opposta agli altri**. Tale comportamento si ottiene modificando leggermente il codice precedente:

```
1     sem_t forchette[N];
2
3     void filosofo(int id_filosofo) {
4       while (true) {
5         think();
6
7         // Per un singolo filosofo rompiamo la simmetria,
8         // provando a prendereprima la forchetta di sinistra
9         // e poi quella di destra
10        if (id_filosofo == N - 1) {
11          semaforo_wait(&forchette[(id_filosofo+1)\%N]);
12          semaforo_wait(&forchette[id_filosofo];
13        } else {
14          semaforo_wait(&forchette[id_filosofo];
15          semaforo_wait(&forchette[(id_filosofo+1)\%N]);
16        }
17
18        eat();
```

```
19
20          // Mette a posto entrambe le forchette
21          semaforo_push(&forchette[id_filosofo)
22          semaforo_push(&forchette[(id_filosofo+1)\%N]);
23        }
24      }
```

7.5 Il deadlock

In questa sezione andiamo ad argomentare brevemente uno dei maggiori problemi della programmazione concorrente, già incontrato in numerosi esempi precedenti: lo **stallo** o **deadlock**. Si tratta di una situazione critica, che non causa errori o bug nell'esecuzione del programma, bensì si verifica quando **due o più thread si bloccano a vicenda attendendo l'accesso ad una risorsa condivisa**.

Osservazione 7.5.1: Rilevamento degli stalli mediante grafi

Il problema di rilevamento degli stalli all'interno di un programma può essere modellato come un grafo orientato, dove i vertici sono rappresentati sia dai singoli thread T_i che dalle risorse R_j. Gli archi di questo grafo possono essere poi di due tipi:

- Arco del tipo $T_i \to R_i$: indica che il thread T_i ha richiesto la risorsa R_i.
- Arco del tipo $R_i \to T_i$: indica che la risorsa R_i è stata concessa al thread T_i.

Dunque, la scrittura $T_i \to R \to T_j$ significa che il thread i ha richiesto la risorsa R che è stata però concessa al thread j. Dunque il thread i è in attesa che j rilasci la risorsa.
Chiaramente questo è solo un modo teorico di individuare un problema, e nessun programma reale implementa mai questo meccanismo per rilevare deadlock, ma tali situazioni corrispondono alla rilevazione di *un ciclo all'interno del grafo.*

In maniera più pragmatica, sono state definite 4 condizioni che devono verificarsi simultaneamente affinché si possa avere la certezza di avere un deadlock. Tali condizioni vengono dette **condizioni di Coffman**:

1. **Mutua esclusione**: vi è mutua esclusione tra i thread, cioè che solo un thread alla volta può possedere tale risorsa.
2. **Hold and Wait**: un thread possiede già almeno una risorsa e nel frattempo si

mette in attesa per prenderne un'altra.

3. **Nessun rilascio forzato**: il sistema operativo non può togliere la risorsa al thread, deve essere lui a rilasciarla.

4. **Attesa circolare**: quando si crea una catena chiusa di attese (A aspetta B, B aspetta C, C aspetta A).

Conoscendo queste 4 condizioni di verifica di un deadlock, si può dunque strutturare il proprio codice in modo da impedire che **almeno una di queste non si verifichi**. Vediamo di seguito le varie strategie.

7.5.1 Prevenzione del deadlock

Si possono prevenire contemporaneamente sia la clausola *Hold-and-Wait* sia la clausola che *impedisce il rilascio forzato* mediante la funzione `trylock(&lock)` che, come suggerisce il nome, **tenta di acquisire il lock**: restituisce 0 se il lock è acquisibile (in quanto il valore del mutex sarebbe 0) mentre `EBUSY` se già un altro thread l'ha acquisito. La soluzione è usare tale funzione per provare ad acquisire entrambi i lock e, se anche uno solo sarà occupato, rilasciare quello acquisito.
Un breve esempio di codice è mostrato di seguito:

```
1  void acquisisci_entrambi(pthread_mutex_t lock_a,
       pthread_mutex_t lock_b) {
2    while (true) {
3      if (pthread_mutex_trylock(&lock_a) == 0) {
4        if (pthread_mutex_trylock(&lock_b) == 0) {
5          // Acquisiti entrambi con successo
6          return;
7        }
8
9        // Se siamo arrivati qui, b non è stato preso
10       // dunque viene rilasciato a
11       pthread_mutex_unlock(&lock_a);
12     }
13   }
14 }
```

Un altro metodo è quello di definire un ordinamento sui lock da acquisire per i vari thread. Ricordiamoci che deadlock $\Rightarrow$ ciclo esiste nel grafo. Se noi impostiamo dunque un

ordinamento che forza ogni thread ad acquisire il lock successivo, ordinando i lock in ordine crescente, otteniamo una struttura *lineare, non più ciclica*, con dunque l'impossibilità di creare un deadlock. Si tratta esattamente della strategia utilizzata per risolvere il problema dei filosofi a cena, dove abbiamo "rotto la simmetria" del tavolo circolare.